EVA-Bench: A New End-to-end Framework for Evaluating Voice Agents

Tara Bogavelli

ABSTRACT

Voice agents, artificial intelligence systems that conduct spoken conversations to complete tasks, are increasingly deployed across enterprise applications. However, no existing benchmark jointly addresses two core evaluation challenges: generating realistic simulated conversations, and measuring quality across the full scope of voice-specific failure modes. We present EVA-Bench*, an end-to-end evaluation framework that addresses both. On the simulation side, EVA-Bench orchestrates bot-to-bot audio conversations over dynamic multi-turn dialogues, with automatic simulation validation that detects user simulator error and appropriately regenerates conversations before scoring. On the measurement side, EVA-Bench introduces two composite metrics:* EVA-A *(Accuracy), capturing task completion, faithfulness, and audio-level speech fidelity; and* EVA-X *(Experience), capturing conversation progression, spoken conciseness, and turn-taking timing. Both metrics apply to all major agent architectures, enabling direct cross-architecture comparison. EVA-Bench includes 213 scenarios across three enterprise domains, a controlled perturbation suite for accent and noise robustness, and pass@1, pass@k, pass^k measurements that distinguish peak from reliable capability. Across 12 systems spanning all three architectures, we find: (1) no system simultaneously exceeds 0.5 on both* $\text{EVA-A}_{\text{pass@1}}$ *and* $\text{EVA-X}_{\text{pass@1}}$*; (2) peak and reliable performance diverge substantially (median pass@k–pass^k gap of 0.44 on EVA-A); and (3) accent and noise perturbations expose substantial robustness gaps, with effects varying across architectures, systems, and metrics (mean Δ up to 0.314). We release the full framework, evaluation suite, and benchmark data under an open-source license.*

1 Introduction

Voice agents are Artificial Intelligence (AI) systems designed to carry out tasks through spoken conversations, and their deployment across a wide range of applications is rapidly growing [?]. Voice agents operate under constraints that are fundamentally distinct from text: speech is ephemeral and linear, real-time timing shapes the naturalness of interaction, and acoustic conditions vary widely across callers. These properties give rise to failure modes with no direct text analog [? ?], and render evaluation frameworks designed for text-based agents [? ? ?] insufficient for assessing voice agent quality. Rigorous evaluation of voice agents must therefore address two distinct challenges: how conversations are

simulated, and how quality is measured.

The simulation challenges concern constructing interactions that are valid proxies for real deployment conditions. This requires complete multi-turn interactions rather than isolated exchanges - only full conversations expose how an agent recovers from misunderstandings, maintains context across turns, and resolves tasks end-to-end. Conversations must reflect the task-oriented nature of real voice agent deployments, user behavior must reflect natural human spoken dialogue, and acoustic conditions must reflect real-world environments, including variation in accents and background noise. Critically, simulated users must themselves be validated: a simulator that drifts from its assigned scenario, abandons realistic conversational behavior, or acts in ways no plausible human caller would, undermines the validity of any downstream evaluation. Finally, user simulators must behave consistently across repeated runs such that evaluation scores reflect agent behavior rather than simulator variance.

The measurement challenge concerns capturing the full scope of voice agent quality once valid simulations are in place. Task completion and turn-taking dynamics, while necessary, leave critical failure modes undetected [? ? ?]. On the accuracy side, an agent may call the correct tools yet violate system policy, comply with adversarial user requests, or produce spoken outputs containing incorrect entities (e.g. wrong confirmation codes, or monetary amounts) that are catastrophic in production yet undetectable from transcript-level evaluation alone. On the user experience side, an agent may achieve low response latency yet fail to make meaningful progress across turns, repeat prior questions, or present users with an excessive number of spoken options that would overwhelm a user's working memory. Addressing the measurement challenge requires evaluation across a broader set of dimensions than existing benchmarks provide. Additionally, voice agents are not architecturally uniform: cascade systems chain separate speech-to-text (STT), large language model (LLM), and text-to-speech (TTS) components, while audio-native systems process audio inputs directly — either end-to-end via speech-to-speech (S2S) models, or via hybrid systems that pair a large audio language model (LALM) with a TTS model (full definitions in Appendix A). These architectures have fundamentally different mechanisms, yet must be evaluated on equal footing for benchmarks to meaningfully compare them.

We present EVA-Bench, a benchmark designed to solve both of these challenges. On the simulation side, EVA-Bench conducts fully automated bot-to-bot audio simulation over dynamic multi-turn dialogues, with validation-gated quality control ensuring consistency across repeated trials. It includes three enterprise domains comprising 213 scenarios and a perturbation suite of controlled acoustic challenges to probe robustness beyond clean-condition baselines. On the measurement side, EVA-Bench introduces two composite scores: EVA-A (Accuracy) and EVA-X (Experience). EVA-A captures task completion, faithfulness to policy and tool outputs, and audio-level entity fidelity. EVA-X captures conversation progression, conciseness for spoken delivery, and turn-taking timing. Both scores are designed to apply directly to cascade and audio-native architectures, enabling direct comparison across system types. Across 12 evaluated systems, EVA-Bench reveals that accuracy and experience remain jointly unsatisfied across all architectures, that peak

capability consistently overstates reliable performance, and that robustness to acoustic perturbations varies substantially — and non-uniformly — across systems and metrics. Our contributions are listed below:

- We introduce EVA-Bench: an end-to-end evaluation framework for voice agents that generates realistic bot-to-bot conversations through a user simulator with validation-gated quality control, and supports controlled acoustic perturbations across independent trials.
- We define EVA-A and EVA-X, joint accuracy and experience metrics that surface failure modes invisible to existing benchmarks and enable direct comparison between audio-native and cascade voice agents.
- We create three enterprise benchmark datasets with a total of 213 scenarios focused on surfacing voice-specific failure modes.
- We show empirical findings on cascade vs. audio-native tradeoffs, perturbation sensitivity, and behavioral consistency across trials.

TABLE 1 Feature comparison of contemporary voice agent evaluation frameworks. ~ denotes partial support, and — for the simulator validation means it doesn't apply because of missing simulator. EVA-Bench is the only framework combining live multi-turn simulation across both speech-to-speech (S2S) and cascade architectures with a realistic audio environment, automated simulator validation, comprehensive metrics exposing a wide range of voice agent failures, and pass^k measurement via multi-trial consistency measurement.

Framework	Interaction	Supported Architectures	Multi-turn	Tool Use	Realistic Audio	Simulator Validation	Comprehensive Metrics	Multi Trial
EVA-Bench	Live bot-to-bot	S2S, Cascade	✓	✓	✓	✓	✓	✓
τ-Voice [?]	Live bot-to-bot	S2S	✓	✓	✓	✗	✗	✗
FDB-v3 [?]	Real human audio	S2S, Cascade	✗	✓	~	—	✗	✗
VoiceAgentBench [?]	Static, TTS-synthesized	S2S, Cascade	✓	✓	✗	—	✗	✗
CAVA [?]	Partial simulation	S2S, Cascade	✓	✓	✗	✗	✗	✗
FDB-v2 [?]	Live, auto examiner	S2S	✓	✗	✗	✗	✗	✗
FD-Bench [?]	Live, simulated	S2S	✗	✗	✗	✗	✗	✗

2 Related Work

Many existing voice benchmarks focus on individual components such as STT robustness [? ? ?], TTS quality [? ?], or conversational dynamics [? ?], rather than the end-to-end behavior of a voice agent. We organize the following discussion around the two challenges introduced above: the fidelity of multi-turn simulation and the comprehensiveness of voice agent quality measurement.

Conversation Simulation. Effective voice agent evaluation requires a simulation methodology that faithfully replicates the dynamic, real-time nature of spoken interaction, where the agent must navigate complete, task-oriented multi-turn conversations with live users whose requests and clarifications may shift throughout the call. Several benchmarks fall short on this requirement in distinct ways. FullDuplex-Bench-v1 (FDB) and FDB-v1.5 [? ?] assess conversation dynamics in a heavily-scripted manner without task completion or tool use, rendering themselves unsuitable for voice agent evaluations. VoiceAgentBench [?] evaluates multi-tool workflows but relies on static TTS-synthesized queries with no conversational back-and-forth. FDB-v3 [?] improves realism via authentic human recordings with disfluency annotation, yet remains single-turn. Both are further constrained by fixed interactions that limit generalization to unseen scenarios. τ-Voice [?] and FDB-v2 [?] represent the closest prior work in terms of live bot-to-bot simulation over multi-turn interactions. However, neither provides automated validation of simulator behavior across trials, leaving open the question of whether evaluation scores reflect agent quality or simulator variance. Furthermore, in τ-Voice, accent variation is coupled with changes in user persona and behavioral style, making it difficult to isolate the acoustic effect of accent from confounding behavioral differences. We address these gaps by introducing a live, multi-trial, bot-to-bot conversation simulator with a controlled perturbation suite and automatic user simulator quality validation.

Voice Agent Quality Measurement. Existing benchmarks that evaluate voice agent behavior converge on a narrow set of metrics. VoiceAgentBench [?] reports tool selection accuracy and structural consistency of tool invocations, but does not assess any dimension of conversational quality. τ-Voice [?] improved on this with a suite of turn-taking measures (response rate, latency, interruption rate, and selectivity) but does not assess whether the agent communicated faithfully or appropriately throughout the interaction. FDB-v3 [?] introduces a response quality dimension judged at the transcript level and latency decomposition, but does not assess policy faithfulness or accuracy of spoken entities at audio level. To the best of our knowledge, none of these frameworks measure whether the agent makes efficient progress, avoids imposing excessive cognitive load on the user, or speaks the correct information. Collectively, a substantial portion of voice agent quality remains unmeasured, particularly those most consequential for enterprise deployment.

3 Methodology

3.1 Conversation Simulation

Data Design. Constructing a benchmark dataset well-suited to voice agent evaluation requires careful attention to both domain relevance and scenario specificity. EVA-Bench comprises three domains reflecting real-world enterprise voice agent deployments: Airline Customer Service Management (CSM), Healthcare Human Resources Service Delivery (HRSD), and Enterprise Information Technology Service Management (ITSM). Scenarios within each domain are designed to reflect the task-oriented nature of real voice agent

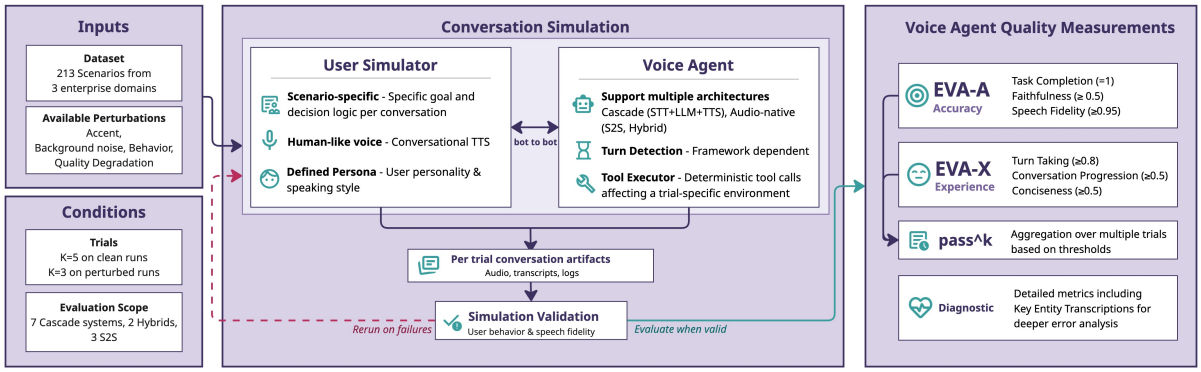


FIGURE 1 EVA-Bench framework overview. The simulation orchestrates parallel per-scenario bot-to-bot audio sessions over WebSocket in which the User Simulator — configured with a scenario-specific goal, persona, and conversational TTS voice — interacts with the Voice Agent under test. The Tool Executor handles all agent tool calls deterministically. Completed conversations pass through Simulator Validation that trigger automatic regeneration on failure before entering the Quality Measurements phase, which produces EVA-A and EVA-X pass@1, pass@k, and pass^k scores in addition to Diagnostic metrics.

interactions — focusing on high-contact cases where users are most likely to call an agent, such as flight rebooking rather than initial booking. Each scenario consists of a user goal specifying the user's intended outcome with explicit constraints (e.g., departure before 10pm, fare below a specified amount), a user persona defining speaking style, patience, and personality, a scenario database containing the data the agent's tools query and modify, and ground truth specifying the expected database state after successful task resolution. User goals are accompanied by a decision tree that eliminates ambiguity about intended outcomes and user choices throughout the conversation, enabling repeatable evaluation. Scenarios are further designed to surface voice-specific failure modes by requiring agents to correctly handle key entities (e.g. confirmation codes, employee identifiers (IDs), names, and domain-specific identifiers) that are frequently misheard in spoken interactions. More details on data domains, scenario examples, and dataset construction and validation can be found in Appendix C.

Multi-Turn Conversations. EVA-Bench evaluates agents through fully automated bot-to-bot conversations. A user simulator, built on a high-quality cascade pipeline, receives the user goal, decision tree, and persona as input and communicates with the agent over a live audio WebSocket. Both sides of the interaction operate over audio, enabling evaluation of cascade and audio-native architectures under identical conditions. See Appendix D for full simulator details.

Controlled Perturbations. EVA-Bench introduces a perturbation suite that varies user acoustic and behavioral conditions independently. Acoustic perturbations include accent variations, background noises, and connection degradation. Behavioral perturbations model caller variation in personality and speaking style. Each perturbation axis is independently controlled, enabling conditions to be applied in isolation or combination to disentangle each factor's effect on performance. See Appendix G.

Simulation Validation. Before any evaluation metrics are computed, each simulated conversation passes through automated validation checks. User Behavioral Fidelity (LLM-as-Judge [?]) checks whether the user simulator faithfully executed its assigned goal without deviations that would corrupt agent evaluation. The judge prompt contains specific corruption types to check for. User Speech Fidelity uses an LALM-as-Judge to verify that the simulator's spoken audio accurately conveyed its intended content, using a nearly identical prompt to the Speech Fidelity judge explained in 3.2.1. Conversations failing any check are automatically regenerated, ensuring that evaluation scores reflect agent behavior rather than simulator artifacts. Across four systems evaluated on all domains, 12.0% of trials required regeneration due to user simulator error (almost exclusively due to user behavioral drift), with speech fidelity accounting for less than 4% of reruns. Full validation details, including judge selection methodology and per-check rerun breakdowns, are provided in Appendix D.

3.2 Voice Agent Quality Measurement

EVA-Bench evaluates each conversation across three layered metric categories: Accuracy (EVA-A), Experience (EVA-X), and Diagnostic Metrics. These are described in the following subsections, and a table summarizing all metrics is provided in Appendix E. Note that for certain metrics, separate implementations are created for audio-native and cascade systems, since the two pipelines differ in which intermediate signals we can observe. See details in Appendices E.1 and E.2. Judge development followed a rigorous multi-stage development process described in Appendix E.3.

3.2.1 EVA-A: Accuracy Metrics

Task completion alone is a necessary but insufficient measure of accuracy. An agent can reach the correct end state while fabricating a policy detail, misreading a confirmation code aloud, or proceeding without required confirmations. Below are the metrics we propose to measure Accuracy.

Task Completion. A deterministic binary metric comparing the SHA-256 hash of the scenario database's final state against the ground-truth state. A score of 1 indicates the agent made exactly the correct tool calls with correct parameters; 0 indicates any deviation, i.e. wrong, missing, or extra changes. Because the user simulator produces repeatable outcomes, failures are unambiguously attributable to agent error.

Faithfulness. An LLM-as-Judge metric evaluating whether the agent actions remain grounded in the instructions, policies, tool results, and user inputs. This complements task completion: high task completion with low faithfulness indicates the task was completed but with material errors along the way (e.g., misrepresenting fees). Notably, the faithfulness prompt differs by architecture: cascade systems are evaluated relative to what the STT layer delivered, while audio-native systems treat mishearing as a faithfulness violation, since audio understanding is the model's own responsibility.

Speech Fidelity. A LALM-as-Judge metric evaluating whether the agent's spoken audio

accurately reproduces the intended text, with particular attention to high-stakes named entities (e.g. confirmation codes, dates, dollar amounts). For speech-to-speech systems where no intended text exists, the metric instead verifies that key entities from user turns and tool responses are correctly spoken. To our knowledge, this is the only metric in any end-to-end voice agent benchmark that evaluates the quality of the agent's spoken output at the audio level.

3.2.2 EVA-X: Experience Metrics

The quality of a conversational experience with a voice agent is shaped by several key factors: whether responses are concise enough to follow without replay, whether the conversation moves purposefully toward resolution, and whether the timing of the agent's replies feels natural.

Conversation Progression. An LLM-as-Judge metric that evaluates whether the agent efficiently moves the conversation forward by avoiding repetition, retaining context across turns, and driving toward task resolution without stalling or backtracking.

Conciseness. An LLM-as-Judge metric that evaluates whether the agent's responses are appropriately brief for spoken delivery. Phone callers cannot skim or re-read long responses; verbose agents fail users when they impose cognitive overload by providing too many details or questions.

Turn-Taking. A timestamp-based metric measuring whether the agent spoke at the right time, neither interrupting the user nor introducing excessive silence. Each turn is routed to a semantically appropriate scoring function: agent-interrupted turns are scored on overlap duration, barge-in count, and post-interrupt recovery latency; user-interruption turns on agent yield latency; and uninterrupted turns on a piecewise-linear latency curve. Turns involving tool calls receive a more lenient latency threshold, reflecting a longer expected duration than a purely conversational turn. This metric also takes into account when an agent fails to respond to a user turn (Conversation Completion).

3.2.3 Diagnostic Metrics

Diagnostic metrics are not included in EVA-A or EVA-X scores. Their purpose is to make main metric failures actionable by providing more granular information on key failure areas. For example, Transcription Accuracy (Key Entities) is an LLM-as-Judge diagnostic metric that identifies domain-specific key entities in user speech (confirmation codes, names, dates, IDs) and verifies whether each was correctly transcribed in cascade systems using semantic rather than exact match. This surfaces failures that word error rate (WER) misses entirely: a confirmation code off by one character scores near-perfect on WER but is functionally unusable. Additional diagnostic metrics cover authentication outcomes, response latency, and further diagnostic signaling (complete list provided in Appendix E.6).

3.2.4 Aggregate Metrics: pass@1, pass@k, and pass^k

Metrics for each dimension are aggregated into per-dimension scores (EVA-A, EVA-X), designed to capture both average and consistent performance. Measuring consistency

requires a binary notion of success per conversation, so that we can assess how often a system succeeds across repeated trials of the same scenario. Simple averaging is problematic for two reasons. First, averaging can mask a serious failure on one metric by a high score on another; we want to set a minimum acceptable bar for each component. Second, the metrics are not on comparable scales; Turn-Taking is continuous, LLM-as-Judge metrics use a three-point scale, and other metrics, like Speech Fidelity, are binary per-turn. The same conversation-level numerical score carries a different meaning across metrics.

We therefore define a *pass threshold* τ_m for each metric m, calibrated to the point at which performance is acceptable given the metric scale and implementation (Appendix E). A conversation passes on a dimension if *every* metric meets its threshold. Concretely, a conversation passes on accuracy if (task completion $= 1.0$) $\wedge$ (faithfulness ≥ 0.5) $\wedge$ (speech fidelity ≥ 0.95) and passes on experience if (turn-taking ≥ 0.8) $\wedge$ (conversation progression ≥ 0.5) $\wedge$ (conciseness ≥ 0.5).

This binary pass/fail gives us three aggregate statistics, each reported as EVA-A and EVA-X variants. pass@1 is the fraction of $T = Nk$ trials (N scenarios, k trials each) that pass, measuring average performance. pass@k is the fraction of scenarios where at least one of k trials passes, measuring ceiling performance. pass^k measures reliability, by raising each scenario's pass rate $\hat{p}_i$ to the k-th power and averaging across all N scenarios. This captures the probability that the system passes all k independent trials in a given scenario. The difference between pass@k and pass^k quantifies the gap between ceiling (peak) and consistent (reliable) performance. Formal definitions are provided in Appendix A.

4 Experiments & Empirical Analysis

4.1 Experiment Setup

We evaluate 12 systems in total: seven cascade, two hybrid, and three S2S. Configuration details are provided in Appendix B. Under the clean (unperturbed) condition, systems are evaluated on all 213 scenarios with $k = 5$ trials per scenario. We additionally evaluate under three perturbation conditions: French-accented user speech, coffee shop background noise, and both combined. To maintain feasibility across 12 systems and 3 conditions, perturbed evaluations use a randomly sampled subset of 90 scenarios (30 per domain) with $k = 3$ trials per scenario; the same subset is used across all systems. GPT-Realtime and Gemini Live models are evaluated using their native SDK [? ?], Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha is evaluated using ElevenAgents [?], and the remaining systems are evaluated using Pipecat [?]. All systems are evaluated using the respective framework's default turn detection configuration.

4.2 Evaluation Reliability

EVA-Bench scores reflect genuine differences in agent behavior rather than evaluation artifacts: judge stochasticity contributes minimally to observed variance, and score differences between systems exceed measurement noise. We substantiate this with two sets of

analyses: human-judge agreement to establish judge validity, and variance decomposition to complement bootstrap confidence intervals (CIs) in characterizing and bounding relative measurement noise.

Human-Judge Agreement We measure inter-annotator agreement (IAA) between the judge and/or human annotators using quadratic-weighted Cohen's κ. Our reference baseline is the IAA of two independent annotations from linguist labellers (IAA-L). For judge inter-annotator agreement (IAA-J), each human annotation is paired with the judge score and pooled, with 95% bootstrap confidence intervals (10,000 resamples) computed at the conversation level. The judge meets the practical human IAA ceiling on all four metrics, with IAA-J κ ranging from 0.777 to 0.845 across metrics. For every metric, Spearman's ρ and κ (both computed on the same judge–human ratings) differ by at most 0.008, supporting the absence of systematic calibration bias. Full scores, details, and confidence intervals shown in Table 14.

Variance Decomposition Observed metric scores for a given model within a domain reflect variance from three sources: scenario difficulty, trial stochasticity (conversation trajectories), and LLM judge stochasticity. We characterize the contribution of each source on a subset of cascade and S2S models through complementary analyses, demonstrating that our main findings reflect genuine differences in model behaviour rather than measurement noise. Trial stochasticity was the dominant source of variance across all metrics and models, consistently exceeding scenario-level variance; judge stochasticity was minimal by comparison (permutation test, $p < 0.0001$ for all 16 model $\times$ metric combinations). For scenario variance, task completion and faithfulness showed the highest sensitivity, consistent with their observed dependence on intrinsic scenario difficulty and policy complexity. We also found that scenario difficulty rank is not shared uniformly across models but reflects model-specific response patterns (two-way random effects ICC, model $\times$ scenario interaction $p < 0.01$ and 4-18% of total variance across evaluated domain $\times$ metric combinations). Full variance decomposition results are reported in Appendix H.

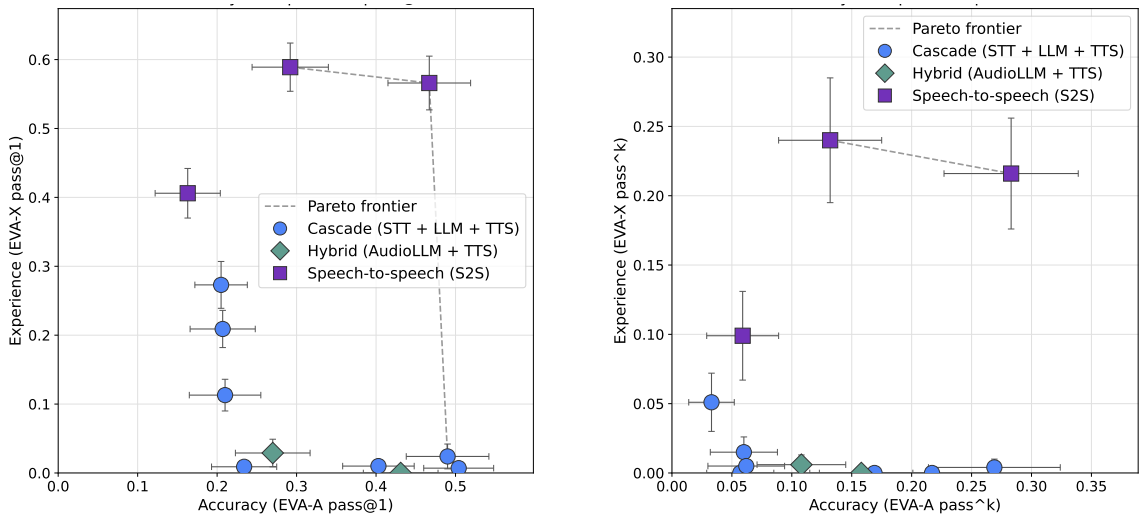
4.3 Main Findings

Accuracy–Experience Frontier

No evaluated system clears 0.5 on both EVA-A pass@1 and EVA-X pass@1 jointly, and only GPT-Realtime-1.5 (0.47, 0.57) clears 0.4 on both. This sparsity is reflected in the pass@1 Pareto frontier (Figure 2a), which contains four systems spanning two disjoint regions.

Within EVA-X, the gap is almost entirely driven by Turn-Taking, where mean scores stratify cleanly by architecture (cascade: 0.28–0.58; S2S: 0.82–0.83) (Table 2). Conciseness and Conversation Progression show no comparable separation. The two hybrid systems both fall within the cascade EVA-X range (0.000, 0.029), suggesting that hybrid systems may not inherit the latency advantages of fully speech-to-speech architectures, though more systems would be needed to confirm this.

Among the evaluated cascade systems, we observe a consistent accuracy–experience trade-off, pointing to a potential underlying capability–latency tension. The three cascade



(a) pass@1 (b) pass^k

FIGURE 2 Accuracy vs Experience overview. Accuracy and Experience scores for pass@1 and pass^k , mean ± 95% CIs across domains. On the pass@1 plot, four systems are on the Pareto frontier, two S2S and two cascade: Gemini-3.1-Flash-Live, GPT-Realtime-1.5, Scribe – Gemini-3-Flash – Conversational v3, and Nova – GPT-5.4 – Sonic (left to right). On the pass^k plot, only the two S2S systems are on the frontier (note axis range differences).

systems that perform best on accuracy (Nova – GPT-5.4 – Sonic, Scribe – Gemini-3-Flash – Conversational v3, Parakeet – Gemma-4-31B – Kokoro) struggle on experience, with mean latencies on tool-call turns above 5 s. In contrast, the two cascade systems with better experience (Whisper – Qwen3.5-27B – Voxtral, Cohere – Gemma-4-26B – Voxtral) achieve tool-call turn latencies below 2.7 s but also lower accuracy. No cascade system exceeds 0.25 on both dimensions, with no overlapping CIs. Full latency breakdowns appear in Table 25.

Consistency Analysis Across all 12 evaluated systems, peak performance (pass@k) substantially exceeds reliable performance (pass^k) on both axes: the median gap is 0.44 on EVA-A and 0.24 on EVA-X. Under the pass^k interpretation (the probability of passing all five trials on a given scenario) even the strongest systems fall well below peak, suggesting that single-trial scores systematically overstate deployment-grade reliability.

Robustness Analysis. We evaluate each system under three perturbation conditions – accented speech, background noise, and both combined – measuring performance degradation against a clean baseline across the 90 subsampled scenarios (see subsection 4.1) via paired sign-flip permutation tests (n=90, Holm–Bonferroni corrected). The two architectures diverge in their failure modes: cascade systems are most vulnerable on accuracy metrics, while S2S systems suffer most on experience metrics. Accented speech drives the largest accuracy failures: cascade task completion drops by a mean 10 points (worst system: 17 points), while no S2S model shows significant accuracy degradation (0/27 model-metric pairs). Background noise exposes S2S experience failures (EVA-X $\bar{\Delta}$=−0.16), though cascade accuracy also suffers under noise (task completion $\bar{\Delta}$=−0.10 vs. S2S $\bar{\Delta}$=−0.04). The combined condition reveals the full spread: cascade task completion drops a mean 19

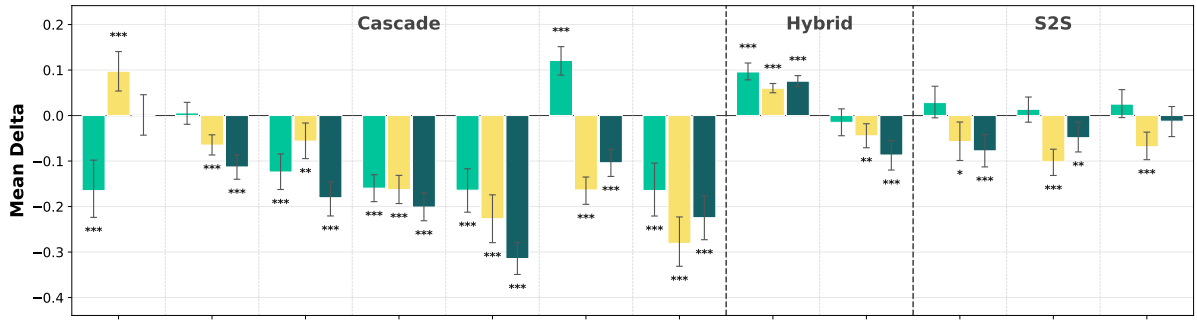


FIGURE 3 Perturbation effect on Turn-Taking pooled across domains. Bars show the mean delta from clean trials; 95% bootstrap (CIs) on the per-scenario delta. Bar colors encode the perturbation condition: ■ accent, ■ background noise, ■ accent + background noise. Asterisks mark cells significant after Holm-Bonferroni correction (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$). Models listed in Appendix G.

points (worst systems: 31 points), while the S2S model mean remains within 5 points.

Within the cascade class, however, robustness varies considerably, with as low as 11% and as high as 87% of per-system metric-perturbation combinations showing significant degradation. The two most robust cascade systems degrade primarily on experience metrics, more closely resembling S2S systems than their cascade peers. Turn-taking is the most perturbation-sensitive metric overall, with 81% of measurements across perturbations and systems showing significant degradation. See Figure 3 and Appendix G for full results.

4.4 Failure Mode Analysis

EVA-Bench's metrics enable detailed error analysis. We report four targeted analyses below, with additional analyses in Appendix F.

Named entity transcription as an accuracy bottleneck. EVA-Bench's diagnostic metrics reveal a consistent association between Transcription Accuracy (Key Entities) and Task Completion across all three domains, pointing to transcription as a candidate bottleneck for cascade system performance. Across seven cascade systems, mean transcription accuracy on key entities is strongly correlated with mean task completion (Pearson $r = 0.93$, $p = 0.002$) and the relationship holds within each domain ($r = 0.88$–0.93, all $p < 0.01$). Cascade systems with transcription accuracy below 70% on key entities show task completion rates 39% lower than systems above this threshold (0.37 vs 0.60), with a consistent drop in each domain (ITSM: −41%, HR: −41%, CSM: −34%). See Figure 6 for the correlation plot and Figure 5 for Transcription Accuracy (Key Entities) scores.

Evaluated S2S systems show contrasting patterns relative to other systems. The higher EVA-X scores for S2S systems reflect a clear turn-taking lead with on-time rate and conversation completion up +27.9 pp and +15.2 pp on average respectively, though the strongest cascade systems come within a few percentage points. The conversation completion gap is partly explained by many cascade systems completely failing to respond to short utterances and spelled content. S2S systems also show policy adherence issues, violating stated policy

more often on average (−24.6 pp) and trailing the strongest cascade systems on this aspect. Confidence intervals, per-system breakdowns, and additional details are in Appendix F.3.

Faithfulness failures are not predicted by task completion. 72.2% of conversations ($n = 12,780$) with task completion = 1 exhibit at least one faithfulness deviation (faithfulness < 1.0), indicating that agents frequently make policy deviations or hallucinate details even when they call the correct tools. Conversely, 50.5% of faithfulness deviations co-occur with task completion = 0, suggesting that some task failures are downstream of faithfulness violations. This decoupling motivates including faithfulness as an independent metric. See Appendix F.2 for full confusion matrix.

Speech fidelity failures concentrate on alphanumeric entities. Entity errors (letter substitutions, digit omissions, spurious insertions, and phonetic confusions between similar-sounding characters) are the dominant speech fidelity failure mode across all evaluated models. This motivates including speech fidelity as an audio-level metric: a caller who receives a misarticulated confirmation code cannot detect the error from context alone, and even 1% per-turn fail rates represent a non-trivial error probability over multi-turn interactions. See Appendix F.4 for examples of failures.

5 Conclusion

We presented EVA-Bench, an end-to-end evaluation framework for voice agents that jointly addresses simulation fidelity and measurement comprehensiveness. Together, validation-gated bot-to-bot simulation, architecture-agnostic composite metrics (EVA-A and EVA-X), and a multi-trial consistency framework (pass@1, pass@k, pass^k) enable comparison across cascade, hybrid, and S2S voice agents under identical conditions.

Our evaluation of 12 systems produces three central findings. First, while the best-performing cascade and S2S systems achieve comparable accuracy, experience quality diverges sharply along architecture lines, with the S2S–cascade gap on EVA-X driven almost entirely by turn-taking. Second, peak and reliable performance diverge substantially across all systems: the median pass@k–pass^k gap is 0.44 on EVA-A, indicating that single-trial evaluation scores systematically overstate deployment-grade quality regardless of architecture. Third, we observe that cascade and S2S systems degrade asymmetrically under acoustic perturbation: accent variation degrades the cascade task completion by an average of 10 points while leaving S2S accuracy unchanged, whereas background noise degrades experience metrics for almost all systems. These observations highlight dimensions of voice agent quality that existing benchmarks leave unmeasured: joint accuracy-experience measurement, peak-versus-reliable performance gaps, and perturbation-specific degradation patterns across voice agent architectures.

EVA-Bench is released as a fully open-source, extensible framework. The metric suite is modular, the simulation interface supports cascade and audio-native architectures, and new domains can be added by configuring agents and scenarios. We invite the community to extend the benchmark with additional domains, languages, and evaluation dimensions as the field evolves.

Limitations

While EVA-Bench is designed to provide a rigorous, extensible end-to-end evaluation framework of conversational voice agents, several limitations are important to acknowledge across the framework.

MetricsLLM-based judge models are known to exhibit stylistic biases and may systematically favor outputs that resemble their own training distribution. When the evaluated system and the judge model share the same model family, the risk of in-family preference artifacts is non-trivial and applies directly to specific evaluated systems: GPT-5.4 and GPT-5.4-mini are evaluated using GPT-5.2-based judges for Conciseness and Conversation Progression, and Claude Haiku 4.5 is evaluated using Claude Opus 4.6 for Faithfulness. Scores for these systems should be interpreted with this potential bias in mind. LALM-based judges, in particular, remain an emerging evaluation paradigm and demonstrate lower reliability compared to text-only judge models under equivalent conditions. Furthermore, the current scoring scheme applies binary task completion judgments, awarding no partial credit to agents that satisfy all but one sub-goal in a multi-step task. This all-or-nothing formulation may suppress fine-grained discriminability between systems that differ primarily in robustness on terminal sub-goals. Finally, EVA-Bench does not currently assess whether agents produce harmful outputs or inadvertently expose sensitive caller information such as personally identifiable information (PII) — a notable gap for evaluating production-readiness in high-stakes domains.

FrameworkA central assumption of EVA-Bench is that bot-to-bot simulation with a validated user simulator constitutes a valid proxy for real human caller interactions. If the simulator systematically differs from real callers in ways that the behavioral fidelity validator does not detect — for example, in how it handles ambiguity, expresses frustration, or recovers from misunderstandings — evaluation scores may not transfer to production conditions, and systems could implicitly optimize for simulator-specific speech patterns rather than genuine robustness. PCM-to-μ-law (Pulse-Code Modulation) audio conversion introduces quality degradation. Bot-to-bot audio interface timing may not fully represent production deployments. Inaccurate pipeline event timing (Voice Activity Detection events, etc.) from differing sources may also lead to imperfect response speed values and timestamps. Log reconciliation between various systems can also have inaccuracies due to imprecise timestamps. Full reproduction requires access to commercial model APIs. Generating $k = 5$ validated trials per scenario across multiple system configurations carries non-trivial cost; at current API pricing, a full evaluation run across all 213 scenarios and 12 systems requires on the order of several hundred API calls per trial, and costs scale linearly with the number of systems and scenarios evaluated. Additionally, EVA-Bench evaluates single-agent configurations in which the agent has direct access to a flat tool list for its domain — supervisor-worker patterns, multi-agent orchestration, and agentic frameworks involving planning or delegation are not currently within scope, limiting applicability to more complex real-world deployments. Finally, latency measurements (which manifest

in turn-taking and response speed metrics) will vary depending on APIs, deployments, and hardware, potentially leading to variation in EVA-X results within the same system. All agent tools are simulated via a declarative mock executor rather than live API calls; while tool schemas are designed to reflect real-world APIs, mock execution cannot capture failure modes, latency variance, partial responses, or schema drift that arise in production integrations. And, real-world voice agent deployments often employ latency reduction strategies — such as speculative tool execution, response pre-fetching, or streaming-aware scheduling — that are absent in EVA-Bench's evaluation setup; reported latency metrics may therefore not reflect the lower bounds achievable in optimized production systems.

SimulationAll scenarios are in English — no multilingual coverage. As our user simulator relies on a commercial system, its behavior may change across versions. The simulator is built on a high-quality STT-LLM-TTS pipeline (scribe-v2.2-realtime – GPT-5.1 – ElevenLabs v3 Conversational), which produces unusually clean, well-formed speech relative to real callers. This means the benchmark under-represents the natural disfluencies, hesitations, and emotional variation exhibited by real callers; systems may score higher than they would under genuine human caller conditions, and performance differences between systems may not fully reflect their relative robustness to real-world speech variation. The simulator does not systematically generate interruption behaviors — a common and consequential real-caller pattern — meaning EVA-Bench under-stresses turn-taking robustness and does not differentiate systems that handle barge-in gracefully from those that do not. The simulator may also occasionally go off policy; while we employ validators to detect such cases, perfect adherence cannot be guaranteed, particularly on subjective validator metrics.

ExperimentsOur evaluation covers 12 systems across three architectural classes. While we observe cross-architecture differences, our sample size precludes inferential claims about model classes broadly. All frameworks (ElevenAgents, Pipecat, OpenAI Realtime, Gemini Live) use default turn detection settings; we deliberately avoided parameter tuning to preserve a fair baseline across frameworks. Similarly, agent system prompts were not optimized for performance — we constructed prompts to convey necessary guidelines and policies without correcting for model-specific errors. We note that targeted system parameter tuning and prompt engineering would likely yield higher scores than those reported here. While the framework allows for a wide range of background noises, accents, and behavioral personas, our experiments include only one accent (French) and one noise environment (coffee shop), each instantiated with a single specific voice (one female, one male); observed robustness differences between cascade and S2S systems under accent perturbation may partly reflect properties of that particular voice rather than accent variation broadly, and results may not generalize to other accents, speakers, or noise environments. Also, the adversarial scenarios are hand-designed around specific policy boundary conditions; coverage of difficult cases within each domain therefore reflects our design choices rather than a systematic sampling of the difficulty distribution, and systems optimized against known failure modes may score well without generalizing to unseen

policy boundaries.

Ethics Statement

Our work focuses on responsible development and evaluation of voice agent systems. All evaluation scenarios in EVA-Bench are fully synthetic — no real caller data, recordings, or personally identifiable information were used in dataset construction or evaluation. Scenario content was generated and manually reviewed to avoid reproducing proprietary materials, and no human subjects were involved in any part of the evaluation pipeline. Also, all simulation tools respect the copyright and privacy guidelines. Although EVA-Bench enables large-scale evaluation of voice agents, we cannot guarantee that models evaluated through it will not generate harmful or biased output during evaluation runs. Researchers and practitioners are strongly encouraged to implement appropriate content filtering and bias detection before deploying evaluated systems in production environments. Additionally, we acknowledge that our current domain coverage is limited, and the simulation is conducted exclusively in English, which may inadvertently reinforce existing representational biases in audio AI systems. We encourage the community to expand EVA-Bench with more diverse languages, accents, cultural contexts, and domains. Regarding the use of language models in manuscript preparation, we utilize LLM assistance solely to refine language for clarity and correctness. No substantive content, experimental claims, or analytical conclusions were generated by LLM tools.

Reproducibility Statement

We are committed to full reproducibility of our evaluation framework and experimental results. All EVA-Bench code, configuration files, evaluation scripts, scenario data, and documentation are publicly released as open-source under an anonymized repository included with the submission. The repository includes setup instructions, environment specifications, and scripts to reproduce all reported evaluations. We provide comprehensive implementation details including all model configurations (Appendix B), data distributions across domains (Appendix C), judge prompts (Appendix M), and metric definitions and thresholds (Appendix E). All LLM-as-judge configurations are fully specified to enable result replication across research groups. We note that full reproduction of the reported scores requires access to commercial model APIs; results may vary across API versions and deployment configurations, as discussed in Section 5.

Acknowledgment

We thank the following individuals for their careful data review and thoughtful contributions to EVA-Bench: Akshay Kalkunte, Jishnu Nair, and Aman Tiwari. We also thank our linguists for their data review and labeling, and help with creating judge datasets: Ryan

Dux, Anne Heaton-Dunlap, Tiffany Do, Maria Kossenko, Keerthana Gopinathan, Nidhi Kumari, and Ranjani Iyer.

A Definitions & Key Terms

STT *Speech-to-Text.* A model or service that transcribes spoken audio into text. In voice agent pipelines, STT serves as the input stage, converting user utterances into a form consumable by a downstream language model.

TTS *Text-to-Speech.* A model or service that synthesizes natural-sounding speech from text. In voice agent pipelines, TTS serves as the output stage, rendering language model responses as audio delivered to the user.

S2S *Speech-to-Speech.* An end-to-end model architecture that accepts raw audio as input and produces audio as output, bypassing discrete STT and TTS stages entirely. S2S systems may also emit text output, used to invoke tool calls, alongside or prior to synthesizing a spoken response. S2S systems minimize modality-crossing latency and preserve paralinguistic signals throughout the pipeline.

LALM A large language model that accepts audio as a direct input modality—without prior transcription—and produces text output. AudioLLMs internalize acoustic features (prosody, speaker characteristics, noise) that would otherwise be lost in an STT transcription step. Also referred to as an AudioLLM or a SpeechLLM.

Hybrid A voice agent architecture that combines an AudioLLM (for audio-aware language understanding) with a discrete TTS module (for speech synthesis). Hybrid systems are Audio-Native on the input side while retaining a text-to-speech output stage, and occupy a middle ground between fully cascaded (STT+LLM+TTS) and fully end-to-end (S2S) pipelines.

Audio-Native An umbrella term for voice agent architectures that process audio directly at one or more stages, rather than relying solely on text-based STT→LLM→TTS cascades. Audio-Native systems include both *S2S* and *Hybrid* configurations.

Multi-turn A conversational interaction comprising more than one exchange between user and agent—i.e., at least one agent response followed by a subsequent user utterance within the same session. Multi-turn evaluation tests an agent's ability to maintain task state, handle clarifications, and recover from errors across dialogue turns.

Bot-to-Bot An automated evaluation protocol in which a *user simulator* (a separate agent or model) plays the role of the human caller, conducting full spoken conversations with the system under evaluation.

pass@1 The fraction of all trials that pass, measuring average performance: pass@1 = $\frac{1}{T}\sum_{t=1}^{T} 1[\text{pass}_t^{(d)}]$, where $T = Nk$ is the total number of trials across N scenarios

and k trials each.

pass@k The fraction of scenarios where at least one of k trials passes, measuring ceiling performance: pass@k $= \frac{1}{N}\sum_{i=1}^{N} 1\left[\sum_{j=1}^{k} 1[\text{pass}_{i,j}^{(d)}] \geq 1\right]$. See also: *Peak*.

pass^k The expected probability that a system passes all k independent future trials on a given scenario, measuring reliable performance: pass^k $= \frac{1}{N}\sum_{i=1}^{N} \hat{p}_i^k$, where $\hat{p}_i = \frac{1}{k}\sum_{j=1}^{k} 1[\text{pass}_{i,j}^{(d)}]$ is the per-scenario pass rate across k trials. See also: *Reliable*.

Peak Generally refers to *pass@k* scores. A peak score characterizes the best-case performance of a system over k independent attempts—i.e., the probability that at least one of k runs produces a correct outcome. Peak scores are informative about a system's ceiling capability but do not reflect consistency.

Reliable Generally refers to *pass^k* scores. A reliability score characterizes the probability that a system produces a correct outcome on *every one* of k independent attempts. Reliability scores penalize variance and are informative about how consistently a system can be expected to succeed across repeated deployments.

TABLE 2 Accuracy and Experience metrics for all evaluated systems under clean-audio conditions, pooled equal-weighted across the three EVA domains. Each cell shows the pooled point estimate ± the percentile bootstrap CI half-width ($\alpha = 0.05$). The three pass-rate columns share a single shading scale so they can be visually compared; each submetric column is scaled independently. Darker = higher point estimate.

		EVA-A			Task Completion	Faithfulness	Speech Fidelity
Arch.	System	pass@1	pass@k	pass^k	Mean	Mean	Mean
Cascade	Cohere – Gemma-4-26B – Voxtral	0.207 ±0.041	0.416 ±0.070	0.060 ±0.028	0.338 ±0.049	0.375 ±0.036	0.983 ±0.003
	Scribe – Gemini-3-Flash – Conversational v3	0.490 ±0.052	0.730 ±0.058	0.269 ±0.055	0.736 ±0.043	0.457 ±0.042	0.977 ±0.006
	Ink-whisper – Haiku-4.5 – Sonic 3	0.234 ±0.041	0.516 ±0.069	0.057 ±0.028	0.374 ±0.044	0.518 ±0.033	0.983 ±0.003
	Nova-3 – GPT-5.4 – Sonic 3	0.504 ±0.044	0.809 ±0.048	0.217 ±0.048	0.609 ±0.043	0.754 ±0.027	0.989 ±0.003
	Nova-3 – GPT-5.4-mini – Aura-2	0.210 ±0.045	0.448 ±0.069	0.062 ±0.032	0.465 ±0.050	0.270 ±0.033	0.974 ±0.005
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.403 ±0.045	0.748 ±0.055	0.169 ±0.046	0.637 ±0.051	0.466 ±0.035	0.954 ±0.009
	Whisper – Qwen3.5-27B – Voxtral	0.205 ±0.033	0.518 ±0.066	0.033 ±0.019	0.417 ±0.051	0.546 ±0.033	0.913 ±0.010
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.431 ±0.047	0.812 ±0.055	0.158 ±0.043	0.674 ±0.041	0.443 ±0.036	0.969 ±0.006
	Ultravox-Realtime	0.270 ±0.047	0.503 ±0.072	0.108 ±0.037	0.473 ±0.055	0.292 ±0.035	0.971 ±0.007
S2S	Gemini-3.1-Flash-Live	0.292 ±0.048	0.552 ±0.069	0.132 ±0.043	0.473 ±0.052	0.238 ±0.035	0.995 ±0.003
	GPT-Realtime-1.5	0.467 ±0.052	0.710 ±0.061	0.283 ±0.056	0.739 ±0.046	0.360 ±0.041	0.996 ±0.002
	GPT-Realtime-mini	0.163 ±0.041	0.318 ±0.063	0.059 ±0.030	0.345 ±0.054	0.125 ±0.031	0.977 ±0.012

		EVA-X			Turn-Taking	Conciseness	Conv. Progression
Arch.	System	pass@1	pass@k	pass^k	Mean	Mean	Mean
Cascade	Cohere – Gemma-4-26B – Voxtral	0.209 ±0.027	0.647 ±0.069	0.015 ±0.011	0.567 ±0.024	0.809 ±0.007	0.598 ±0.032
	Scribe – Gemini-3-Flash – Conversational v3	0.024 ±0.018	0.061 ±0.035	0.004 ±0.006	0.451 ±0.019	0.774 ±0.007	0.804 ±0.023
	Ink-whisper – Haiku-4.5 – Sonic 3	0.009 ±0.006	0.042 ±0.031	0.000 ±0.000	0.312 ±0.020	0.784 ±0.007	0.710 ±0.023
	Nova-3 – GPT-5.4 – Sonic 3	0.007 ±0.006	0.031 ±0.024	0.000 ±0.000	0.283 ±0.019	0.835 ±0.007	0.737 ±0.020
	Nova-3 – GPT-5.4-mini – Aura-2	0.113 ±0.023	0.416 ±0.070	0.005 ±0.004	0.583 ±0.019	0.835 ±0.008	0.428 ±0.025
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.010 ±0.009	0.035 ±0.032	0.000 ±0.000	0.308 ±0.015	0.829 ±0.007	0.774 ±0.024
	Whisper – Qwen3.5-27B – Voxtral	0.273 ±0.034	0.684 ±0.065	0.051 ±0.021	0.561 ±0.029	0.685 ±0.010	0.612 ±0.026
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000	0.019 ±0.003	0.801 ±0.007	0.618 ±0.029
	Ultravox-Realtime	0.029 ±0.020	0.081 ±0.039	0.006 ±0.007	0.417 ±0.020	0.750 ±0.010	0.429 ±0.030
S2S	Gemini-3.1-Flash-Live	0.589 ±0.035	0.979 ±0.021	0.240 ±0.045	0.830 ±0.017	0.801 ±0.009	0.636 ±0.029
	GPT-Realtime-1.5	0.566 ±0.039	0.939 ±0.034	0.216 ±0.040	0.815 ±0.013	0.801 ±0.008	0.679 ±0.024
	GPT-Realtime-mini	0.406 ±0.036	0.893 ±0.043	0.099 ±0.032	0.818 ±0.015	0.722 ±0.009	0.388 ±0.033

B Experiment Parameters

Below are the turn detection and model configurations for all evaluated and judge models; the user simulator is detailed separately in Appendix D.

B.1 Self-Hosted Models

All self-hosted models were served on NVIDIA H100 GPUs. Models served via vLLM used `vllm-openai v0.19.0`. Table 3 lists the hardware and serving configurations.

Gemma-4-26B and Gemma-4-31B were called with `temperature=1.0`, `top_p=0.95`, `top_k=64`, and `max_tokens=12000`. Thinking mode was disabled via `enable_thinking=false` and special tokens were preserved (`skip_special_tokens=false`).

Qwen3.5-27B was called with `temperature=1.0`, `top_p=0.95`, `top_k=20`, `min_p=0.0`, `presence_penalty=1.5`, and `repetition_penalty=1.0`. Thinking mode was likewise disabled via `enable_thinking=false`.

TABLE 3 Self-hosted model configurations.

Model	Model Abrv	Model ID	Type	GPU	CPU	Precision	Deployment
Gemma-4-26B	Gemma-4-26B	google/gemma-4-26B-A4B-it	LLM	2× H100	–	BF16	vLLM
Gemma-4-31B	Gemma-4-31B	google/gemma-4-31B-it	LLM	2× H100	–	BF16	vLLM
Qwen3.5-27B	Qwen3.5-27B	Qwen/Qwen3.5-27B	LLM	4× H100	8× 8GB	BF16	vLLM
Parakeet-1.1	Parakeet	nvidia/parakeet-ctc-1.1b	STT	1× H100	–	BF16	Nvidia NIM
Whisper-Large-v3	Whisper	openai/whisper-large-v3	STT	1× H100	4× 64GB	FP16	vLLM
Cohere-transcribe	Cohere	CohereLabs/cohere-transcribe-03-2026	STT	1× H100	8× 64GB	BF16	vLLM
Kokoro	Kokoro	hexgrad/Kokoro-82M	TTS	1× H100	8× 32GB	FP32	Remsky Kokoro
Voxtral-4B-TTS	Voxtral	mistralai/Voxtral-4B-TTS-2603	TTS	1× H100	–	BF16	vLLM

B.2 API-Hosted Models

For ElevenLabs, we used ElevenAgents with the following models: Scribe-v2.2-Realtime, Gemini-3-Flash, and TTS-Conversational-v3-Alpha. We used the default agent parameters, listed in Table 4.

Table 5 lists all the other API-hosted models.

B.3 Turn Detection Configurations

We use the default turn detection configurations for each framework in our experiments. Each framework offers varying levels of configurability, making it difficult to standardize exact parameters and turn strategies across evaluations.

Pipecat. The default start strategy uses VAD (voice activity detection) or transcription receipt to determine when the user begins speaking, and the stop strategy uses AI-powered turn detection via `LocalSmartTurnAnalyzerV3` to determine when the user finishes speaking.[1]

[1] https://docs.pipecat.ai/api-reference/server/utilities/turn-management/user-turn-strategies#base-parameters-2

TABLE 4 ElevenLabs ElevenAgents parameters.

Component	Parameter	Value
STT	Filter background speech	disabled
TTS	Expressive mode	enabled
TTS	Voice	Lauren B - Friendly & Engaging Customer Care Agent
TTS	Voice ID	3liN8q8YoeB9Hk6AboKe
LLM	Temperature	0
	Reasoning effort	minimal
	Limit token usage	–1
	Parallel tool calling	disabled
	Cascade timeout	8 s
Tools	Wait for response	enabled
	Pre-tool speech	auto
	Execution mode	immediate
	Tool call sound	none
	Response timeout	20 s
Agent	Eagerness	normal
	Spelling patience	auto
	Speculative turn	enabled
	Re-transcribe on timeout	disabled
	Take turn after silence	15 s
	End call after silence	disabled
	Max conversation duration	600 s

OpenAI Realtime. We use the default server VAD, which uses periods of silence to detect turn boundaries. Default values are used for `threshold`, `prefix_padding_ms`, and `silence_duration`.[2]

ElevenAgents. The turn "eagerness" parameter is left at its default setting (`normal`).[3]

Gemini Live. We use the default automatic VAD provided.[4]

EVA-Bench makes turn detection parameters and options configurable via the CLI, so practitioners can run experiments using the turn detection settings available to their chosen framework. The only exception is ElevenAgents, where users must register and configure their agents separately prior to evaluation.

[2] https://developers.openai.com/api/docs/guides/realtime-vad
[3] https://elevenlabs.io/docs/eleven-agents/customization/conversation-flow#turn-eagerness
[4] https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-reference/multimodal-live

B.4 Experiments Compute Resources

Running EVA-Bench experiments involves costs across three distinct components of the pipeline. First, the user simulator is powered by ElevenLabs' ElevenAgents (Appendix D), a fully hosted conversational AI system, incurring costs per interaction across all simulated dialogue turns. Second, the agent under evaluation introduces inference costs whenever the model is proprietary. Closed-source models are billed per token for LLM inference, and per second of audio for STT and per character for TTS when the agent operates as a full voice pipeline. Note that a conversation will on average last 4 to 5 minutes. Third, the LLM-as-judge component adds a separate layer of inference cost, as 8 of the evaluated metrics rely on model-based scoring computed per sample. We evaluated 213 scenarios over 5 trials on clean data and 90 scenarios over 3 trials per perturbation across 3 perturbation types, yielding $213 \times 5 + 90 \times 3 \times 3 = 1{,}875$ evaluation samples per model. Costs are further compounded by re-runs triggered when a simulation fails validation gates. Across four representative systems evaluated on all domains, 24.1% of trials required regeneration (roughly half due to simulator error, the other half due to infrastructure failures or timeouts), effectively inflating the true number of simulations and judge calls to approximately $1{,}875 \times 1.241 \approx 2{,}327$ per system evaluated, on average.

Researchers should therefore treat full EVA-Bench runs as non-trivial compute investments, particularly when benchmarking multiple proprietary models simultaneously, and plan accordingly for parallelization and cost budgeting prior to large-scale experiments.

TABLE 5 API-hosted model configurations. In this work, due to space constraints we occasionally use the following model abbreviations: Ink-whisper→ Ink, Sonic 3→ Sonic, Aura-2→ Aura, Ultravox-Realtime→ Ultravox. Note that Ultravox-Realtime is offered as a realtime model service in Pipecat.

Model	Provider	Type	Model ID	Parameters
GPT-5.2	OpenAI	LLM	`gpt-5.2`	reasoning: default
GPT-5.4	OpenAI	LLM	`gpt-5.4`	reasoning: default
GPT-5.4-mini	OpenAI	LLM	`gpt-5.4-mini`	reasoning: default
GPT-Realtime-1.5	OpenAI	S2S	`gpt-realtime-1.5`	voice: Marin
GPT-Realtime-mini	OpenAI	S2S	`gpt-realtime-mini`	voice: Marin
Gemini-3-Flash	Google	LLM	`gemini-3-flash-preview`	reasoning: default
Gemini-3.1-Flash-Live	Google	LALM	`gemini-3.1-flash-live-preview`	voice: Leda
Gemini-3.1-Flash-TTS	Google	TTS	`gemini-3.1-flash-tts-preview`	voice: provider default
Claude Opus 4.6	AWS Bedrock	LLM	`us.anthropic.claude-opus-4-6-v1`	reasoning: default
Haiku-4.5	AWS Bedrock	LLM	`us.anthropic.claude-haiku-4-5-20251001-v1:0`	–
Ultravox-Realtime	Ultravox	LALM	–	–
Ink-whisper	Cartesia	STT	`ink-whisper`	–
Sonic 3	Cartesia	TTS	`sonic-3`	–
Nova-3	Deepgram	STT	`nova-3`	–
Aura-2	Deepgram	TTS	`aura-2-helena-en`	–

C Data

We created three datasets on different enterprise domains, each selected to target a distinct axis of difficulty for voice agents. All three require accurate transcription of structured named entities over voice (e.g., confirmation codes and employee identifiers), but differ in their primary challenge. Airline Customer Service Management (CSM) tests temporal reasoning and complex policy adherence in high-stakes flight rebooking scenarios. Healthcare Human Resources Service Delivery (HRSD) stresses entity density, requiring callers to communicate multiple registration and license numbers across clinical and administrative HR workflows. Enterprise Information Technology Service Management (ITSM) introduces branching conversational flows (e.g., incident resolution attempts must fail before ticket escalation is permitted) and tiered authentication reflecting the access sensitivity of different workflows. Dataset statistics are summarized in Table 6.

Within each domain, scenarios span three dimensions: Single-Intent (one workflow per call), Multi-Intent (one to four concurrent workflows, testing compositional task completion without context loss), and Adversarial (hard policy constraints under social pressure, e.g., refusing compensation to an ineligible caller).

Section C.1 introduces the workflows for the three data domains, and more details on each are provided in Appendix I, with scenario examples (one for each domain, and a single-intent, a multi-intent, and an adversarial example) in Appendix J. Section C.2 describes the data generation pipeline.

C.1 Workflows

The Airline CSM domain covers 50 scenarios across seven workflow categories backed by 15 tools. It is high-stakes and time-pressured, with heavy dependence on accurate transcription of named entities such as confirmation codes, flight numbers, and passenger names.

The Healthcare HRSD domain covers 83 scenarios across 12 single-intent workflows backed by 47 tools, extended with dual-intent, triple-intent, and adversarial variants. It has the highest per-workflow complexity in EVA-Bench, averaging 8.7 expected tool calls per scenario. Its defining challenge is the density of named entities communicated over voice — NPI numbers, DEA registration numbers, state license numbers, and OTP codes — where a single transcription error can cascade into authentication or policy failures.

The Enterprise ITSM domain covers 80 scenarios across 21 workflows backed by 59 tools, spanning single- to quadruple-intent and adversarial variants. Its defining characteristic is a branching flow structure, where incident workflows gate escalation on failed resolution attempts. Authentication is tiered across three levels — standard, OTP-elevated, and manager-level — reflecting the sensitivity of different workflows.

C.2 Data Generation Pipeline

TABLE 6 Comparison of EVA-Bench data domains. Multi-intent scenarios present one to four concurrent workflows within a single call. Auth tiers refers to distinct levels of caller authentication required across flows.

	Airline CSM	Healthcare HRSD	Enterprise ITSM
Scenarios	50	83	80
Workflows	7	12	21
Tools	15	47	59
Avg. tool calls	3.14	8.7	8.3
Min / max tool calls	1 / 6	1 / 18	1 / 18
Auth tiers	1	2	3

Synthetic Data Generation with SyGra

Scenarios are generated using SyGra [?], a graph-based synthetic data generation pipeline. Each scenario requires three jointly consistent components: a user goal (including a decision tree that constrains the user simulator to a deterministic outcome), a scenario database (the backend state the agent's tools query and modify), and an expected final database state (the ground truth against which task completion is evaluated). Joint generation is essential: the expected final state must be consistent with both the user goal and the initial database. Independent generation would introduce silent inconsistencies; for example, a flight number referenced in the user goal that does not exist in the scenario database would corrupt the evaluation signal.

Generation proceeds in the following stages:

1. Policy specification. Domain policies and workflow constraints are defined and reviewed prior to generation.
2. Joint generation. SyGra generates user goals, initial databases, and expected final states jointly from a workflow graph, using GPT-5.2 as the generative backbone.
3. Multi-intent composition. Multi-intent scenarios are constructed by combining single-intent records into coherent multi-workflow user goals, with expected final states merged accordingly.
4. Adversarial scenario design. Adversarial scenarios are hand-designed around specific policy boundary conditions, then verified against tool executor behavior to confirm that the policy violation is achievable but detectable by a correctly behaving agent.

Human Review

Following generation, all scenarios went through multiple rounds of manual review. Reviewers verified that: (1) policies were applied consistently across scenarios within a domain; (2) user goals were specific enough to admit exactly one correct resolution; (3) expected final states were internally consistent with both the user goal and the initial database; and (4) adversarial scenarios were correctly specified, with a clearly identifi-

able policy violation. Records identified as ambiguous or inconsistent were corrected or discarded.

Frontier Model Stress Testing

As a final validation step, we ran three frontier models — OpenAI/Gpt-5.4, Google/Gemini 3.1 Pro, and Anthropic/Claude Opus 4.6 — on a text-only version of each scenario, bypassing the audio pipeline and providing conversation transcripts directly. For every scenario on which any model scored zero on task completion, we manually investigated whether the failure reflected genuine model error or a dataset issue: an ambiguous policy, an under-specified user goal, a bug in the tool executor, or an inconsistency between the initial and expected database states. Records with identified dataset issues were corrected or removed. All selected samples had a task completion of 1 on at least one of the frontier models.

This process provides high confidence that task completion failures in the full audio evaluation reflect real agent errors rather than evaluation artifacts.

D User Simulator Details

We use ElevenLabs ElevenAgents [?] as the user simulator with the following cascade system: `Scribe v2.2 Realtime + GPT-5.1 + Eleven V3 Conversational`. We select these models for their high transcription accuracy, User Behavioral Fidelity, user realism for GPT-5.1 [?], and for their naturalness and realism for Eleven v3 Conversational. ElevenLabs also provides a large voice library, enabling testing of a wide variety of user accents, languages, speaking styles, etc.

We created four ElevenLabs agents for the user simulator, covering two accents (English and French) and two genders each. When creating a new agent, select *Blank Agent* as the starting template, then apply the configuration described in Table 7. The four voice names are listed in Table 8. All parameters not listed are set to their default values provided by ElevenLabs at agent creation.

TABLE 7 ElevenLabs User Simulator Configuration

Parameter	Value
TTS model family	V3 Conversational
Expressive mode	Enabled (no tags selected)
Language	English
LLM	GPT-5.1
System prompt	`{{prompt}}`
Default personality	Disabled
First message	None
Interruptible	Disabled
Advanced > Input audio	μ-law telephony, 8000 Hz
Advanced > Eagerness	Eager
Advanced > Take turn after silence	15 s
Advanced > Max conversation duration	600 s
Tools > System tools	`End conversation` (enabled)

TABLE 8 ElevenLabs Voice Name per Agent

Accent	Gender	Voice Name
English	Female	Natalee Champlin
English	Male	Eric - Smooth, Trustworthy
French	Female	Mariva Viva Muse - Warm and Energetic
French	Male	Jamie - French Accent & Charismatic

When enabling the ”End Conversation” system tool, the name must be `end_call`, and the description to provide is shown below. This allows the simulator to hang up programmatically.

”End Conversation” Tool Description

Use this to end the phone call and hang up.
Call this function when its time to end the call and one of the following is true:

1. The agent has confirmed your request is resolved (all steps are completed) and you have said goodbye
2. The agent has initiated a transfer to a live agent
3. The agent has been unable to make progress for at least 5 consecutive turns
4. The agent says goodbye or indicates the conversation is over
5. The agent indicates that the remainder of your request cannot be fulfilled.
6. If the assistant says something along the lines of ”I’m sorry I encountered an error processing your request.”

IMPORTANT: never call this tool in the same turn that you provide the agent with data, an identifier, a request to transfer to a live agent, an approval to proceed, or any kind of additional information.
Before calling this tool, always say a brief goodbye first.

Once the agent is configured, click `Publish` in the top-right corner. The `agent-id` can be retrieved from the `Widget` tab of the agent dashboard, under `Embed code`.

The simulator is prompted in EVA-Bench with a specific user goal and is instructed to stay on task, communicate all required named entities clearly, and terminate the conversation when the goal is accomplished, or the task is clearly unlikely to succeed. The system prompts are provided in Appendix K.

TABLE 9 EVA-Bench validation metrics. These metrics ensure the quality of the user simulator and the integrity of each conversation prior to evaluation.

Metric	Type	Scale	Pass Thresholds
Conversation Finished	Deterministic	$\{0, 1\}$	1.0
User Behavioral Fidelity	LLM-as-Judge	$\{0, 1\}$	1.0
User Speech Fidelity	LALM-as-Judge	$1–3 \rightarrow [0, 1]$	–

D.1 User Simulation Validation

Table 9 lists the three validation metrics used to verify user-simulator behavior and conversation integrity before any EVA-Bench evaluation is performed.

D.1.1 Conversation Valid End.

Before invoking any LLM judges, we run a deterministic check on the conversation logs to verify that each simulation terminated correctly. A valid end state is one in which either the agent failed to respond to the user, or the user invoked the end-call tool. Conversations meeting this criterion proceed to the User Speech and Behavioral Fidelity judges; all others are rerun.

This gate primarily catches infrastructure-level failures: WebSocket connections that closed unexpectedly, conversations that failed to start, or user simulator timeouts. These are pipeline errors rather than agent errors, and filtering them deterministically before any judge invocation avoids wasting LLM calls on malformed simulations.

D.1.2 User Behavioral Fidelity.

Motivation. A key assumption underlying our evaluation is that the user simulator faithfully follows its assigned goal and decision logic, since the ground truth end database state is derived from this assumption. If the user deviates, the agent may fail to reach the ground truth state through no fault of its own. We therefore define a validation gate that detects user behavior errors capable of corrupting the evaluation, organized into five *corruption types*:

1. Extra modifications. The user makes requests beyond its stated goal that invoke modification tools writing to the scenario database. The user simulator prompt explicitly instructs the user to decline any such offers from the agent, but we check for violations regardless.
2. Premature ending. In our simulations, the user is responsible for ending the call once its goal is complete. If the user hangs up prematurely—for example, providing actionable information and ending the call in the same turn—the agent has no opportunity to execute the required tool calls. We therefore verify that the user does not terminate the conversation in the same turn it provides critical information to the agent.
3. Missing information. If the user fails to provide information the agent needs to complete the task, the evaluation is corrupted since task success cannot reasonably be expected.
4. Duplicate modifications. Occasionally, the user simulator (particularly when using non-primary models) enters a loop and repeats requests the agent has already fulfilled. The agent then acts on the duplicate request, causing redundant writes to the scenario database that cause the final state comparison to fail.
5. Decision tree violations. Each user is given a structured decision tree governing how to navigate choices during the interaction (e.g., *"accept the earlier flight if the price difference is under $200, otherwise decline"*). We verify that the user adheres to this logic, since deviations would cause the agent to reach a final state inconsistent with the ground truth.

Implementation. We prompt a judge model with a description of the five corruption

types, the full conversation trace (including agent tool calls), and the list of available agent tools. The judge first produces a *corruption analysis* assessing each corruption type in turn, then outputs a binary flag per corruption type, and finally an overall binary rating for the conversation. Conversations receiving a rating of 0 are rerun. See subsection M.7 for the full prompt.

Validation. To validate the judge, we constructed a human-annotated dataset of real user simulator failures from earlier evaluation artifacts alongside correct behavior examples, labeled with both overall ratings and per-corruption-type annotations. GPT-5.2 with medium reasoning achieved 100% accuracy across three independent runs, and was therefore selected as the judge model.

Error Categorization. Across 714 trials flagged by the User Behavioral Fidelity judge—spanning four evaluated systems across all domains—premature ending was the most frequent corruption type (63.9%), followed by decision tree violations (52.9%), missing information (28.6%), extra modifications (1.3%), and duplicate modifications (0%). Notably, 42.4% of flagged trials exhibited two or more corruption types simultaneously.

D.1.3 User Speech Fidelity.

Motivation. As noted in subsection 3.1, speech fidelity flags are rare in practice, which may invite questions about why this validation gate was included at all. The answer lies in one of the primary failure modes we observe in voice agents: failure to transcribe and understand key entities. To attribute such failures confidently to the agent rather than the user simulator, we must verify that the user's synthesized speech correctly conveys all critical entities. This is particularly consequential because most flows begin with an authentication step requiring the agent to correctly capture entities such as names, confirmation codes, and account IDs—if the user's speech corrupts these, the agent cannot proceed regardless of its own capability.

Implementation. We adapt the same prompt used to evaluate agent speech fidelity (Section M.3), with one key modification to the rating scale. Unlike for the agent, we do not require the user's speech to precisely mirror every word of the user-side LLM output; only key entities and major informational content must be conveyed accurately. The judge therefore rates each turn on a 3-point scale: 3 indicates full fidelity, 2 indicates minor errors that do not affect the agent's ability to progress (e.g., slight disfluencies, but all entities intact), and 1 indicates an entity error or significant omission or addition that would prevent the conversation from proceeding sensibly. Any conversation in which any turn receives a rating of 1 is rerun. The full prompt can be found in subsection M.6.

Validation. Because this prompt is closely derived from the agent speech fidelity judge, we inherit its validation. That judge achieved high inter-annotator agreement with human linguists ($\kappa = 0.777$, 95% CI [0.704, 0.835]), and the core capability it requires—accurately parsing audio and detecting entity-level errors—is shared. The rating scale and its interpretation are sufficiently well-defined that additional annotation studies were not deemed necessary. See table 14 for more details on human-judge agreement.

TABLE 10 Distribution of corruption types among trials flagged by the User Behavioral Fidelity judge. Percentages sum to more than 100% because a single trial may exhibit multiple corruption types.

Corruption Type	Cases	% of Flagged
Premature ending	456	63.9%
Decision tree violation	378	52.9%
Missing information	204	28.6%
Extra modifications	9	1.3%
Duplicate modifications	0	0%

E Metric Details

TABLE 11 EVA-Bench metrics organized by category. All EVA-A and EVA-X scores are normalized to $[0, 1]$ prior to aggregation. Thresholds are used for pass@k and pass^k computation: a run is considered successful if all EVA-A and EVA-X metrics meet their respective thresholds simultaneously.

Category	Metric	Type	Scale	Pass Thresholds
EVA-A (Accuracy)	Task Completion	Deterministic	$\{0, 1\}$	1.0
	Faithfulness	LLM-as-Judge	$1–3 \rightarrow [0, 1]$	0.50
	Speech Fidelity	LALM-as-Judge	$[0, 1]$	0.95
EVA-X (Experience)	Conciseness	LLM-as-Judge	$1–3 \rightarrow [0, 1]$	0.50
	Conversation Progression	LLM-as-Judge	$1–3 \rightarrow [0, 1]$	0.50
	Turn-Taking	Deterministic	$[0, 1]$	0.80
Diagnostic	Authentication Success	Deterministic	$\{0, 1\}$	—
	Response Latency	Deterministic	seconds	—
	Speakability	LLM-as-Judge	$\{0, 1\}$	—
	STT Word Error Rate	Deterministic	$[0, \infty)$	—
	Tool Call Validity	Deterministic	$[0, 1]$	—
	Transcription Key Entities	LLM-as-Judge	$[0, 1]$	—

E.1 Log Processing and Variable Extraction

Available LogsEvery conversation typically produces three independent log streams, which we merge and replay deterministically to recover the variables needed by our metrics. The streams are:

- Audit log (`audit_log.json`) — always present. The agent's internal record: user-side STT transcripts (in cascade pipelines), the assistant's full LLM output, and tool calls/responses with their parameters and timestamps.
- Framework events (`framework_logs.jsonl`) — written by a generic framework logger that any speech pipeline (Pipecat, S2S, custom audio LLM, …) can attach to in order to record TTS-stage text events with wall-clock timestamps. The two records the processor depends on are `tts_text` (the chunk actually sent to TTS) and `llm_response` (the LLM-side text). Some pipelines emit `tts_text`, some emit `llm_response`, some emit both. Pipelines that emit neither (notably S2S systems with no separable TTS step) effectively contribute nothing to this stream, in which case the variables that draw from it are left empty. When present, these events give the canonical *intended* assistant text in the sense of *what was actually sent to the TTS engine*. This is *not* the assistant's full LLM output: that lives in the audit log and may include continuations beyond an interruption point that

were never sent to the TTS step. Concretely, `intended_assistant_turns` is populated directly from these framework events, while audit-log assistant entries enter only the conversation trace, where they are post-hoc truncated to the longest prefix attested in the framework log (entries with no spoken overlap are dropped).

- ElevenLabs events (`elevenlabs_events.jsonl`) — events from the user simulator and the shared audio bus: per-speaker `audio_start`/`audio_end` markers, the user simulator's own outgoing text (`user_speech`, treated as the user's *intended* utterance), and the provider's ASR of the assistant audio channel (`assistant_speech`, treated as the assistant *transcribed* utterance).

The available streams are concatenated and sorted by timestamp into a single timeline, then traversed in one pass. Turn boundaries are driven *exclusively* by `audio_start(elevenlabs_user)` events: turn 0 is reserved for the assistant greeting (anything before the first user audio session), and each subsequent user audio session, provided the assistant has spoken since the last advance, increments the turn counter so that index i aligns assistant turn i as the response to user turn i. Several edge cases require care: (a) empty user sessions (background noise without any `user_speech` payload) are rolled back so they do not consume a turn index; (b) `user_speech` events that arrive before their `audio_start` are buffered and replayed once the correct turn is known; and (c) after a barge-in, a `hold_turn` flag suppresses the next advance from a late STT chunk belonging to the interrupted utterance, while still allowing a fresh `audio_start` to advance normally.

InterruptionsInterruptions are detected whenever one speaker's `audio_start` fires while the other's audio session is still open, producing two disjoint sets `assistant_interrupted_turns` and `user_interrupted_turns`; the corresponding text fields are decorated with `[assistant interrupts]` / `[user interrupts]` (entry-level prefixes) and `[likely cut off by user]` / `[likely cut off by assistant]` / `[likely cut off on its own]` (turn-level suffixes). We deliberately mark these labels as *likely* rather than excising the post-interruption text: the intended-text streams record *what was sent to TTS*, not what was vocalised, and there is typically a non-trivial delay between text being handed to TTS and audio reaching the speaker. Words queued in the final moments before an interruption may therefore have been buffered but never played, so the precise truncation point in the intended text is not recoverable from the logs alone, and we let the annotation flag the ambiguity rather than make a hard cut at a position we cannot identify with confidence. This help the downstream judges to adjust their scoring based on the presence of interruptions. For example, AgentSpeechFidelity should not penalize if some words that are present in the intended text were never said due to an interruption.

Extracted VariablesFor each turn we extract four per-role variables: `intended_*_turns`, `transcribed_*_turns`, `audio_timestamps_*_turns`, and entries of a linearised `conversation_trace` that interleaves user/assistant turns with tool calls. The default mapping from log source to variable is given in Table 12. Crucially, the table distinguishes the per-turn text fields (which are sourced directly from a single stream) from the `conversation_trace` (which is

built from the audit log and post-hoc reconciled against the other streams). The `conversation_trace` is the linear, tool-call-interleaved view used by judge metrics that need a faithful chronological transcript, while the per-turn fields are useful for specific metrics that need intermediate states, such as TranscriptionAccuracyKeyEntities. `audio_start`/`audio_end` pairs are matched greedily by speaker, and used to compute any latency measurements.

TABLE 12 Default mapping from log source to extracted variable. The upper block lists the per-turn text and audio fields; the lower block lists the entries that compose the linear `conversation_trace`. Pipeline-specific overrides are listed in the text.
* Empty if the framework emits neither record. † Falls back to `assistant_speech` for S2S. ‡ Uses `user_speech` for audio-native pipelines (S2S/Hybrid).

Variable	Source
`transcribed_user_turns[i]`	`audit_log` / user
`intended_user_turns[i]`	`elevenlabs` / `user_speech`
`intended_assistant_turns[i]`	`framework_logs` / `tts_text`, `llm_response`*
`transcribed_assistant_turns[i]`	`elevenlabs` / `assistant_speech`
`audio_timestamps_{role}_turns[i]`	`elevenlabs` / `audio_start`, `audio_end`
`tool_params`, `tool_responses`	`audit_log` / `tool_call`, `tool_response`
`conversation_trace` (assistant)	`audit_log` / assistant, truncated to framework-log prefix†
`conversation_trace` (user)	`audit_log` / user‡
`conversation_trace` (tools)	`audit_log` / `tool_call`, `tool_response`

Pipeline type modifies this default in two principled ways, reflecting which signals are trustworthy for each architecture. These differences impact the metrics definition, as discussed in E.2.

1. S2S. There is no separable TTS step, so the framework log carries no `tts_text` or `llm_response` records and is dropped from the merge. Consequently `intended_assistant_turns` is left empty — S2S models typically do not expose any separate text intent — and the assistant's entries in `conversation_trace` are sourced from ElevenLabs `assistant_speech` (transcribed) rather than from the audit log. Symmetrically, S2S models consume the user audio directly and do not produce a trustworthy STT transcript of the user, so the user entries in `conversation_trace` are sourced from `user_speech` (the simulator's intended text) rather than from the audit log.
2. Hybrid. The framework log is populated with `tts_text` or `llm_response` (depending on the backend), and `intended_assistant_turns` is built as in cascade. On the input side, however, hybrid audio-native models bypass the agent's STT — as in S2S — so the audit-log user transcripts are unreliable, and the user entries in `conversation_trace` are again sourced from `user_speech` (intended) rather than from the audit log.
3. Cascade. All three streams are used unmodified: audit-log user transcripts feed both `transcribed_user_turns` and the trace, the framework log supplies the assistant's in-

tended text, and ElevenLabs supplies the user's intended text and the assistant's transcribed text.

A final post-processing step (i) aligns the per-turn dictionaries so that all sources share the same key set, with missing slots back-filled by the most informative remaining source; and (ii) reconciles the trace by ensuring the greeting is the first entry, appending any final user turn that arrived after the last audit-log entry, and propagating trailing-cutoff labels consistently across `intended_*`, `transcribed_*`, and the trace.

Audio recordings.In addition to the event logs, every session writes three 16-bit PCM mono WAV files, captured directly from the audio bus: `audio_user.wav` contains *only* the user simulator's outgoing audio, `audio_assistant.wav` contains *only* the assistant's outgoing audio, and `audio_mixed.wav` is a sum-mixed mono recording of both speakers, used as a reference of what either party would actually have heard. Splitting the channels at capture time — rather than diarising a mixed recording post-hoc — is what allows the speech-fidelity metrics to operate without speaker-attribution noise: AgentSpeechFidelity consumes `audio_assistant.wav` to compare what the assistant *said* against its corresponding `intended_assistant_turns`, and similarly for UserSpeechFidelity with `audio_user.wav` to validate that the user simulator's actual speech matches its scripted persona and goal. The mixed channel is not used by any metric, it is a reference that can be used for human review.

Limitations and fidelity.The three log streams are emitted by independent components — the agent, the speech framework, and the user simulator's provider — and can drift from one another in edge cases, particularly around interruptions and rapid turn switches. We have observed, for example, some ElevenLabs `audio_end(elevenlabs_user)` arriving noticeably after the user simulator actually stopped speaking (so the audio session appears to "stay open" past its useful end), occasional transcripts that are missing or delayed, and misalignments between the framework log timestamps and the ElevenLabs ones. Making turn boundaries align across all sources is also a challenge, as one user turn can be detected as two on the framework side. The audio recordings themselves are also not always unambiguous, and observed latencies do not always match the latency estimates derived from log timestamps. The heuristics described above (empty-session rollback, late-transcript buffering, prefix-truncation against the framework log, the `hold_turn` flag after barge-ins) are designed to absorb the most common of these inconsistencies. To validate that residual drift is not a confound, we manually inspected a stratified sample of conversations across all three pipeline types: the extracted variables were high-fidelity in aggregate, per-turn alignment between intended and transcribed text was consistent, interruption labels matched the audio, and trace ordering matched the perceived chronology of the conversation. Cases where a single log entry was missing, a turn boundary was off by one, or interruption tags were wrong did occur but were rare and did not systematically bias any of the metrics reported in this work.

E.2 Equitable Evaluation Across Cascade, Hybrid, and S2S Architectures

The three pipeline types differ in what is observable about the agent's reasoning (Appendix E.1), and a naive single-view evaluation would systematically favour one architecture over another. We therefore provide pipeline-aware variants of the reasoning-oriented metrics — Faithfulness, Conciseness, Conversation Progression— so that every system is judged on the most faithful proxy of *what its LLM actually saw and produced*.

In a cascade system, the LLM consumes user turns as STT transcripts and emits agent turns as text before TTS rendering. Both halves of this LLM-internal view are recoverable: `transcribed_user_turns` captures what the LLM read, and `intended_assistant_turns` captures what it wrote. Scoring on these two fields ensures that STT and TTS errors — the responsibility of the surrounding pipeline, not the LLM — are not attributed to the model under test.

In an S2S system, the model consumes and emits audio directly; there is no intermediate text on either side. The conversation trace is therefore built from the user simulator's *intended* text on the user side and a post-hoc ASR transcript of the assistant's audio also on the user side (see Appendix E.1). The same judges then evaluate the agent as a whole — speech understanding and synthesis included — since these capabilities are part of the S2S system's responsibility. To avoid charging the agent for transcription errors of its output, the judge prompt explicitly defer such errors to Speech Fidelity, which receives the raw audio.

Hybrid systems are evaluated with a mixed view that follows the same observability principle as the other two. Like S2S, hybrid models bypass the agent-side STT, so user turns are taken from the simulator's intended text rather than from a transcript; like cascade, hybrid models retain a separable text-to-TTS step, so agent turns are taken from `intended_assistant_turns` via the framework log.

E.3 Judge Development and Validation

LLM-as-judge evaluations are only as reliable as the judges themselves. We developed each judge through a structured five-stage pipeline: metric definition, prompt construction, development dataset construction, prompt improvements and judge model selection, and final validation against human annotation.

Metric definition and rating scales.For each judge metric, we first defined the rating scale and the explicit failure modes that distinguish each rating level. Each failure mode specifies both what the defect looks like and how a judge should detect and categorize it. This categorical and granular framing is intentional: it preserves actionable signal about *how* a voice agent fails, rather than collapsing everything into a single score. It also helps the judge accurately score by offering detailed categories.

Judge prompt construction.The failure mode definitions were operationalized into judge prompts, with explicit criteria for each rating category and targeted guidance for edge cases on the boundary between adjacent ratings.

Development dataset construction and labeling.We constructed a development dataset for each metric using synthetic data generation followed by multi-model consensus labeling. For data generation, we prompted frontier models to produce conversations exhibiting specific, targeted failure modes within each metric. This targeted generation was key to achieving class balance: on top of sampling from naturally occurring conversations, we forced the generator to produce examples of each failure mode explicitly. All generated data used the same domains, policies, and agent configurations as in EVA-Bench, ensuring the development set is in-distribution with respect to our evaluation targets. One exception is Speech Fidelity, where the class distribution is intentionally skewed toward failures (122 failing vs. 25 passing out of 147 records). This metric is evaluated by a Large Audio Language Model (LALM), and not a text-based LLM judge, and LALM-as-judge is a relatively nascent paradigm. We had limited confidence that the model would reliably attend to subtle acoustic artifacts without explicit exposure to a wide range of failure modes during development. Furthermore, because naturally occurring synthesized speech is predominantly artifact-free, a naive judge could achieve high accuracy simply by predicting "pass" for every sample. To guard against both of these risks, we oversampled failure cases during data generation, forcing a wide variety of speech artifacts to ensure the judge is sensitive to errors. In practice, we observe that the selected judge rarely predicts a passing label when the true label is failing, validating this design choice.

For labeling, we called three frontier models—Gemini 3.1 Pro, Claude Opus 4.6, and GPT-5.2—as judges on each generated sample. When all three models agreed on a rating, that rating was assigned as ground truth. When they disagreed, a human reviewer examined the sample and selected the correct label.

Prompt improvements and Judge model selection.We used the development sets to refine the prompts, focusing on samples where judges disagreed and analyzing their explanations. This process revealed ambiguities in the judge instructions and informed targeted prompt improvements.

We then formally evaluated each of the three frontier models as judge candidates on the development datasets, reporting accuracy alongside macro-averaged F1. We selected the judge model that achieved the highest combined score on each metric's development set (see Table 13). For Speech Fidelity, Gemini 3.1 Pro achieved the highest performance, but we selected Gemini 3 Flash for deployment: it achieved nearly identical performance at substantially lower inference cost.

Test set validation and human agreement.To assess the validity of the finalized judge prompts and selected models, we separately constructed a held-out test set of 63 samples per metric, never used during prompt development. Each sample was labeled by two expert linguists. For text-based metrics (Faithfulness, Conversation Progression, Conciseness), linguists were shown the conversation sample and the judge outputs from all three frontier model candidates, presented blind to which model produced which output. We provided judge outputs to the linguists because voice agent transcripts can be long and complex; without the judge's surfaced evidence and analysis, annotators found it difficult to reliably

TABLE 13 Judge validation datasets and results per candidate model. Size = number of annotated examples. Dist. = class distribution. Acc. = judge accuracy against adjudicated human labels. F1 = macro F1-score. Bold indicates the selected judge model for each metric.

	Dataset	Claude Opus 4.6		GPT-5.2		Gemini 3 Flash		Gemini 3.1 Pro	
Metric	Size	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
Faithfulness	137	83.9%	80.7%	70.8%	68.9%	–	–	81.0%	78.4%
Conciseness	100	90.8%	73.4%	92.3%	84.0%	–	–	91.2%	74.4%
Conv. Progression	136	74.3%	73.1%	78.7%	76.7%	–	–	69.1%	66.6%
Speech Fidelity	147	–	–	–	–	89.1%	83.2%	91.2%	85.9%

identify subtle failure modes in long traces. Linguists verified the judge's reasoning against the transcript directly rather than accepting it uncritically. For Speech Fidelity, linguists listened to the raw audio only along with the intended text, with no judge output, since this metric requires direct perceptual evaluation of synthesized speech.

We report inter-annotator agreement using Cohen's κ in Table 14: linguist–judge agreement (L_J, pooled across 126 pairs per metric), linguist–judge agreement using a single randomly selected linguist per record (L_J_{rand}, 10,000 iterations), and linguist–linguist agreement (LL, 63 pairs per metric). To verify that the pooled pairing design does not distort agreement estimates, we computed κ using a single randomly selected annotator per record (10,000 iterations combining labeler randomization with record-level bootstrap); results fell within 0.007 of the pooled estimates across all metrics. Quadratic-weighted κ is used for the three ordinal metrics; unweighted κ for the binary Speech Fidelity metric. 95% confidence intervals are from 10,000 record-level bootstrap resamples.

Linguist–judge κ ranges from 0.777 to 0.845 across the four metrics—a strong result, and notably one that meets or exceeds the linguist–linguist agreement ceiling in every case. This indicates that our judges are not merely consistent with human raters, but that human–judge agreement is at least as high as the agreement between two human experts annotating the same data.

E.4 Accuracy Metrics

E.4.1 Task Completion

What it measures. Task Completion is the bottom-line accuracy check: it verifies if the agent actually accomplished what it was asked to do. Concretely, it asks whether the changes the agent committed to the scenario database during the conversation match the expected end state encoded in the dataset's ground truth. Unlike the judge metrics, this is a deterministic code-based check with no LLM in the loop; the same conversation always yields the same task-completion verdict, and the verdict is binary.

Method.Each scenario in the dataset specifies an `expected_scenario_db`: the database state we expect after a successful run. During execution, every state-mutating tool call writes

TABLE 14 Human–judge agreement (κ, Spearman ρ) for the primary judge (L_J) versus the human linguist inter-annotator baseline (L_L). Quadratic-weighted κ is used for the 1–3 ordinal metrics; unweighted κ for the binary Agent Speech Fidelity metric. 95% CIs from 10,000 record-level bootstrap resamples. To verify that the pooled pairing design does not distort agreement estimates, we computed κ using a single randomly selected annotator per record (10,000 iterations combining labeler randomization with record-level bootstrap); results fell within 0.007 of the pooled estimates across all metrics. L_J κ ranges from 0.777 to 0.845 across the 4 metrics; Spearman ρ agrees with κ within 0.008 for every metric, indicating no systematic calibration bias. Note that IAA-L is a practical rather than strict ceiling: when two human annotators disagree and the judge agrees with one, IAA-J is inflated relative to IAA-L. Nonetheless, similar IAA-J and IAA-L values indicate that the judge is at least as consistent as the human annotators.

Metric	L_J κ	95% CI	L_J ρ	L_J$_{\text{rand}}$ κ	95% CI	L_L κ	95% CI	L_L ρ
Faithfulness	0.836	[0.729, 0.915]	0.844	0.832	[0.697, 0.932]	0.740	[0.566, 0.870]	0.756
Conv. Progression	0.845	[0.753, 0.911]	0.843	0.841	[0.724, 0.931]	0.769	[0.627, 0.875]	0.782
Conciseness	0.823	[0.754, 0.874]	0.826	0.823	[0.745, 0.883]	0.825	[0.749, 0.881]	0.825
Speech Fidelity	0.777	[0.704, 0.835]	0.781	0.770	[0.683, 0.846]	0.754	[0.685, 0.817]	0.754

TABLE 15 Judge model identifiers and inference parameters used during evaluation. Parameters not listed were left at provider defaults.

Model	Model ID	Parameter	Value
GPT-5.2	`gpt-5.2`	`max_tokens`	100,000
Claude Opus 4.6	`us.anthropic.claude-opus-4-6-v1`	—	—
Gemini 3 Flash	`gemini-3-flash-preview`	`temperature`	0.0
		`max_tokens`	40,000
		`reasoning_effort`	`minimal`

through to a per-record copy of the scenario database, and the final state is captured at the end of the conversation (`final_scenario_db`). The metric canonically serialises both states (sort keys, no whitespace), computes their SHA-256 hashes, and reports a pass if the two hashes match. When they do not match, a structured diff is computed, including tables added/removed/modified, records added/removed/modified within tables, and field-level changes within records.

Inputs.Task Completion does not consume the conversation trace, the audio, or any text-side variables produced by log processing. It uses only:

- `expected_scenario_db` — the dataset's ground-truth final state for this record;
- `final_scenario_db` — the actual final state captured at end-of-run;
- `final_scenario_db_hash` — the SHA-256 hash of `final_scenario_db`.

This separation is deliberate: the metric measures *outcome* on the database, decoupled from how the agent got there. Path-quality concerns (correct tool usage, faithful disclosure, efficient progression) are the responsibility of the corresponding judge metrics (Appendices E.4.2, E.5.1).

Authentication gate.Authentication state lives in a dedicated `session` field of the scenario database, and is verified separately from the hash comparison rather than folded into it. The reason is that some scenarios can be satisfied by different valid authentication paths — different combinations of identifying fields the agent may legitimately collect — and a hash computed over the full database would mark any such variant as incorrect. We therefore (i) strip the `session` key from both the expected and the actual scenario database before computing either hash, and (ii) verify authentication via a superset check: every key-value pair in the expected session must be present in the actual session (string comparisons are case-insensitive), but the actual session may carry additional fields without penalty. If the superset check fails, the metric short-circuits to a fail with details describing which session keys mismatched; otherwise the run proceeds to the hash comparison on the rest of the database.

Determinism by construction.For task completion to be a meaningful metric, a scenario must yield the same expected end state regardless of conversational variation, provided the agent does not make mistakes. We enforce this by tightly constraining the user simulator, as discussed in D, so two valid conversations on the same scenario commit produce identical final-state hashes. Variation between systems (or between runs of the same system) only arises when an agent's behavior deviates from the dataset's intended outcome.

Rating scale and aggregation.Task completion is binary: 1.0 when the expected and actual hashes match, 0.0 otherwise.

Pass/fail thresholding.For pass-related aggregations such as pass@k, the pass threshold is set to 1.0 (i.e. exact match). There is no middle ground for this metric; a run either commits the expected writes or it does not. The structured diff in the failure details supports finer-grained downstream analysis when one wants to understand *how* a run fell short, but the headline metric remains binary.

E.4.2 Faithfulness

What it measures. Faithfulness is a conversation-level accuracy judge that asks whether the assistant remained grounded in the information, policies, and instructions available to it throughout the run. Unlike Task Completion — which only checks whether the goal was achieved — Faithfulness penalises the path: a conversation that concludes successfully but along the way hallucinates a fee, skips a required confirmation, or commits a fabricated identifier to a write tool will receive a low faithfulness score.

Judge and Prompt Template.The judge is run as a single LLM-as-judge call per conversation, using *Claude Opus 4.6*. The full prompt is available in Appendix M.2.

Inputs.The judge consumes the full conversation trace alongside everything needed to evaluate it against agent policy:

- the linearised `conversation_trace` (with tool calls and responses inline);
- the agent's configuration — `agent_role`, `agent_instructions`, and the JSON schema of the available tools;
- the simulated `current_date_time`, used to resolve temporal references and policy windows;
- the pipeline-aware shared fragments documenting how user and assistant turns are sourced and how interruption tags are used (Appendix M.1);
- two faithfulness-specific pipeline-aware fragments, `disambiguation_context` and `misrepresentation_pipeline_note`, described below.

Failure modes.The judge scores five disjoint dimensions, each defined to be non-overlapping so that any given issue maps to exactly one.

1. **`fabricating_tool_parameters`** — the assistant called a tool with a parameter value that cannot be traced to any user statement, prior tool result, policy entitlement, simple arithmetic, or standard domain mapping. Includes invented IDs, empty placeholder values, and wrongly chosen enum buckets.
2. **`misrepresenting_tool_result`** — the assistant inaccurately conveyed something a tool actually returned: wrong field value, contradicted status, omitted material caveats (e.g. a non-zero fee), or arithmetic errors when computing values from tool data.
3. **`violating_policies`** — the assistant contradicted the agent instructions: skipped a required verification step, executed an irreversible write without the disclosure or confirmation the policy requires, or stated a policy incorrectly.
4. **`failing_to_disambiguate`** — the assistant proceeded on ambiguous or contradictory user input without clarification (multiple options, conflicting values, suspicious lookups failing on uncommon names or codes).
5. **`hallucination`** — a residual category for information stated to the user that has no source at all in any tool response, user utterance, agent instruction, or system context, and that is not already captured by the four preceding dimensions.

Pipeline-aware adaptations.Faithfulness applies the general framing of Appendix E.2 and adds two faithfulness-specific deltas. First, the `disambiguation_context` fragment changes the bar for clarification: in cascade, the assistant is reminded to account for STT-style transcription errors before write actions; in audio-native pipelines, the assistant is held to a *higher* clarification bar because mishearing letters, numbers, names, and codes is intrinsic to consuming raw audio, and the model is expected to anticipate it. Second, the `misrepresentation_pipeline_note` explicitly scopes the `misrepresenting_tool_result` dimension for audio-native pipelines: because assistant turns in those traces are post-hoc ASR of the

assistant audio, token-level discrepancies between an assistant utterance and a tool result (dropped dashes, single-character substitutions, missing/extra digits in long IDs) typically reflect TTS-rendering or post-hoc-ASR artifacts and are scored by Speech Fidelity, not here; only structural or semantic discrepancies (wrong field, wrong order of magnitude, wrong category, or downstream signals indicating the agent was operating on a wrong value) are flagged on this dimension. In cascade, this note is empty: assistant turns are intended TTS text, so any discrepancy with a tool result *is* attributable to the LLM. The result is that mishearing the user is a faithfulness violation in S2S and Hybrid but not in cascade (where it is the STT's responsibility).

Rating scale.Each dimension is rated on a 3-point integer scale:

- 3 — no issue on this dimension.
- 2 — minor or ambiguous issue with low user impact (e.g. a fabricated parameter that reaches a read-only tool, is caught quickly, and never surfaces to the user; a borderline policy deviation; a small phrasing embellishment that does not alter any decision).
- 1 — clear violation with material impact (e.g. a fabricated parameter passed to a write tool, regardless of whether the call succeeds; an irreversible action without the policy-required disclosure or confirmation; misstating a fee, balance, or eligibility rule the user could act on).

The dimension-level ratings are aggregated by *minimum* into a single overall rating, so a single rating-1 dimension produces an overall rating of 1. The overall rating is normalised to $[0, 1]$ via $(r - 1)/2$, giving $3 \to 1.0$, $2 \to 0.5$, $1 \to 0.0$. The flagged dimensions are preserved in the metric details and aggregated across samples.

Pass/fail thresholding.For pass-related aggregations, such as pass@k, we count a conversation as a faithfulness pass if its overall rating is ≥ 2, i.e. its normalised score is ≥ 0.5. This threshold sits exactly at the rubric's own load-bearing boundary: rating 1 is reserved for violations with *material* impact on the user — financial consequences, irreversible actions taken without the policy-required disclosure or confirmation, misstatements of policy that the user could act on later — while rating 2 is explicitly defined as covering minor or ambiguous issues that do *not* materially affect the outcome (a small phrasing embellishment, a quickly self-corrected read-only fabrication, a borderline judgement call). As such, rating-2 conversations are still treated as acceptable: at the current capability level of voice-LLM systems, demanding strict rating-3 perfection on every run would be unrealistic and would compress meaningful differences between systems into a uniformly low pass-rate.

E.4.3 Speech Fidelity

What it measures. Speech Fidelity asks whether the assistant's spoken audio actually matches what the system intended (or was expected) to communicate. It is the audio-side complement to Faithfulness: faithfulness scores what the LLM *decided* to say, speech fidelity scores whether the user could correctly hear the entities the LLM (or the upstream tool

responses) returned. Errors here would result in the user receiving the wrong information, such as a garbled flight number, a confirmation code with one substituted character, or a dollar amount whose digit is dropped. The metric is computed per-turn on the assistant audio channel only (`audio_assistant.wav`, see Appendix E.1), and aggregated by mean across rated turns.

Judge and Prompt Template.Speech Fidelity is an audio-judge metric: it sends a multimodal request (audio + textual context) to *Gemini 3 Flash*. The audio is encoded as `base64 WAV` and accompanied by the per-turn entity context the judge needs to verify. To reduce upstream noise and cost, the assistant audio is silence-trimmed before being sent to the judge. The full prompt is available in Appendix M.3.

Two pipeline-specific variants.Unlike the text judge metrics, speech fidelity does not have a single prompt with pipeline-aware fragments: the cascade/hybrid case and the S2S case have qualitatively different inputs and use different prompts.

1. Cascade and Hybrid. Both architectures expose an *intended* text-side reference for the assistant, i.e., the LLM's text output before TTS (`intended_assistant_turns` E.1). The judge task is a direct word-for-word comparison: did the audio reproduce the intended text, with particular attention to TTS-critical entities (confirmation codes, flight numbers, dollar amounts, dates, names, spelled-out alphanumeric codes, segmented reference IDs).
2. S2S. S2S systems do not typically expose any text-side intent, so there is nothing to compare the audio against in the cascade sense. We instead reformulate the question as an *entity articulation* check: does the assistant clearly and correctly speak the entities it was supposed to convey? The judge receives a *redacted conversation trace* in which assistant entries are replaced by an `"[Assistant speaks]"` placeholder per turn, while user utterances and tool responses are preserved verbatim. These are the entity sources the assistant was supposed to articulate. The judge transcribes the assistant audio itself and checks, per turn, that any entities it speaks that originate from the trace (a confirmation code returned by a tool, a name supplied by the user) are clearly audible. Turns where the assistant speaks no in-trace entities (greetings, questions, clarifications using only system-side phrasing) are flagged `has_entities: false` and excluded from aggregation.

Critically, the S2S variant explicitly excludes faithfulness/correctness from its scope. If the agent says `"$315"` when the tool returned `"$300"`, that is a faithfulness violation (Appendix E.4.2), not a speech-fidelity issue: the metric only flags the turn if the dollar amount is garbled in audio. Hallucinated entities not present in the trace are likewise out of scope for this metric. This is the symmetric counterpart of the `misrepresentation_pipeline_note` carve-out on the faithfulness side: token-level audio artifacts are scored here, not under faithfulness.

Failure modes.A turn is rated 0 when any of the following are observed; otherwise 1:

- an entity spoken with wrong digits, letters, amounts, or numbers (cascade/Hybrid: against the intended text; S2S: against the corresponding source in the trace);
- missing words that change the meaning of the turn or omit an entity;
- added words that introduce a factually different entity;
- substituted words that alter an entity value;
- for spelled-out codes ("Z K three F F W"), any letter or digit that is unclear, missing, or substituted;
- for segmented reference IDs (`REF-8JVSDF-001`, `MEAL-FAR0UM-PAX0`), any segment that is unclear or wrong (e.g. "M E L" versus "M E A L").

Carve-outs.The prompt explicitly does *not* penalise:

- minor pronunciation variations that do not change entity identity ("Ms." vs "Miss");
- filler words ("um", "uh", "so") added or omitted;
- slight pacing or prosody differences;
- non-spoken audio-direction tags in the intended text (`[slow]`, `[firm]`, `[annoyed]`) — these describe how the words should be spoken and were never expected in the audio;
- words in regions flagged by interruption tags as likely not spoken (Appendix E.1) — if a tag indicates a span was likely cut off, missing words in that span are not penalised;
- missing words at the very end of the *last* turn only (audio cutoff at the end of the conversation);
- for the S2S variant, any entities the agent speaks that are *not* in the conversation trace (those are out of scope — the metric does not evaluate hallucinations).

Rating scale and aggregation.Each rated turn is given a binary rating in $\{0, 1\}$, where 1 means the audio is correct, and 0 means there is an issue. The conversation-level score is the mean across rated turns; for S2S, turns excluded by `has_entities: false` are dropped from both the numerator and the denominator. Per-turn ratings, the judge's audio transcript per turn, and the natural-language explanation citing intended-vs-actual mismatches are preserved in the metric details for inspection.

Pass/fail thresholding.For pass-related aggregations such as pass@k, the pass threshold is set to 0.95, i.e. a conversation passes if at most a small fraction of rated turns are flagged with an entity error. We use a stricter threshold than the rubric-grounded 0.5 used for the 3-point judge metrics for two reasons. First, the rating scale is already binary: there is no rating-2 "minor issue" band whose inclusion this threshold has to negotiate, so the rubric-grounding argument used elsewhere does not apply. Second, the underlying error is high-stakes: a single garbled confirmation code or dollar amount can render a whole conversation operationally wrong, so tolerating more than the occasional turn-level slip would mask exactly the kind of failure this metric exists to catch. The 0.95 ceiling is therefore

set to absorb sporadic per-turn judge noise (e.g. a single ambiguous spell-out) while still requiring the audio-side reproduction to be substantively faithful across the run.

Limitations.Manual inspection of a stratified sample of rated turns surfaced a few false positives (the judge rates 0 on a turn whose audio actually reproduced the intended content). They are typically caused by upstream log-processing artifacts rather than by judge mistakes. The metric compares the audio against `intended_assistant_turns`, and that reference is only as good as the merge of the framework log and the ElevenLabs audio events described in Appendix E.1. Two failure patterns recur. First, the intended text contains a trailing span that was handed to TTS but never vocalized because of a barge-in whose interruption tag was not raised. The heuristics in Appendix E.1 absorb most cases, but residual mis-tagging still occurs, and the judge then legitimately flags "missing words" against an intended text that was never meant to be heard. Second, turn-boundary drift between the framework log and the ElevenLabs timeline aligns the wrong intended utterance with a given assistant turn, so the judge compares audio for turn i against intended text for turn $i\pm1$, producing a spurious mismatch. These false positives are concentrated on turns adjacent to interruptions and rapid turn switches.

E.5 Experience Metrics

E.5.1 Conversation Progression

What it measures. Conversation Progression is a conversation-level experience judge that asks whether the assistant moved the conversation forward without redundancy: consistent progress toward the user's goal, no repeated tool calls with identical parameters, no restating of information already communicated, retention of established facts, and well-formed clarification questions. Unlike Faithfulness, which scores the *correctness* of the assistant's choices, conversation progression scores the *efficiency* of those choices — a run that arrives at the right outcome but loops or re-asks for known information will receive a low progression score even if everything it says is faithful.

Judge and Prompt Template.The judge is run as a single LLM-as-judge call per conversation, using *GPT-5.2*. The prompt template is available in Appendix M.4.

Inputs.Compared to faithfulness, conversation progression deliberately operates on a smaller input bundle, because policy reasoning is explicitly out of scope:

- the linearized `conversation_trace`;
- the pipeline-aware shared fragments documenting how user and assistant turns are sourced and how interruption tags are used (Appendix M.1);
- one progression-specific pipeline-aware fragment, `information_loss_pipeline_note`, described below.

The judge does *not* receive the agent role, instructions, available-tools schema, or current date/time: those drive faithfulness/policy reasoning, and giving the progression judge

access to them invites it to silently re-litigate faithfulness questions under a different label. The prompt enforces this scope explicitly — if an issue is primarily a policy or faithfulness violation (e.g. taking an action the user said not to, not disclosing a fee), the judge is instructed to leave it to faithfulness even if the violation also affects conversational flow; only issues where the assistant's *conversational choices* (questions asked, information repeated, tools called) are themselves inefficient should be flagged here.

Failure modes.The judge scores four disjoint dimensions:

1. `unnecessary_tool_calls` — the assistant called a tool without justification: same tool with the same parameters after a successful prior response, a tool with empty/missing required parameters that produced a predictable error, or a tool whose result was already available from a previous response. As a hard caveat in the rubric, three or more unnecessary tool calls in a run automatically rates this dimension at the lowest level.
2. `information_loss` — the assistant failed to retain or act on an established fact: re-asked the user for information already provided, ignored a constraint the user explicitly stated, or failed to use a value returned by a prior tool when needed for the next step. The dimension is about *forgetting or ignoring* known facts; if the assistant proceeded against a user-stated preference deliberately, that is a faithfulness issue and is excluded from this dimension.
3. `redundant_statements` — the assistant restated information it had already communicated to the user (repeated explanations, repeated status updates, multiple recaps across non-final turns). The exception is a single recap at the very end of the conversation, which the rubric explicitly allows.
4. `question_quality` — the assistant's questions were poorly formed or missing: overly broad/vague questions when enough information was on hand to act, multiple questions bundled in a turn that a single tool call could resolve, missing clarifications when input was genuinely ambiguous, or proceeding to an irreversible action without confirming an ambiguous value. Standard policy-required readbacks of error-prone values (alphanumeric IDs, codes, dates, amounts) are explicitly *not* flagged here.

Two cross-cutting carve-outs apply to all four dimensions. First, an *interruption-tag carve-out* (Appendix E.1): truncated speech caused by an interruption is not a progression issue per se; only its observable downstream consequences (information genuinely lost because the cut-off content was never restated, or the assistant repeating already-heard content after being interrupted) are flagged. Second, a *voice-context carve-out*: when the assistant repeats a request because the previous attempt was clearly misheard or garbled, that repetition is expected behaviour in a voice interface and is not a progression issue — but only when the transcript shows visible evidence of an ASR failure on the prior attempt; re-asking without cause is still a flag.

Pipeline-aware adaptations.Conversation progression follows the general framing of Appendix E.2 and adds one progression-specific delta. The `information_loss_pipeline_note` explicitly scopes the `information_loss` dimension for audio-native pipelines: because assistant turns in those traces are post-hoc ASR of the assistant audio, variant token-level readings of the same alphanumeric identifier across nearby assistant turns (dropped/added dashes, single-character substitutions, missing/extra digits within long IDs, altered spacing or capitalisation) typically reflect TTS-rendering or post-hoc-ASR artefacts on a value the agent is reading consistently in audio. Such surface variance is scored by AgentSpeechFidelity, not here; only structural or semantic discrepancies (different entity, wrong field, wrong category — e.g. addressing the user by an entirely different first name or referencing a different person/record than the tool returned) or downstream signals indicating the agent was operating on a wrong value (subsequent tool calls with a wrong parameter, follow-up actions on stale data, user objections that the agent then fails to incorporate) are flagged on this dimension. In cascade, this note is empty: assistant turns are intended TTS text, so any inconsistency between two assistant utterances of the same fact *is* attributable to the LLM. The voice-context carve-out described above also lands somewhat differently across architectures: in cascade, the trace exposes the actual STT artefacts that justify a re-ask, so the carve-out is concrete; in audio-native pipelines, the trace shows the user simulator's clean intended text, so re-asks may look less self-evidently justified, and the carve-out instead reminds the judge that mishearings are still possible at the audio layer even when not visible in the trace.

Rating scale.Each dimension is rated on a 3-point integer scale:

- 3 — no issue on this dimension.
- 2 — a single isolated issue that does not significantly impact conversation flow (e.g. one unnecessary tool call that didn't slow things down, a single redundant restatement, one vague question), or a borderline case where it is unclear whether the issue constitutes a real progression problem.
- 1 — multiple instances of the same type of issue in this dimension, or a single severe issue that clearly derailed or stalled the conversation (e.g. ignoring a stated user constraint before a write operation, failing to ask for required information before taking action, asking an overly vague question when the user's goal was clear).

Unlike faithfulness, the dimension-level ratings are *not* aggregated by simple minimum, because conversational efficiency is more sensitive to the breadth of issues than to a single worst dimension: a run that has one minor question-quality issue is qualitatively different from a run that has one minor issue in each of three dimensions, even though the per-dimension minimum is the same. The overall rating is therefore:

- 3 if no dimension is flagged (all dimensions rated 3);
- 2 if one or two dimensions are flagged at rating 2 *and* no dimension is rated 1;
- 1 if any dimension is rated 1, *or* if three or more dimensions are flagged — the latter

captures the case where issues are individually minor but spread across many areas, which the rubric treats as a clear overall progression problem.

The overall rating is normalized to $[0, 1]$ via $(r - 1)/2$, giving $3 \to 1.0$, $2 \to 0.5$, $1 \to 0.0$. The judge's per-dimension JSON output and the count of flagged dimensions are preserved in the metric details for inspection.

Pass/fail thresholding.As for faithfulness (Appendix E.4.2), we count a conversation as a conversation-progression pass if its overall rating is ≥ 2, i.e. its normalized score is ≥ 0.5. The threshold again sits at the rubric's own load-bearing boundary: rating 1 is reserved for runs where progression *materially* broke down, while rating 2 covers a small number of isolated minor inefficiencies that did not impede the outcome; conversations at rating 2 are still treated as acceptable.

E.5.2 Conciseness

What it measures. Conciseness is an experience metric that asks whether each assistant turn is appropriately brief and voice-appropriate: a listener consuming the response in real time should be able to absorb its content in a single pass, without filler, without excessive enumeration, and without information density beyond what working memory can comfortably retain. Unlike Faithfulness and Conversation Progression, which produce one rating per conversation, conciseness is rated *per turn*: each assistant's turn is scored independently, and the per-conversation score is the mean of those per-turn ratings.

Judge and Prompt Template.The judge is run as a single LLM-as-judge call per conversation (one call returning a per-turn array), using *GPT-5.2*. The prompt template is available in Appendix M.5.

Inputs.The judge consumes a deliberately minimal bundle:

- `conversation_turns` — the linearised conversation trace, grouped by `turn_id`, including user, assistant, tool-call, and tool-response entries. The non-assistant entries are provided as context only; the judge rates only assistant content. Multiple assistant entries within a single turn (e.g. a partial response, a tool call, then a continuation) are explicitly evaluated together as a single unit;
- `interruption_tags_reference` — the shared interruption-tag glossary (Appendix M.1), with a strong instruction not to penalise truncated or fragmented content caused by interruptions.

Notably, Conciseness does *not* receive the pipeline-aware `user_turns_disclaimer` or `assistant_turns_disclaimer`, as they don't have a significant impact on Conciseness assessment.

Failure modes.When a turn is rated below 3, the judge tags it with one or more of the following (a turn may carry multiple tags):

1. **`verbosity_or_filler`** — unnecessary wording, hedging, or repetition within the same turn beyond what the context requires.

2. `excess_information_density` — too many distinct facts, options, numbers, steps, or requests packed into one turn for a listener to retain in real time. Bundling closely related transactional details that the user must act on together (e.g. confirming a reference number, date, and one or two key details in a single turn) is explicitly *not* flagged — only volume that genuinely exceeds working-memory limits.
3. `over_enumeration_or_list_exhaustion` — reading out long lists exhaustively rather than summarising, or presenting multiple options with excessive per-option detail rather than inviting follow-up.
4. `contextually_disproportionate_detail` — more background, clarification, or explanation than the situation actually warrants, given what the user asked for.

The rubric also defines an explicit set of *allowed exceptions* that are never penalised on this metric, even when they produce longer turns: phonetic spell-out of confirmation codes (NATO alphabet) when clarification is needed, full delivery of reference/identifier values the user needs to note down (ticket numbers, voucher codes), and a slightly longer end-of-call recap or wrap-up. Truncated content caused by user or assistant interruptions is also exempt, mirroring the carve-out used by the other judges.

Pipeline-aware adaptations.Conciseness has none. As noted above, the metric only rates assistant content, and the cascade-vs-audio-native distinction in user-side text sourcing (Appendix E.2) is therefore immaterial to its judgements. The interruption-tag carve-out is the only piece of pipeline-derived context the prompt makes use of, and it is shared with the other judges via the same `interruption_tags_reference` fragment.

Rating scale and aggregation.Each assistant-bearing turn is rated on a 3-point integer scale:

- 3 (highly concise) — the response is clear, appropriately scoped for voice, and comfortably digestible in real time. No failure modes are present. A turn that delivers a few closely related facts as part of a single transactional step still qualifies as 3 if a listener can absorb it in one pass.
- 2 (adequate but not optimally concise) — exactly one minor failure mode is present, but the response remains processable in a voice setting and does not meaningfully overwhelm the listener. Reserved for turns where one can identify specific content that should have been omitted or deferred — not merely for turns that happen to contain several necessary details.
- 1 (not concise) — one or more significant failure modes are present that would materially increase cognitive load and hinder comprehension when spoken.

Each per-turn rating is normalised to $[0, 1]$ via $(r - 1)/2$, giving $3 \to 1.0$, $2 \to 0.5$, $1 \to 0.0$, and the conversation-level Conciseness score is the *mean* of these per-turn normalised ratings across all rated turns. Unlike Faithfulness (minimum across dimensions) and Conversation Progression (count-aware aggregation across dimensions), Conciseness is therefore a continuous score in $[0, 1]$ rather than a discrete one in $\{0.0,\ 0.5,\ 1.0\}$. Per-turn ratings, failure-mode tags, and per-turn explanations are preserved in the metric details for inspection,

and per-failure-mode rates (the fraction of rated turns flagged with each tag) are surfaced as sub-metrics.

Pass/fail thresholding.Following the same convention as Faithfulness and Conversation Progression (Appendices E.4.2, E.5.1), we count a conversation as a Conciseness pass if its conversation-level normalised score is ≥ 0.5, i.e. if its mean per-turn rating is ≥ 2. Because Conciseness aggregates by mean rather than by minimum, this threshold has a slightly different rubric-level interpretation: a pass means the assistant was on average at least "adequate but not optimally concise" across the conversation, with isolated rating-1 turns (significant verbosity in a small number of turns) tolerated as long as they are offset by rating-3 turns elsewhere. The threshold is intentionally lenient: verbosity degrades the listening experience but does not cause the material harm that the rating-1 cut captures for Faithfulness and Conversation Progression.

E.5.3 Turn-taking

Unlike prior works [? ?] that treat turn-taking as a collection of independent flat scalars across all turns — conflating qualitatively distinct events and penalizing tool-call latency equivalently to slow conversational responses — our Turn-Taking metric introduces two key distinctions: (1) tool-call-aware Turn-Taking evaluation, which applies more adaptive latency thresholds to turns involving or not involving tool execution, decoupling architectural latency from conversational responsiveness; and (2) a unified per-turn score that routes each turn to a semantically appropriate scoring function conditioned on what actually occurred — penalizing agent interruptions based on overlap severity and recovery latency, rewarding immediate agent yield on user interruptions, and scoring uninterrupted turns on a principled response latency curve — producing a single, interpretable score that reflects the full diversity of turn-taking events in task-oriented voice interaction. The metric is deterministically computed from the event timestamps and latencies recorded in the simulation logs.

Unified Per-Turn Scoring Regime

Rather than aggregating flat scalars across all turns, each turn is first classified by its interrupt condition and then routed to a semantically appropriate scoring function. This ensures that qualitatively distinct turn-taking events — agent interruptions, user interruptions, and uninterrupted exchanges — are each evaluated according to the behavioral properties that matter most for that event type. The major event types and their corresponding score functions are summarized in Table 16.

Uninterrupted TurnsFor uninterrupted turns, Turn-Taking score is computed from the agent's response latency ℓ (ms), defined as the elapsed time between the end of the user's utterance and the onset of the agent's response. This latency is mapped to a score in $[0, 1]$ via a piecewise-linear curve encoding four regions as detailed below:

- Hard-zero early ($\ell \leq \ell_{\text{hard-early}}$): The agent begins speaking before the user finishes their

TABLE 16 Per-turn scoring regime. Each turn is routed to a semantically appropriate scoring function conditioned on the interrupt condition, rather than contributing to a single aggregated scalar.

Turn Condition	Score Function
Agent interrupted user	$s_{\text{agent}} = \min(s_{\text{overlap}}, s_{\text{count}}, s_{\text{post}})$
User interrupted agent	s_{yield}
Both	$\min(s_{\text{agent}}, s_{\text{yield}})$
Uninterrupted	s_{latency}

utterance, indicating a premature interruption. 500 ms is set conservatively relative to the perceptual detection threshold for overlapping speech of approximately 120 ms [?], acknowledging that voice agent deployments introduce audio buffering and streaming artifacts that can produce small spurious negative latencies. Score is hard-clamped to 0 regardless of how early the response is.

- Early ramp ($\ell_{\text{hard-early}} < \ell \leq \ell_{\text{sweet-low}}$): The response arrives before the natural conversational window but is not severely premature. While the modal inter-turn gap in human conversation falls between 0-200 ms [? ?], voice agents might be subject to additional processing overhead, making 500 ms the realistic minimum achievable latency of a well-optimized voice agent system. Score ramps linearly from 0 to 1 as latency increases toward the optimal window, penalizing responses proportionally to how early they arrive.
- Sweet spot ($\ell_{\text{sweet-low}} < \ell \leq \ell_{\text{sweet-high}}$): The response arrives within the optimal window for natural conversational flow, consistent with psycholinguistic norms for inter-turn gaps [? ?]. The upper bound of 2,000 ms is motivated by evidence that gaps exceeding 700–1,000 ms are perceived as problematic [? ?], while telephony contexts tolerate slightly longer delays than face-to-face interaction [?]. Score is flat at 1 across the entire time.
- Late ramp ($\ell_{\text{sweet-high}} < \ell \leq \ell_{\text{hard-late}}$): The response begins to feel delayed, introducing noticeable silence that disrupts conversational rhythm. At latencies approaching 2,000 ms, users in voice agent interactions begin to check in or repeat themselves [? ?]; the ramp to 3,500 ms provides a graduated penalty for responses that are slow but not yet conversation-ending. Score ramps linearly from 1 to 0.
- Hard-zero late ($\ell > \ell_{\text{hard-late}}$): The silence is long enough to cause conversational breakdown, likely prompting the user to disengage from the conversation any further [?]. Score is hard-clamped to 0.

The breakpoints defining these regions are detailed in Table 17, which presents both standard and tool-call-aware variants discussed in Section E.5.3.

Agent Interruption ScoreWhen the agent interrupts the user, the score is governed by three sub-dimensions that jointly capture the severity and quality of the interruption. All three are capped at $M = 0.5$, reflecting the principle that an interruption is never cost-free

TABLE 17 Latency curve breakpoints by turn type, with descriptions. Tool-call turns receive a more lenient upper threshold ($\ell_{\text{sweet-high}}$ and $\ell_{\text{hard-late}}$) to account for inherent tool execution latency; lower breakpoints are unchanged since early-response behavior is unaffected by tool calls.

Breakpoint	Standard Turn	Tool-Call Turn	Description
$\ell_{\text{hard-early}}$	−500 ms	−500 ms	Response begins before the user finishes speaking; score drops to 0 at or below this threshold.
$\ell_{\text{sweet-low}}$	500 ms	500 ms	Lower bound of the optimal response window; score reaches 1 at this point.
$\ell_{\text{sweet-high}}$	2,000 ms	3,000 ms	Upper bound of the optimal response window; score begins ramping down beyond this threshold.
$\ell_{\text{hard-late}}$	3,500 ms	5,000 ms	Response is considered excessively delayed; score drops to 0 at or beyond this threshold.

regardless of its brevity or recovery quality:

$$s_{\text{agent}} = \min\left(s_{\text{overlap}},\ s_{\text{count}},\ s_{\text{post}}\right) \tag{1}$$

User Interruption ScoreWhen the user interrupts the agent, the score measures how promptly the agent yields, rewarding agents that stop speaking immediately. Yield latency Δt is computed across the turn boundary (agent's last-end in turn $t{-}1$ minus user's first-start in turn t), and is linearly penalized up to a maximum tolerable yield duration $\Delta t_{\max}$, beyond which the score reaches 0:

$$s_{\text{yield}} = \max\left(0,\ 1 - \frac{\Delta t}{\Delta t_{\max}}\right) \tag{2}$$

where $\Delta t_{\max} = 2{,}000$ ms defines the upper bound of the acceptable yield latency. An agent that continues speaking beyond this threshold after the user begins is considered to have failed to yield entirely, resulting in a score of 0. Unlike the agent interruption score, s_{yield} is not capped at 0.5: an agent that yields immediately receives a perfect score of 1, reflecting that deferring to the user is always the correct behavior when interrupted.

Tool-Call-Aware Turn-taking Score

A uniform latency threshold treats all response delays equally — yet in task-oriented voice agents, a delay caused by tool execution is fundamentally different from a delay caused by slow conversational processing. Applying the same thresholds to both would unfairly penalize agents for latency they cannot avoid, conflating infrastructure overhead with genuine conversational responsiveness. To address this, we introduce tool-call-aware latency thresholds: turns in which the agent issued a tool call before responding receive more lenient upper breakpoints ($\ell_{\text{sweet-high}}$ and $\ell_{\text{hard-late}}$), extending both the optimal response

TABLE 18 Agent interruption sub-scores, each capped at $M = 0.5$. The minimum of the three is taken, ensuring that the weakest dimension dominates — a single poorly-recovered interruption cannot be masked by favorable scores on the other dimensions. $o_{\max} = 2{,}000$ ms defines the maximum tolerable overlap duration beyond which $s_{\text{overlap}} = 0$; $N_{\max} = 3$ defines the maximum number of distinct overlapping segments beyond which $s_{\text{count}} = 0$.

Sub-score	Definition	Formula
Overlap	Total simultaneous-speech duration o (ms) across all user/agent segment pairs, penalized up to a maximum tolerable overlap duration $o_{\max}$, beyond which the agent is considered to have unacceptably interrupted the user.	$s_{\text{overlap}} = \max\left(0,\ M\left(1 - \frac{o}{o_{\max}}\right)\right)$
Count	Number of distinct agent segments n overlapping user speech (each with >1 ms intersection), penalized as interruption frequency increases toward the maximum tolerable segment count $N_{\max}$.	$s_{\text{count}} = \max\left(0,\ M\left(1 - \frac{n-1}{N_{\max}-1}\right)\right),\ n \geq 1$
Post-interrupt	Silent gap from the end of the user's last segment to the start of the agent's next settled response, measuring recovery quality after an interruption. Scored via the latency curve (Table 17). Omitted when no settled response exists or the agent was still speaking at the user's last segment end.	$s_{\text{post}} = s_{\text{latency}}(\Delta t)$

window and the tolerable delay ceiling to absorb the cost of tool execution. The lower breakpoints ($\ell_{\text{hard-early}}$ and $\ell_{\text{sweet-low}}$) remain unchanged, since early-response behavior is independent of whether a tool call was issued. This decoupling ensures that the Turn-Taking score reflects genuine conversational responsiveness, enabling fair and meaningful comparison across systems with different tool execution speeds and architectures. The full set of breakpoints for both standard and tool-call turns is provided in Table 17.

Pass/fail thresholding.

The EVA-X composite admits a trial only if its Turn-Taking score clears a pass-threshold τ_{tt}, set to 0.8 in this work. Unlike other metrics—e.g., Faithfulness and Conciseness, which are judged on coarse ordinal scales whose thresholds follow directly from their rubric definitions—turn-taking scores are continuous and lack such an intrinsic cutoff. We therefore justify $\tau_{\text{tt}} = 0.8$ on three grounds: its conversational interpretation, its calibration to current model capability, and the empirical finding that EVA-X model rankings remain mostly stable across threshold values.

Conversational interpretation: $\approx 0.804/5$ on-time turns.The per-record Turn-Taking score is the mean of per-turn scores in $[0, 1]$, where each turn earns a score of 1 when its latency is acceptable and a correspondingly lower score outside it. A threshold of 0.8 thus admits at most one off-bracket turn per five—a concrete, conversation-level reading.

The on-time bracket is already lenient.Natural human turn-taking typically centers around 200,ms [? ?], yet EVA's latency thresholds are deliberately permissive relative to this norm as discussed in Section E.5.3). Because the per-turn turn-taking scoring is already permissive about which turns count as on-time, a low τ_{tt} would make the metric trivially easy to pass, compounding the two layers of leniency and reducing the metric's ability to discriminate between systems at the higher end of the performance range.

Calibration to current turn-taking capability.Among the 12 systems benchmarked here (Tab. 25), a wide gap separates systems that achieve real-time turn-taking from those that do not. The three S2S systems score 0.815–0.830; the next-highest system scores 0.583, a gap of more than 0.23. Any threshold in the range $[0.6, 0.8]$ would likely therefore produce the same binary partition of systems—the precise value does not determine which systems pass or fail given the current landscape.

EVA-X model rankings remain mostly stable across τ_{tt}.We sweep $\tau_{tt} \in \{0.50, 0.55, \ldots, 0.95\}$ and recompute EVA-X pass@1 at each value (Fig. 4). System rankings are preserved across the entire range: Spearman $\rho = 0.968$ between rankings at $\tau_{tt} = 0.5$ and $\tau_{tt} = 0.95$, and 61/66 (92.4%) of pairwise system orderings hold strictly across all ten thresholds (63/66, 95.5%, when statistically tied pairs are counted as preserved). The three pairs that flip somewhere in the sweep are systems that score within 0.15 pass@1 of each other.

Linear stability of the per-system pass@1 *vector* around the production anchor $\tau_{tt} = 0.8$ is correspondingly high: Pearson $r = 0.998$ at $\tau_{tt} = 0.75$ and $r = 0.995$ at $\tau_{tt} = 0.85$; r never falls below 0.910 across the entire $[0.50, 0.95]$ range (minimum at $\tau_{tt} = 0.50$).

TABLE 19 Pearson correlation between per-system EVA-X pass@1 vectors at $\tau_{tt} = 0.8$ and other candidate thresholds ($n = 12$ systems).

τ_{tt}	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95
Pearson r vs. $\tau_{tt} = 0.80$	0.910	0.947	0.967	0.979	0.991	0.998	1.000 (anchor)	0.995	0.980	0.950

Why 0.8?The sensitivity sweep confirms that neither plateau nor cliff appears at any candidate threshold. More importantly, the 0.23-point gap between the highest non-S2S system and the lowest S2S system means that any $\tau_{tt} \in [0.6, 0.8]$ yields similar pass/fail assignments for all 12 systems. We select the top of this range for two reasons: (i) it maps cleanly to the "4/5 turns on-time" interpretation, and (ii) it avoids compounding two layers of leniency, given that the on-time bracket itself already extends beyond natural human turn-taking latency. A higher threshold (e.g., 0.9) is currently miscalibrated to model capability—no system in our pool would reliably qualify.

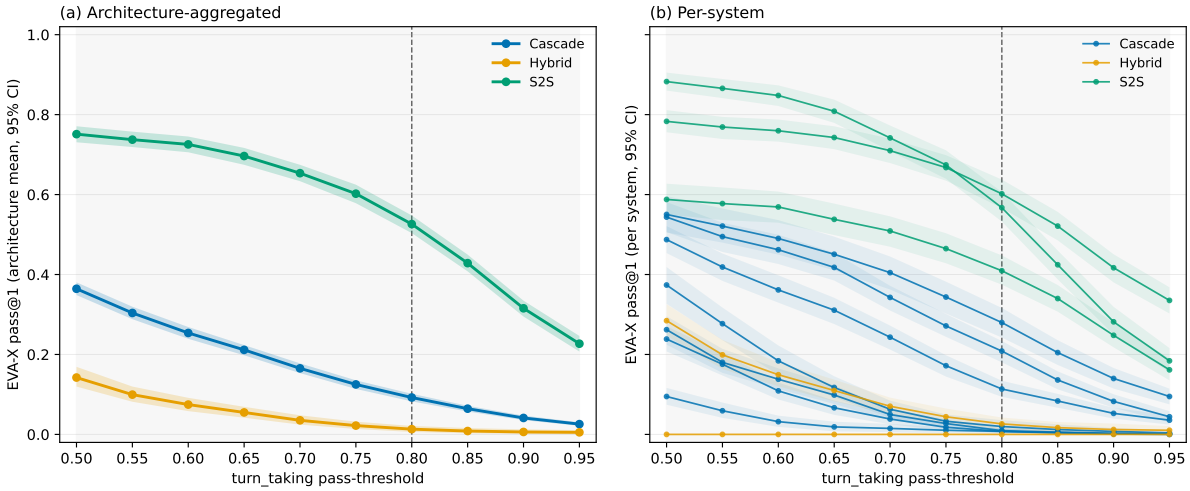


FIGURE 4 Sensitivity of EVA-X pass@1 to the turn-taking pass-threshold τ_{tt}. We sweep τ_{tt} from 0.50 to 0.95 in 0.05 increments and recompute EVA-X pass@1 at each value, holding the Conversation Progression and Conciseness thresholds fixed at 0.5. The dashed vertical line marks the pass threshold value $\tau_{\text{tt}} = 0.8$. (a) Architecture-aggregated. Each curve is the mean per-scenario EVA-X pass@1 (averaged over $k = 5$ trials per scenario), pooled with equal weight across all (system, scenario) pairs within an architecture class (cascade, hybrid, S2S); shaded bands are percentile bootstrap 95% CIs from 1,000 resamples of (system, scenario) pairs. (b) Per-system. One curve per benchmarked system, colored by architecture; shaded bands are bootstrap 95% CIs from 1,000 resamples of scenarios within the system. Architecture ordering (S2S > cascade > hybrid) is preserved at every threshold; per-system rankings flip only between close competitors.

Modularity and forward compatibility.EVA exposes `pass_at_k_threshold` as a configurable metric parameter, and practitioners benchmarking different system classes are encouraged to override the default. The 0.8 value reflects 2026 model capability; we expect this default to rise as model latencies improve. The defense above is therefore a statement about the *current* calibration, not a fixed property of the metric.

E.6 Diagnostic Metrics

Diagnostic metrics include: *Authentication Success Rate* (deterministic), *Response Latency* in seconds decomposed into turns with and without tool calls or not (deterministic), speakability — whether agent text is voice-friendly prior to synthesis (LLM-as-Judge), STT word error rate computed via `jiwer` (deterministic), tool call validity — the fraction of tool calls with correctly formatted parameters (deterministic), and transcription accuracy for key entities — STT accuracy specifically on named entities such as names, dates, and alphanumeric codes (LLM-as-Judge). Together these metrics allow practitioners to distinguish, for example, between a task completion failure caused by an STT transcription error on a confirmation code versus one caused by incorrect LLM reasoning.

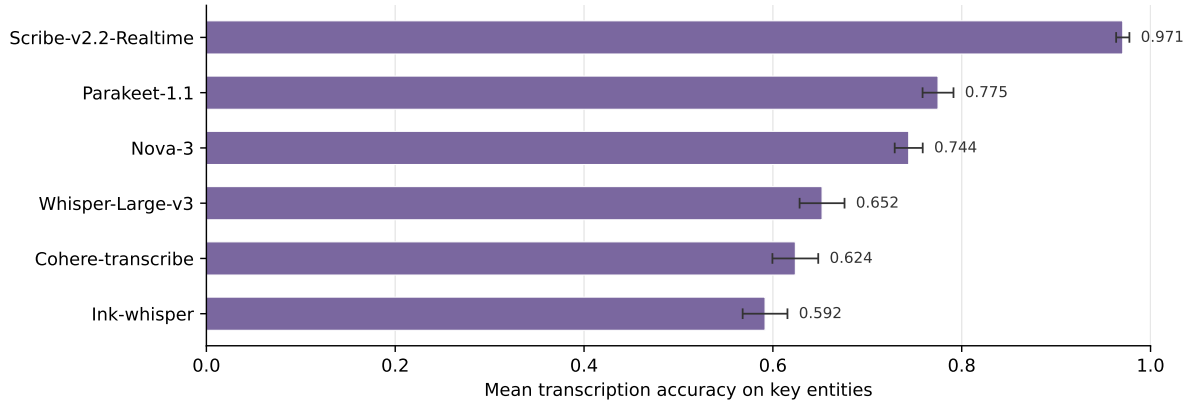


FIGURE 5 Mean transcription accuracy on key entities per cascade STT model, pooled across the three EVA-Bench domains. Each bar is the mean of per-scenario accuracies (one value per scenario, averaged over k=5 trials); whiskers are 95% normal-approximation CIs on that mean. Models are sorted ascending. Nova-3 aggregates the two cascades that share it (Nova-3 – GPT-5.4 – Sonic 3 and Nova-3 – GPT-5.4-mini – Aura-2), giving it twice as many scenario observations and a correspondingly tighter interval. Scribe-v2.2-Realtime saturates near 1.0 on this benchmark; the gap to the next-best STT (Parakeet-1.1) exceeds 0.19.

F Metrics Analysis

F.1 Key Entity Transcription Accuracy & Task Completion

The domains in EVA-Bench are entity-dense — agents must extract confirmation codes, names, authentication codes, employee IDs, and similar information not only to authenticate users, but also to complete downstream task steps (e.g., looking up a ticket, updating a record, or routing a request). This explains the correlation shown in Figure 6: since authentication is a prerequisite for nearly every task, and both authentication and task execution depend on accurate entity transcription, transcription accuracy and task completion are tightly coupled. This relationship may not generalize to domains with fewer key entities or where task success is independent of entity transcription. That said, we consider entity-heavy flows a representative and common voice agent use case, which is why our tasks and domains are designed to stress-test this capability. Transcription accuracy on entities is not sufficient on its own to predict task completion, however. Among the two systems using Nova-3 as the STT model, transcription scores are similar, yet the system backed by GPT-5.4 achieves meaningfully higher task completion than the one using GPT-5.4-mini—indicating that LLM reasoning over transcribed entities also plays a role.

F.2 Faithfulness & Task Completion

Table 20 shows the confusion matrix between Task Completion and Faithfulness (score of 1.0 vs. below 1.0) across all evaluated systems under clean conditions. Only 14.5% of trials achieve both, underscoring that Task Completion alone is a weak proxy for overall agent accuracy. Faithfulness failures are distributed nearly uniformly across task outcomes

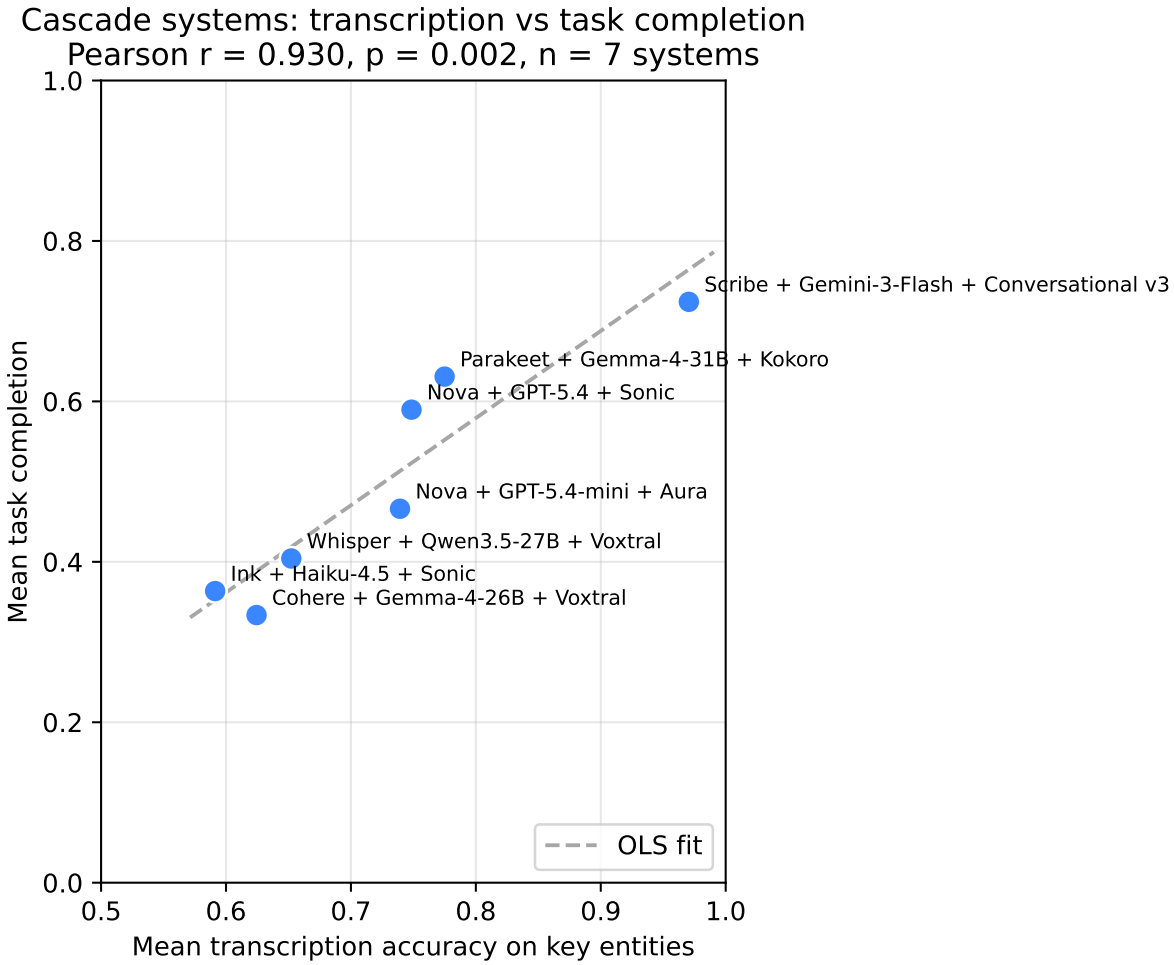


FIGURE 6 Mean key entity transcription accuracy and mean task completion correlation across the 7 evaluated cascade systems.

(38.3% vs. 37.6%), reinforcing that the two dimensions capture distinct aspects of agent behavior.

E.3 Patterns Across Pipeline Architectures

The S2S systems we evaluated tend to lead cascade on responsiveness facets — though by a narrow margin against the strongest cascades — and trail on policy adherence. We focus on four facets that surface these patterns most clearly: on-time turns (↑, a sub-metric of turn-taking), conversation completion (↑), authentication success (↑), and policy violations (↓, derived from the faithfulness judge).

- On-time rate (↑) — a sub-metric of the Turn-Taking score (App. E.5.3). We report it directly because its definition is unambiguous: the share of agent turns whose response latency falls in $[200, 4000)$ ms, or $[200, 6000)$ ms when the turn involves a tool call. Note that the boundaries are not exactly the same as in the turn-taking definition, given that

TABLE 20 Joint distribution of task completion and faithfulness across 12,780 clean trials from all evaluated systems. Cells show % of total; shading scales with cell count (darker = more trials).

	faithfulness = 1.0	faithfulness < 1.0	Row total
task_completion = 0	9.5%	38.3%	47.9%
task_completion = 1	14.5%	37.6%	52.1%
Column Total	24.0%	76.0%	12,780

here we don't have the piecewise-linear curves.

- Conversation Completion (↑) — captures the failure mode where the agent fails to respond to a user turn, leading the conversation to time out.
- Authentication success (↑) — the proportion of conversations in which the agent successfully authenticated the user before proceeding to the main task. This facet is informative because it probes how well each architecture handles entity-dense input — IDs, names, dates of birth — without requiring a transcript-level evaluation.
- Policy violations (↓) — the dominant failure mode contributing to low faithfulness scores, since policy adherence requires strict instruction-following. Across all pipelines, 61% of conversations are flagged with at least one policy violation. Common cases include the agent performing write actions without explicit user authorization, or fabricating policies absent from its instructions.

For each facet, we compute a per-system mean by averaging across that system's per-domain values. The pipeline-type mean is the unweighted mean of its constituent system means, and its 95% percentile bootstrap interval is obtained by resampling those system means with replacement (10,000 iterations, fixed seed). Pairwise effect sizes $\Delta = \mu_A - \mu_B$ between pipeline types are reported with a percentile bootstrap interval obtained by independently resampling each group's per-system means. Differences are in percentage points ($\Delta \times 100$); metrics are bounded in $[0, 1]$. All numbers are computed on the clean (unperturbed) runs of the systems described in the main paper: 7 cascade, 3 S2S, and 2 hybrid, evaluated on all three domains, yielding 1,065 clean conversations per system.

Given the small number of systems per pipeline type, the analysis is descriptive: bootstrap intervals are reported as uncertainty bands, not as confidence intervals in the inferential sense. We do not claim these patterns generalize to systems beyond those we evaluated; we report them as observations that may motivate further study. For some facets, the difference visible at the pipeline-type level conceals notable within-class exceptions, particularly within cascade. We also refrain from drawing pipeline-type conclusions about hybrid systems: the two we evaluated diverge substantially from one another. We also performed the analysis stratified by domain (omitted here for space); the ordering of the three pipeline-type means is preserved in every domain.

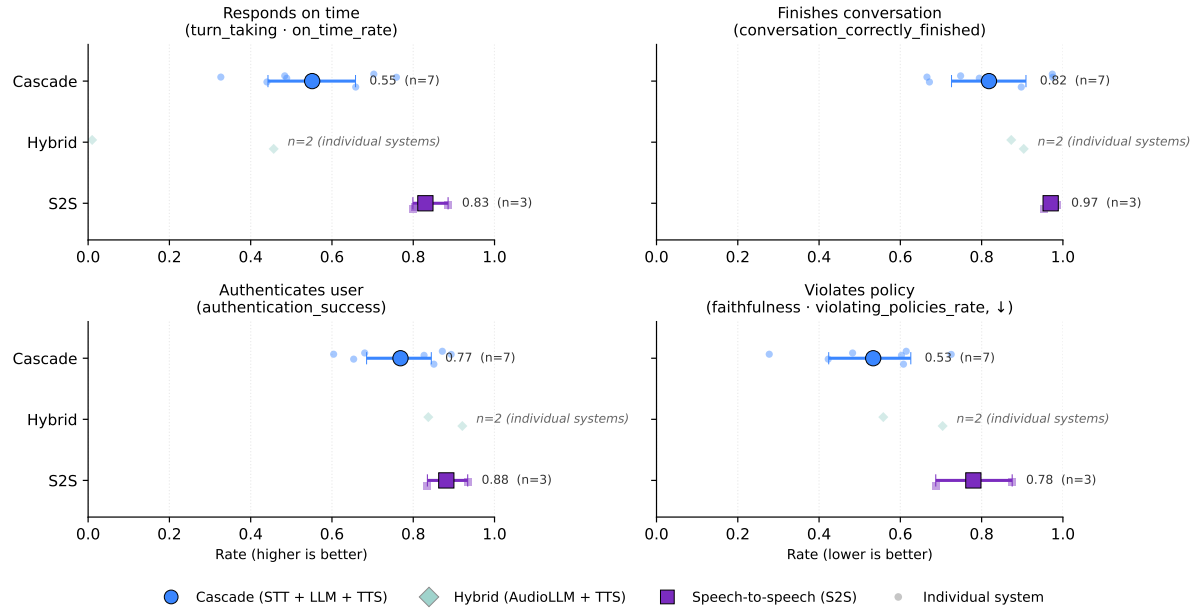


FIGURE 7 Per-pipeline-type means on the four facets, computed on clean runs only. Error bars are 95% percentile bootstrap intervals over systems-within-type, shown as descriptive uncertainty bands; faint dots are per-system means. Hybrid ($n = 2$) is shown as individual system points only.

F.3.1 Observations

TABLE 21 Per-pipeline-type means with 95% bootstrap intervals over systems-within-type, and the S2S−Cascade pairwise difference (Δ) with its 95% bootstrap interval. Computed on clean runs only. Higher is better for the first three facets; lower is better for `violating_policies_rate`. n is the number of systems contributing to each row. Mean rows use the $[0, 1]$ metric scale; the Δ row is reported in percentage points ($\Delta \times 100$), with its bootstrap interval obtained by independently resampling each group's per-system means.

		Facet (mean [95% interval])			
Pipeline type	n	On-time↑	Finishes↑	Auth.↑	Violates policy↓
Cascade	7	0.55 [0.44, 0.66]	0.82 [0.74, 0.93]	0.77 [0.66, 0.86]	0.53 [0.41, 0.65]
Hybrid	2	0.23 [0.01, 0.46]	0.89 [0.87, 0.90]	0.88 [0.84, 0.92]	0.63 [0.56, 0.70]
Speech-to-Speech	3	0.83 [0.80, 0.89]	0.97 [0.95, 0.99]	0.88 [0.83, 0.93]	0.78 [0.69, 0.88]
Δ S2S−Cascade (pp)	—	+27.9 [+16.4, +39.6]	+15.2 [+6.0, +24.5]	+11.3 [+2.4, +20.9]	+24.6 [+11.7, +38.4]

Table 21 reports the per-pipeline-type means with their 95% bootstrap intervals, and the S2S−Cascade pairwise difference (in percentage points) with its bootstrap interval as a bottom row; Fig. 7 visualizes the means with per-system points overlaid. The S2S systems sit at the top of all facets, including the policy-violation facet (where lower is better). However, there are some differences at the system level, as shown in Table 22, and the per-facet patterns below describe both where S2S systems excel or trail, and where individual cascades narrow or close the gap.

- On-time rate (↑) — all three S2S systems we evaluated sit above every cascade in our sample, but the gap is narrow for the strongest cascades: *Whisper-Large-v3 + Qwen3.5-27B + Voxtral-4B-TTS* (0.76) and *Cohere-transcribe + Gemma-4-26B + Voxtral-4B-TTS* (0.70) come within 4–10 pp of the lowest S2S system (0.80). The remaining five cascades trail more substantially (0.33–0.66).
- Conversation Completion (↑) — none of the S2S pipelines we evaluated struggled with completing conversations, whereas cascade is more variable. Only two cascade systems reached S2S-comparable values, *Scribe-v2.2-Realtime + Gemini-3-Flash + TTS-Conversational-v3-Alpha* and *Parakeet-1.1 + Gemma-4-31B + Kokoro* . Section F.3.3 examines the likely causes of agent-side failures, per pipeline.
- Authentication success (↑) — on raw scores, *GPT-Realtime-1.5* (0.93) and *Gemini-3.1-Flash-Live* (0.88) lead, while *GPT-Realtime-mini* (0.83) sits in the cascade mid-range, outperformed by *Scribe-v2.2-Realtime + Gemini-3-Flash + TTS-Conversational-v3-Alpha* (0.89), *Parakeet-1.1 + Gemma-4-31B + Kokoro* (0.87), and *Nova-3 + GPT-5.4 + Sonic 3* (0.85). Much of the apparent cascade-vs-S2S gap on this facet is a completion artifact, however: when restricted to conversations that finished, several additional cascades reach the S2S range (*Nova-3 + GPT-5.4 + Sonic 3* rises to 0.94, matching *GPT-Realtime-1.5*). See Sec. F.3.2.
- Policy violations (↓) — the most variable facet within both pipeline classes. *GPT-Realtime-1.5* (0.69, the lowest S2S violation rate) lands in the same range as several cascades (e.g., *Cohere-transcribe + Gemma-4-26B + Voxtral-4B-TTS* at 0.60, *Scribe-v2.2-Realtime + Gemini-3-Flash + TTS-Conversational-v3-Alpha* and *Parakeet-1.1 + Gemma-4-31B + Kokoro* at 0.61); the other two S2S systems sit higher. The lowest cascade violation rate comes from *Nova-3 + GPT-5.4 + Sonic 3* (0.28), and remains the lowest after conditioning on completed conversations (0.32). Conditioning matters here for the cascades that hang often: *Ink-whisper + Haiku-4.5 + Sonic 3* rises from 0.48 to 0.63 (+15 pp), revealing that its low raw rate partly reflects fewer opportunities to violate rather than stronger instruction-following. No S2S system reaches the conditional violation rate of *Nova-3 + GPT-5.4 + Sonic 3*, suggesting a tension between latency and instruction-following accuracy that none of the systems we evaluated fully resolves.

F.3.2 Authentication: completion vs. ability

Raw `authentication_success` mixes two distinct phenomena: how often the agent completes the authentication exchange without hanging, and how often it correctly authenticates the user when it does. Cascade pipelines frequently hang *during* the authentication flow — typically when the user reads out an ID or a short utterance that the pipeline mishandles. To separate these, Table 23 reports each system's authentication rate restricted to conversations that finished correctly.

S2S systems are essentially unchanged: they finish 95–99% of clean conversations, so the raw and conditional rates coincide ($\Delta \leq 0.8$ pp). Several cascades, by contrast, move

TABLE 22 Per-system means on clean runs, averaged across the three domains.

Pipeline	System	On-time↑	Finishes↑	Auth.↑	Violates↓
Cascade	Nova-3 – GPT-5.4 – Sonic 3	0.48	0.66	0.85	0.28
	Ink-whisper – Haiku-4.5 – Sonic 3	0.44	0.67	0.65	0.48
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.76	0.75	0.60	0.42
	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.70	0.79	0.68	0.60
	Nova-3 – GPT-5.4-mini – Aura-2	0.66	0.90	0.83	0.73
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.49	0.97	0.89	0.61
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.33	0.97	0.87	0.61
Hybrid	Ultravox-Realtime	0.46	0.87	0.84	0.70
	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.01	0.90	0.92	0.56
Speech-to-Speech	GPT-Realtime-1.5	0.80	0.99	0.93	0.69
	GPT-Realtime-mini	0.80	0.97	0.83	0.88
	Gemini-3.1-Flash-Live	0.89	0.95	0.88	0.78

substantially: *Nova-3 + GPT-5.4 + Sonic 3* rises from 0.85 to 0.94, matching the best S2S (*GPT-Realtime-1.5* at 0.94); *Ink-whisper + Haiku-4.5 + Sonic 3* from 0.65 to 0.86 (+20 pp); and *Whisper-Large-v3 + Qwen3.5-27B + Voxtral-4B-TTS* from 0.60 to 0.73 (+13 pp). The cascade authentication deficit visible in Table 21 is therefore in large part a completion artifact: when a cascade gets through the authentication exchange without timing out, several configurations authenticate at rates comparable to S2S systems. Among the systems we evaluated, *GPT-Realtime-1.5* stands out for combining both axes: it finishes 99% of clean conversations and authenticates correctly 93% of the time when it does, the highest values in the table on both axes.

TABLE 23 Per-system authentication success on clean runs, raw vs. conditional on conversation completion. *Auth (raw)* is computed over all 1,050 clean conversations; *Auth | finished* is computed only on conversations where `conversation_correctly_finished` = 1. Δ *(pp)* is their difference. Systems with low finish rates move the most.

Pipeline	System	Finish rate	Auth (raw)	Auth | finished	Δ (pp)
Cascade	Nova-3 – GPT-5.4 – Sonic 3	0.66	0.85	0.94	+8.8
	Ink-whisper – Haiku-4.5 – Sonic 3	0.67	0.65	0.86	+20.5
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.75	0.60	0.73	+12.6
	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.79	0.68	0.73	+4.9
	Nova-3 – GPT-5.4-mini – Aura-2	0.90	0.83	0.86	+2.9
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.97	0.89	0.90	+1.1
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.97	0.87	0.88	+0.3
Hybrid	Ultravox-Realtime	0.87	0.84	0.86	+2.0
	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.90	0.92	0.94	+2.4
Speech-to-Speech	GPT-Realtime-1.5	0.99	0.93	0.94	+0.5
	GPT-Realtime-mini	0.97	0.83	0.84	+0.8
	Gemini-3.1-Flash-Live	0.95	0.88	0.88	+0.7

F.3.3 Inactivity-timeout failures

To better understand why conversations do not always finish correctly, we examined every conversation that ended with an agent-side inactivity timeout. We classify each such

conversation by the immediately preceding user turn: *short turn* (under 5 words), *short ∧ confirm* (short turn opening with *yes/no/sure/ok*), and *spelled content* (the user turn contained NATO phonetic words, isolated letters or digits read out one-by-one, alphanumeric codes, or numbers spoken as words). A turn can match more than one pattern.

Table 24 breaks the timeouts down per system. Five of the seven cascade systems show short-turn shares of 52–68% of their timeouts; a substantial fraction of those short turns also begin with a confirmation token (e.g., "Yes, that's correct"), after which the agent fails to produce a follow-up turn. Several pipelines do not exhibit this pattern: *Scribe-v2.2-Realtime + Gemini-3-Flash + TTS-Conversational-v3-Alpha*, *Parakeet-1.1 + Gemma-4-31B + Kokoro*, *Ultravox-Realtime*, and all three S2S systems. With the exception of *Ultravox-Realtime*, all of these have an overall conversation failure rate below 5%. Spelled-content failures, by contrast, persist across all pipelines and are the dominant timeout cause among the systems that handle short turns well. These spelled-content hangs occur predominantly during the authentication exchange, which is also why the cascade authentication deficit shrinks substantially under the completion-conditional view (Sec. F.3.2).

TABLE 24 Per-system view on clean runs (1,065 conversations per system). *Conv. fail rate* = 1 − *conversation_correctly_finished* (any failure type, conversation-pooled). *Timeouts* = number of those failures that ended with the agent going silent. The three right-most columns are the share of *this system's* timeouts whose preceding user turn matched each pattern. Systems are ordered within each pipeline class by timeout count (descending).

Pipeline	System	Conv. fail rate	Timeouts	Short turn	Short ∧ confirm	Spelled content
Cascade	Ink-whisper – Haiku-4.5 – Sonic 3	36.5%	395	63%	43%	17%
	Nova-3 – GPT-5.4 – Sonic 3	35.8%	381	68%	39%	12%
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	28.2%	300	62%	52%	14%
	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	22.6%	241	56%	37%	21%
	Nova-3 – GPT-5.4-mini – Aura-2	10.7%	117	52%	20%	24%
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	2.6%	28	0%	0%	68%
	Parakeet-1.1 – Gemma-4-31B – Kokoro	2.5%	27	19%	19%	48%
Hybrid	Ultravox-Realtime	14.0%	149	6%	1%	63%
	Gemini-3-Flash – Gemini-3.1-Flash-TTS	10.5%	112	30%	14%	46%
S2S	Gemini-3.1-Flash-Live	4.5%	52	2%	0%	58%
	GPT-Realtime-mini	2.9%	32	0%	0%	66%
	GPT-Realtime-1.5	1.4%	15	0%	0%	73%

F.4 Speech Fidelity

Across all models, entity mispronunciation (letter substitutions, digit omissions, spurious insertions, and phonetic confusions between similar-sounding characters) is by far the dominant failure class, accounting for the majority of flagged turns.

Substitutions.In a single alphanumeric code, one character is replaced by a visually or acoustically unrelated one. *Sonic 3* consistently rendered the airport code `LAX` as `LEX` (CSM, record 1.2.3) and substituted `S C 3` for `A C 3` in an ITSM request prefix across multiple turns (record 16). *Aura-2* substituted `L` for `I` in a medical license number, saying `L 1 C` instead of `L I C` (HRSD, record A6). *Ultravox-Realtime* misread the confirmation code `ZKLX8E` as `ZKLXIE`, substituting `8` for `I` (CSM, record 7.2.9), and rendered the hotel voucher

prefix GQSIHM as DQSIHM, substituting D for G (record 7.3.1).

Omissions.One or more characters are silently dropped from an entity. *GPT-Realtime-1.5* read the 10-digit NPI 3342331444 as only 9 digits, dropping the trailing 4 (HRSD, record 1.1), and omitted the 6 from registration ID 358607, producing 35807 (record 9.1). *Kokoro* dropped the A from the spelled-out code MEAL across multiple trials, producing MEL (CSM, record 2.2.5), and omitted the letter K from flight number SK915, rendering it as S 915 consistently across three consecutive turns (record 2.2.2). *Aura-2* omitted one of the five repeated digits, saying EMP-0-5-5-5-5 instead of EMP-0-5-5-5-5-5 (ITSM, record 22).

Insertions.Spurious characters or words are injected into an entity that should be reproduced verbatim. *Gemini-3.1-Flash-Live* inserted the name Pavel into the middle of the visa petition number EHZ3328710672, breaking the code entirely (HRSD, record 11.2). *Ultravox-Realtime* added a spurious E to the meal voucher prefix in multiple trials, saying MEAL-7EMMHTS-PAX0 instead of MEAL-7MMHTS-PAX0 with the E after the 7 (CSM, records 2.3.2). *Voxtral-4B-TTS* appended a trailing 3 to the request identifier REQ-SW-9befdac7c2e6, producing REQ-SW-9befdac7c2e63 (ITSM, record 76).

Phonetic confusions.A character is replaced by one that sounds similar when spoken aloud. *GPT-Realtime-1.5* repeatedly confused C and P, rendering the facility code IXC as IXP across two consecutive turns (HRSD, record D3.3), and swapped D for B in the policy number PDZP6L → PBZP6L (record T2.1). *Gemini-3.1-Flash-Live* confused Z and V, articulating the DEA prefix ZS as VS (HRSD, record A2). *Kokoro* persistently mispronounced SK130 as S-Cone 130 across two turns (record 4.2.4).

E.5 Turn-Taking & Response Speed

Table 25 reports per-system latency and turn-taking sub-metrics that feed into the aggregate turn-taking score, which in turn contributes to EVA-X pass@1. The early, on-time, and late columns give the share of agent turns falling into each timing bucket and sum to 1.0 per system; the response-speed columns report mean per-turn latency in seconds, broken down by whether the turn involved a tool call. A turn is classified as early when latency < 200 ms, late when latency ≥ 2.75 s (or ≥ 4 s if the turn involves a tool call), and on-time otherwise. Across systems the turn-taking score appears to track on-time and late rates more closely than early rate, which stays small for nearly every system — the largest observed early rate is 0.148 (*GPT-Realtime-mini*), and most systems sit below 0.06. Comparing response speed with vs. without tool calls, every system in the table shows a tool-call latency increase (no overlapping CIs) so the slowdown is consistent. The magnitude varies substantially, however: from roughly 0.1–0.2 s for the fastest cascades (*Whisper-Large-v3 + Qwen3.5-27B + Voxtral-4B-TTS*; *Ink-whisper + Haiku-4.5 + Sonic 3*) up to several seconds for *Ultravox-Realtime* and *Gemini-3-Flash + Gemini-3.1-Flash-TTS*, indicating that tool-call overhead is system-specific rather than a uniform property of the benchmark. These scores help illustrate what drives the difference in EVA-X pass@1 scores betewen cascade and S2S models.

TABLE 25 Per-system turn-taking and response-speed detail under clean-audio conditions, pooled equal-weighted across the three domains. Each cell shows the pooled point estimate ± the percentile bootstrap CI half-width ($\alpha = 0.05$). Turn-taking rates are in $[0, 1]$; response speed is mean per-turn latency in seconds. Cells are shaded per column with darker = better (lower for latency and error-rate columns; higher otherwise).

		EVA-X	Turn-Taking					Response Speed (s)		
Arch.	System	pass@1	Score	Early	On-time	Late	Interrupt.	Overall	w/ tools	w/o tools
Cascade	Cohere – Gemma-4-26B – Voxtral	0.209 ±0.027	0.567 ±0.024	0.113 ±0.009	0.703 ±0.015	0.184 ±0.014	0.110 ±0.009	2.415 ±0.107	2.660 ±0.253	2.171 ±0.077
	Scribe – Gemini-3-Flash – Conversational v3	0.024 ±0.018	0.451 ±0.019	0.008 ±0.002	0.489 ±0.022	0.504 ±0.022	0.006 ±0.002	4.158 ±0.118	5.294 ±0.166	2.420 ±0.119
	Ink-whisper – Haiku-4.5 – Sonic 3	0.009 ±0.006	0.312 ±0.019	0.024 ±0.004	0.440 ±0.015	0.536 ±0.016	0.023 ±0.004	3.399 ±0.045	3.535 ±0.081	3.354 ±0.060
	Nova-3 – GPT-5.4 – Sonic 3	0.007 ±0.006	0.283 ±0.021	0.029 ±0.005	0.484 ±0.017	0.487 ±0.018	0.028 ±0.005	3.943 ±0.112	5.191 ±0.189	2.914 ±0.078
	Nova-3 – GPT-5.4-mini – Aura-2	0.113 ±0.021	0.583 ±0.020	0.042 ±0.006	0.658 ±0.016	0.299 ±0.016	0.041 ±0.006	3.148 ±0.074	3.648 ±0.095	2.377 ±0.113
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.010 ±0.009	0.308 ±0.015	0.030 ±0.005	0.327 ±0.017	0.643 ±0.018	0.030 ±0.005	4.832 ±0.114	5.373 ±0.128	4.291 ±0.216
	Whisper – Qwen3.5-27B – Voxtral	0.273 ±0.035	0.561 ±0.029	0.056 ±0.008	0.759 ±0.015	0.186 ±0.012	0.054 ±0.008	2.251 ±0.050	2.309 ±0.069	2.164 ±0.067
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.000 ±0.000	0.019 ±0.003	0.017 ±0.004	0.010 ±0.003	0.973 ±0.005	0.017 ±0.004	7.466 ±0.257	9.100 ±0.413	5.244 ±0.189
	Ultravox-Realtime	0.029 ±0.019	0.417 ±0.021	0.040 ±0.007	0.457 ±0.019	0.503 ±0.020	0.040 ±0.007	4.838 ±0.225	6.994 ±0.301	1.393 ±0.063
S2S	Gemini-3.1-Flash-Live	0.589 ±0.034	0.830 ±0.017	0.011 ±0.005	0.886 ±0.012	0.104 ±0.012	0.011 ±0.005	2.288 ±0.070	2.846 ±0.088	1.288 ±0.086
	GPT-Realtime-1.5	0.566 ±0.040	0.815 ±0.013	0.121 ±0.012	0.799 ±0.016	0.079 ±0.009	0.072 ±0.011	1.798 ±0.061	2.395 ±0.089	1.011 ±0.031
	GPT-Realtime-mini	0.406 ±0.035	0.818 ±0.015	0.148 ±0.013	0.805 ±0.014	0.047 ±0.006	0.106 ±0.011	1.481 ±0.045	1.868 ±0.063	1.067 ±0.040

F.6 Cross Domain Variability

Tables 29 and 33 report, for every (system, metric) pair, the sample standard deviation (ddof=1) of the per-domain point estimates across the three EVA domains (CSM, ITSM, HR). These complement Tables 26–31 (per-domain values) and the pooled tables 2, by quantifying how much of the pooled headline number is hiding domain dispersion.

F.6.1 Metric-level findings

Speech rendering is essentially domain-invariant. Across all twelve systems, the cross-domain standard deviation of Speech Fidelity is between 0.002 and 0.013 (mean 0.007), against a point-estimate range of 0.913–0.996. No other metric comes within an order of magnitude of this stability. This is consistent with the fact that this metric is determined by the TTS/S2S audio path, which depends less on conversational content.

Within accuracy, Faithfulness is the most domain-coupled metrics. Mean standard deviations across systems are 0.087 for Faithfulness and 0.064 for Task Completion. Faithfulness shows the largest single standard deviation in either table (Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha : 0.184; Ink-whisper – Haiku-4.5 – Sonic 3 : 0.166).

Within experience, Conversation Progression is the noisiest metric and Conciseness the most stable. Mean standard deviations are 0.052 (Conversation Progression) versus 0.020 (Conciseness); Conversation Progression peaks at 0.125 (Ultravox-Realtime) and concise-

ness never exceeds 0.043 (Ultravox-Realtime). Turn-taking sits between them at mean SD 0.042. The pattern suggests that whether a system is terse is largely a system property, while whether it makes progress effectively depends on the conversational structure of the domain. EVA-X pass-rates are uniformly less variable across domains than EVA-A pass-rates. Mean standard deviations on EVA-X are 0.029 / 0.037 / 0.018 for pass@1/pass@k/pass^k, versus 0.074 / 0.101 / 0.050 on EVA-A — a roughly 2.5× gap on every variant. Even controlling for the floor effect by restricting to systems with non-zero EVA-X performance, EVA-A standard deviations remain the larger of the two. Domain choice perturbs task accuracy substantially more than it perturbs the experience overlay.

F.6.2 Model-level findings

Some systems pair high pooled scores with large task-completion swings, which the pooled tables conceal. *GPT-Realtime-1.5* reports a pooled task-completion mean of 0.739 ± 0.045 in Table 51, but its per-domain SD on the same metric is 0.174 — by far the largest task-completion SD in either table — meaning the pooled mean averages over substantial across-domain variation. *Scribe-v2.2-Realtime + Gemini-3-Flash + TTS-Conversational-v3-Alpha* shows the same effect on faithfulness (pooled 0.457; standard deviation 0.184).

TABLE 26 CSM domain accuracy metrics for all evaluated systems under clean-audio conditions. Each cell shows the point estimate ± the percentile bootstrap CI half-width ($\alpha = 0.05$). The three pass-rate columns share a single shading scale (so pass@1 vs. pass@k vs. pass^k are visually comparable); each submetric column is scaled independently. Darker = higher point estimate.

		EVA-A			Task Completion	Faithfulness	Speech Fidelity
Arch.	System	pass@1	pass@k	pass^k	Mean	Mean	Mean
Cascade	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.246 ±0.084	0.500 ±0.140	0.066 ±0.061	0.368 ±0.092	0.398 ±0.070	0.989 ±0.006
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.656 ±0.092	0.900 ±0.080	0.385 ±0.122	0.808 ±0.080	0.666 ±0.088	0.981 ±0.012
	Ink-whisper – Haiku-4.5 – Sonic 3	0.272 ±0.076	0.640 ±0.140	0.065 ±0.064	0.440 ±0.092	0.374 ±0.068	0.989 ±0.004
	Nova-3 – GPT-5.4 – Sonic 3	0.628 ±0.076	0.940 ±0.060	0.278 ±0.108	0.732 ±0.068	0.784 ±0.050	0.996 ±0.003
	Nova-3 – GPT-5.4-mini – Aura-2	0.216 ±0.096	0.460 ±0.140	0.098 ±0.081	0.456 ±0.092	0.298 ±0.078	0.979 ±0.009
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.540 ±0.096	0.880 ±0.100	0.294 ±0.111	0.672 ±0.096	0.600 ±0.076	0.966 ±0.019
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.272 ±0.071	0.680 ±0.140	0.045 ±0.049	0.504 ±0.096	0.470 ±0.072	0.918 ±0.017
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.488 ±0.092	0.840 ±0.100	0.188 ±0.089	0.696 ±0.088	0.478 ±0.080	0.965 ±0.012
	Ultravox-Realtime	0.324 ±0.100	0.540 ±0.140	0.131 ±0.081	0.428 ±0.104	0.334 ±0.076	0.971 ±0.015
S2S	Gemini-3.1-Flash-Live	0.356 ±0.104	0.660 ±0.140	0.170 ±0.102	0.504 ±0.096	0.342 ±0.084	1.000 ±0.000
	GPT-Realtime-1.5	0.424 ±0.116	0.640 ±0.140	0.271 ±0.120	0.540 ±0.112	0.424 ±0.092	0.998 ±0.003
	GPT-Realtime-mini	0.176 ±0.092	0.300 ±0.120	0.085 ±0.075	0.288 ±0.104	0.164 ±0.076	0.971 ±0.034

TABLE 27 HR domain accuracy for all evaluated systems under clean-audio conditions. Each cell shows the point estimate ± the percentile bootstrap CI half-width ($\alpha = 0.05$). The three pass-rate columns share a single shading scale (so pass@1 vs. pass@k vs. pass^k are visually comparable); each submetric column is scaled independently. Darker = higher point estimate.

		EVA-A			Task Completion	Faithfulness	Speech Fidelity
Arch.	System	pass@1	pass@k	pass^k	Mean	Mean	Mean
Cascade	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.229 ±0.070	0.386 ±0.108	0.087 ±0.049	0.318 ±0.089	0.408 ±0.055	0.985 ±0.005
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.373 ±0.082	0.614 ±0.108	0.199 ±0.074	0.655 ±0.080	0.320 ±0.066	0.973 ±0.009
	Ink-whisper – Haiku-4.5 – Sonic 3	0.239 ±0.063	0.482 ±0.108	0.052 ±0.033	0.318 ±0.077	0.700 ±0.052	0.982 ±0.006
	Nova-3 – GPT-5.4 – Sonic 3	0.422 ±0.080	0.651 ±0.108	0.213 ±0.073	0.496 ±0.087	0.740 ±0.053	0.986 ±0.005
	Nova-3 – GPT-5.4-mini – Aura-2	0.217 ±0.060	0.434 ±0.108	0.042 ±0.032	0.455 ±0.084	0.298 ±0.051	0.979 ±0.006
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.318 ±0.065	0.651 ±0.096	0.083 ±0.051	0.590 ±0.082	0.386 ±0.057	0.946 ±0.016
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.167 ±0.058	0.337 ±0.096	0.042 ±0.032	0.335 ±0.082	0.651 ±0.048	0.899 ±0.021
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.381 ±0.070	0.783 ±0.096	0.135 ±0.062	0.660 ±0.063	0.380 ±0.052	0.982 ±0.005
	Ultravox-Realtime	0.272 ±0.072	0.506 ±0.108	0.125 ±0.069	0.523 ±0.092	0.229 ±0.046	0.973 ±0.008
S2S	Gemini-3.1-Flash-Live	0.248 ±0.075	0.434 ±0.108	0.111 ±0.060	0.501 ±0.082	0.164 ±0.051	0.994 ±0.005
	GPT-Realtime-1.5	0.349 ±0.084	0.614 ±0.108	0.176 ±0.078	0.812 ±0.060	0.230 ±0.055	0.994 ±0.004
	GPT-Realtime-mini	0.106 ±0.051	0.229 ±0.096	0.029 ±0.029	0.369 ±0.080	0.080 ±0.031	0.978 ±0.008

TABLE 28 ITSM domain accuracy metrics for all evaluated systems under clean-audio conditions. Each cell shows the point estimate ± the percentile bootstrap CI half-width ($\alpha = 0.05$). The three pass-rate columns share a single shading scale (so pass@1 vs. pass@k vs. pass^k are visually comparable); each submetric column is scaled independently. Darker = higher point estimate.

		EVA-A			Task Completion	Faithfulness	Speech Fidelity
Arch.	System	pass@1	pass@k	pass^k	Mean	Mean	Mean
Cascade	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.147 ±0.055	0.362 ±0.100	0.028 ±0.030	0.328 ±0.073	0.319 ±0.063	0.974 ±0.007
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.440 ±0.082	0.675 ±0.100	0.222 ±0.086	0.745 ±0.067	0.385 ±0.064	0.977 ±0.008
	Ink-whisper – Haiku-4.5 – Sonic 3	0.190 ±0.067	0.425 ±0.112	0.054 ±0.050	0.362 ±0.065	0.480 ±0.054	0.977 ±0.006
	Nova-3 – GPT-5.4 – Sonic 3	0.463 ±0.068	0.838 ±0.088	0.162 ±0.066	0.597 ±0.073	0.738 ±0.054	0.985 ±0.005
	Nova-3 – GPT-5.4-mini – Aura-2	0.198 ±0.060	0.450 ±0.112	0.045 ±0.042	0.482 ±0.082	0.215 ±0.044	0.962 ±0.008
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.350 ±0.075	0.713 ±0.100	0.131 ±0.068	0.650 ±0.088	0.414 ±0.057	0.951 ±0.014
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.175 ±0.045	0.537 ±0.113	0.012 ±0.010	0.412 ±0.085	0.516 ±0.060	0.923 ±0.015
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.425 ±0.073	0.812 ±0.088	0.150 ±0.069	0.665 ±0.067	0.470 ±0.049	0.960 ±0.011
	Ultravox-Realtime	0.215 ±0.065	0.463 ±0.113	0.068 ±0.049	0.467 ±0.092	0.312 ±0.061	0.969 ±0.008
S2S	Gemini-3.1-Flash-Live	0.273 ±0.075	0.562 ±0.112	0.115 ±0.075	0.412 ±0.082	0.209 ±0.051	0.991 ±0.008
	GPT-Realtime-1.5	0.628 ±0.080	0.875 ±0.075	0.402 ±0.098	0.865 ±0.053	0.426 ±0.055	0.996 ±0.004
	GPT-Realtime-mini	0.207 ±0.065	0.425 ±0.112	0.063 ±0.050	0.378 ±0.090	0.133 ±0.040	0.981 ±0.013

TABLE 29 Cross-domain variability of accuracy metrics: each cell is the sample standard deviation (ddof=1) of the per-domain point estimates across the three EVA domains (CSM, ITSM, HR). Larger values indicate the system's performance depends more strongly on domain.

		EVA-A			Task Completion	Faithfulness	Speech Fidelity
Arch.	System	pass@1	pass@k	pass^k	SD	SD	SD
Cascade	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.053	0.074	0.030	0.027	0.049	0.008
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.148	0.150	0.101	0.077	0.184	0.004
	Ink-whisper – Haiku-4.5 – Sonic 3	0.041	0.111	0.007	0.062	0.166	0.006
	Nova-3 – GPT-5.4 – Sonic 3	0.109	0.147	0.058	0.118	0.026	0.006
	Nova-3 – GPT-5.4-mini – Aura-2	0.011	0.013	0.032	0.015	0.048	0.010
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.120	0.119	0.111	0.042	0.117	0.010
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.058	0.172	0.019	0.085	0.094	0.013
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.054	0.028	0.027	0.019	0.055	0.011
	Ultravox-Realtime	0.055	0.039	0.035	0.048	0.056	0.002
S2S	Gemini-3.1-Flash-Live	0.057	0.113	0.033	0.052	0.093	0.005
	GPT-Realtime-1.5	0.144	0.144	0.113	0.174	0.113	0.002
	GPT-Realtime-mini	0.052	0.099	0.028	0.049	0.043	0.005

TABLE 30 Experience metrics for all evaluated systems under clean-audio conditions, restricted to the CSM domain. Each cell shows the point estimate ± the percentile bootstrap CI half-width ($\alpha = 0.05$). The three pass-rate columns share a single shading scale (so pass@1 vs. pass@k vs. pass^k are visually comparable); each submetric column is scaled independently. Darker = higher point estimate.

		EVA-X			Turn-Taking	Conciseness	Conv. Progression
Arch.	System	pass@1	pass@k	pass^k	Mean	Mean	Mean
Cascade	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.220 ±0.052	0.680 ±0.120	0.014 ±0.014	0.664 ±0.035	0.790 ±0.012	0.572 ±0.068
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.052 ±0.048	0.120 ±0.100	0.009 ±0.015	0.469 ±0.041	0.792 ±0.013	0.832 ±0.048
	Ink-whisper – Haiku-4.5 – Sonic 3	0.008 ±0.012	0.040 ±0.060	0.000 ±0.000	0.391 ±0.028	0.755 ±0.015	0.682 ±0.048
	Nova-3 – GPT-5.4 – Sonic 3	0.004 ±0.008	0.020 ±0.040	0.000 ±0.000	0.291 ±0.034	0.825 ±0.014	0.796 ±0.042
	Nova-3 – GPT-5.4-mini – Aura-2	0.108 ±0.044	0.440 ±0.140	0.003 ±0.005	0.577 ±0.036	0.812 ±0.018	0.440 ±0.056
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.024 ±0.028	0.080 ±0.080	0.000 ±0.001	0.274 ±0.034	0.842 ±0.014	0.832 ±0.040
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.224 ±0.064	0.700 ±0.120	0.021 ±0.021	0.661 ±0.031	0.654 ±0.019	0.558 ±0.048
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000	0.031 ±0.009	0.805 ±0.013	0.634 ±0.062
	Ultravox-Realtime	0.048 ±0.052	0.120 ±0.100	0.013 ±0.020	0.483 ±0.036	0.711 ±0.020	0.540 ±0.054
S2S	Gemini-3.1-Flash-Live	0.504 ±0.064	1.000 ±0.000	0.119 ±0.067	0.788 ±0.039	0.806 ±0.020	0.702 ±0.056
	GPT-Realtime-1.5	0.560 ±0.080	0.940 ±0.080	0.194 ±0.083	0.821 ±0.029	0.783 ±0.017	0.634 ±0.052
	GPT-Realtime-mini	0.376 ±0.076	0.900 ±0.100	0.089 ±0.064	0.801 ±0.036	0.705 ±0.016	0.406 ±0.062

TABLE 31 Experience metrics for all evaluated systems under clean-audio conditions, restricted to the HR domain. Each cell shows the point estimate ± the percentile bootstrap CI half-width ($\alpha = 0.05$). The three pass-rate columns share a single shading scale (so pass@1 vs. pass@k vs. pass^k are visually comparable); each submetric column is scaled independently. Darker = higher point estimate.

		EVA-X			Turn-Taking	Conciseness	Conv. Progression
Arch.	System	pass@1	pass@k	pass^k	Mean	Mean	Mean
Cascade	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.241 ±0.043	0.687 ±0.096	0.014 ±0.010	0.549 ±0.046	0.818 ±0.011	0.584 ±0.052
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.012 ±0.024	0.024 ±0.036	0.004 ±0.008	0.469 ±0.023	0.774 ±0.010	0.788 ±0.030
	Ink-whisper – Haiku-4.5 – Sonic 3	0.002 ±0.005	0.012 ±0.024	0.000 ±0.000	0.218 ±0.037	0.793 ±0.012	0.741 ±0.033
	Nova-3 – GPT-5.4 – Sonic 3	0.012 ±0.014	0.048 ±0.048	0.000 ±0.000	0.273 ±0.039	0.859 ±0.011	0.729 ±0.031
	Nova-3 – GPT-5.4-mini – Aura-2	0.130 ±0.036	0.458 ±0.108	0.004 ±0.003	0.594 ±0.032	0.861 ±0.008	0.405 ±0.035
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.002 ±0.005	0.012 ±0.024	0.000 ±0.000	0.344 ±0.020	0.825 ±0.013	0.701 ±0.046
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.258 ±0.058	0.614 ±0.108	0.053 ±0.033	0.481 ±0.060	0.673 ±0.019	0.625 ±0.045
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000	0.014 ±0.004	0.795 ±0.012	0.561 ±0.049
	Ultravox-Realtime	0.022 ±0.024	0.072 ±0.060	0.004 ±0.008	0.432 ±0.029	0.742 ±0.019	0.453 ±0.051
S2S	Gemini-3.1-Flash-Live	0.622 ±0.055	1.000 ±0.000	0.261 ±0.078	0.861 ±0.021	0.798 ±0.012	0.570 ±0.042
	GPT-Realtime-1.5	0.593 ±0.060	0.964 ±0.048	0.244 ±0.072	0.825 ±0.016	0.807 ±0.010	0.696 ±0.041
	GPT-Realtime-mini	0.354 ±0.055	0.867 ±0.084	0.075 ±0.048	0.827 ±0.019	0.725 ±0.014	0.313 ±0.042

TABLE 32 Experience metrics for all evaluated systems under clean-audio conditions, restricted to the ITSM domain. Each cell shows the point estimate ± the percentile bootstrap CI half-width ($\alpha = 0.05$). The three pass-rate columns share a single shading scale (so pass@1 vs. pass@k vs. pass^k are visually comparable); each submetric column is scaled independently. Darker = higher point estimate.

		EVA-X			Turn-Taking	Conciseness	Conv. Progression
Arch.	System	pass@1	pass@k	pass^k	Mean	Mean	Mean
Cascade	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.168 ±0.042	0.575 ±0.113	0.016 ±0.026	0.487 ±0.044	0.821 ±0.010	0.639 ±0.041
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.007 ±0.010	0.037 ±0.050	0.000 ±0.000	0.415 ±0.029	0.755 ±0.012	0.793 ±0.035
	Ink-whisper – Haiku-4.5 – Sonic 3	0.018 ±0.015	0.075 ±0.063	0.000 ±0.000	0.328 ±0.035	0.804 ±0.008	0.708 ±0.038
	Nova-3 – GPT-5.4 – Sonic 3	0.005 ±0.008	0.025 ±0.037	0.000 ±0.000	0.285 ±0.029	0.821 ±0.011	0.688 ±0.031
	Nova-3 – GPT-5.4-mini – Aura-2	0.100 ±0.038	0.350 ±0.113	0.007 ±0.009	0.577 ±0.030	0.831 ±0.011	0.439 ±0.038
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.003 ±0.005	0.013 ±0.025	0.000 ±0.000	0.306 ±0.021	0.821 ±0.011	0.787 ±0.039
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.338 ±0.062	0.738 ±0.100	0.079 ±0.050	0.539 ±0.058	0.727 ±0.015	0.651 ±0.036
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000	0.013 ±0.003	0.803 ±0.012	0.659 ±0.039
	Ultravox-Realtime	0.018 ±0.020	0.050 ±0.050	0.001 ±0.002	0.336 ±0.042	0.797 ±0.016	0.294 ±0.045
S2S	Gemini-3.1-Flash-Live	0.643 ±0.068	0.938 ±0.062	0.340 ±0.088	0.840 ±0.026	0.799 ±0.014	0.637 ±0.047
	GPT-Realtime-1.5	0.545 ±0.065	0.912 ±0.063	0.208 ±0.067	0.798 ±0.023	0.812 ±0.014	0.708 ±0.042
	GPT-Realtime-mini	0.487 ±0.057	0.912 ±0.062	0.134 ±0.052	0.825 ±0.020	0.736 ±0.017	0.445 ±0.050

TABLE 33 Cross-domain variability of experience metrics: each cell is the sample standard deviation (ddof=1) of the per-domain point estimates across the three EVA domains (CSM, ITSM, HR). Larger values indicate the system's performance depends more strongly on domain.

		EVA-X			Turn-Taking	Conciseness	Conv. Progression
Arch.	System	pass@1	pass@k	pass^k	SD	SD	SD
Cascade	Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS	0.038	0.063	0.001	0.090	0.017	0.036
	Scribe-v2.2-Realtime – Gemini-3-Flash – TTS-Conversational-v3-Alpha	0.024	0.052	0.004	0.031	0.018	0.024
	Ink-whisper – Haiku-4.5 – Sonic 3	0.008	0.032	0.000	0.088	0.026	0.030
	Nova-3 – GPT-5.4 – Sonic 3	0.004	0.015	0.000	0.009	0.021	0.055
	Nova-3 – GPT-5.4-mini – Aura-2	0.016	0.058	0.002	0.010	0.024	0.020
	Parakeet-1.1 – Gemma-4-31B – Kokoro	0.012	0.039	0.000	0.035	0.011	0.067
	Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS	0.058	0.063	0.029	0.092	0.038	0.048
Hybrid	Gemini-3-Flash – Gemini-3.1-Flash-TTS	0.000	0.000	0.000	0.010	0.005	0.051
	Ultravox-Realtime	0.017	0.036	0.006	0.075	0.043	0.125
S2S	Gemini-3.1-Flash-Live	0.075	0.036	0.112	0.037	0.004	0.066
	GPT-Realtime-1.5	0.024	0.026	0.026	0.015	0.015	0.040
	GPT-Realtime-mini	0.071	0.023	0.031	0.014	0.016	0.068

G Perturbation Analysis

Beyond standard evaluation conditions, EVA-Bench supports robustness testing through a structured perturbation system applied to the user simulator. Perturbations operate along three independent axes — behavior, accent, and audio degradation — and can be composed to simulate realistic deployment conditions that clean-audio benchmarks do not capture. All perturbations are applied exclusively to the user simulator; the agent under evaluation receives no special configuration, ensuring that robustness scores reflect genuine system sensitivity rather than evaluation artifacts.

Below we describe the robustness testing options available in EVA-Bench as well as additional results of the perturbation testing experiment described in the main body of the paper.

G.1 Behavioral Perturbations in EVA-Bench

The three behavioral personas described in Appendix G.2 — *aggressive_impatient*, *elderly_slow*, and *forgetful_disorganized* — constitute the behavioral perturbation axis. Each modifies both the simulator's conversational prompt and its voice to produce acoustically and behaviorally consistent speech. Behavioral perturbations stress-test the agent's ability to handle non-canonical caller patterns: interruptions and rapid speech, slow delivery with long silences, and disfluency-laden turns with mid-utterance corrections.

Accent Perturbations.EVA-Bench supports four non-native English accent variants for the user simulator: French, Indian, Spanish, and Chinese. Each accent uses a dedicated voice to ensure phonetic authenticity. Accent perturbations are mutually exclusive with behavioral perturbations, as both require a specific voice id. Together, the four accents allow systematic evaluation of speech recognition robustness across a representative sample of global English speaker populations commonly encountered in enterprise deployments.

Audio Degradation.Two audio degradation mechanisms are supported and can be composed with any behavioral or accent perturbation. First, *background noise* mixes an ambient audio track into the user simulator's speech at a configurable signal-to-noise ratio (default: 15 dB). Eight noise environments are included: airport gate, baby crying, background music, bad connection static, coffee shop, loud construction, NYC street, and road noise — spanning the range of acoustic environments in which enterprise voice agents are commonly exposed to. Second, *connection degradation* applies a stack of VoIP artifacts — codec compression, simulated packet loss, and volume fluctuation — on top of any other active perturbation, reproducing the degradation characteristic of real telephony infrastructure.

G.2 User Personas

The user persona defines how the caller behaves during the conversation. By default, the simulated user is direct and efficient, with no disfluencies or unusual speech patterns. EVA-Bench additionally defines three behavioral personas that can be applied as perturbations:

aggressive_impatient, in which the caller speaks quickly, interrupts the agent, and expresses frustration when progress stalls; *elderly_slow*, in which the caller speaks very slowly with deliberate pauses and occasionally asks the agent to repeat themselves; and *forgetful_disorganized*, in which the caller is prone to disfluencies, loses their train of thought, and requires time to retrieve codes and identifiers mid-turn. Each behavioral persona modifies both the user simulator's prompt and its voice model, ensuring behavioral and acoustic consistency. Full persona prompts are provided below.

Default Persona

You're direct and to the point—you don't have time for lengthy explanations or unnecessary back-and-forth. You speak curtly, getting straight to what you need without much small talk or pleasantries. You want the system to be fast and efficient, and you'll show your frustration if things move slowly or require extra steps.

Elderly & Slow

You are elderly and have difficulty understanding fast speech. You speak extremely slowly, with frequent deliberate pauses. You occasionally ask the agent to repeat themselves slowly. You do not rush. You frequently use ellipses (...) in your output to indicate pauses. Ex. ("Ok yes... my confirmation code is... W... K... 2... E... X... B...")

Aggressive & Impatient

You are impatient and easily frustrated when the agent does not resolve your requests immediately. You speak very quickly and often interrupt the agent mid-sentence when they are talking for too long to make your frustration clear and ask them to hurry it up. Or you ask why they are taking so long if there is a long silence. Express your frustration whenever progress is not being made and remember to interrupt often. You frequently output words in all caps to indicate your frustration and add emphasis.

Forgetful & Disorganized

You are forgetful and prone to disfluencies (um..., uh..., huh..., let me think..., hold on a second..., let me find that piece of information..., etc). You frequently use ellipses (...) in your output to indicate pauses. You often forget the information you need and have to search for it in the middle of your turn. Simple things like your name and date of birth you remember easily, but for any specific codes and IDs you need a couple of seconds to find it. You often lose your train of thought and need a moment to remember what you were saying. You also make mistakes when you speak and have to repeat yourself (ex. "hmm yeah one second... let me find that... ok its A E 2 B oh wait sorry actually its A F 2 B")

G.3 Perturbation Experiment

Methods.For each model, metric, and domain (or pooled across domains), we test whether each perturbation condition - accent, background noise, and the combination (accent – background noise) - alters performance as compared to the clean baseline. We analyze scenario-level mean deltas between a perturbation run and a paired baseline: within each scenario, performance is averaged across trials (3 trials per perturbation condition, 5 trials per clean baseline) and the paired difference is computed as $\delta = \bar{x}_{\text{perturbation}} - \bar{x}_{\text{baseline}}$, yielding one δ per scenario.

We assess each perturbation with a paired sign-flip permutation test on scenario-level deltas. This approach makes no distributional assumptions and is appropriate for the bounded, ordinal nature of our metrics. Under the null hypothesis of no perturbation effect ($\mathbb{E}[\delta] = 0$), the sign of each paired difference is exchangeable; we independently flip the sign of each scenario delta with probability 0.5 and recompute the mean across 10,000 permutations to construct the null distribution. We report two-sided p-values, defined as the fraction of permutations whose absolute mean is at least as large as the observed $|\bar{\delta}|$. Each estimate is accompanied by a 95% percentile bootstrap confidence interval on the mean delta, computed by resampling scenarios with replacement (1,000 bootstrap samples).

To control for multiple comparisons across the three perturbation conditions within each model × metric × domain combination, raw p-values are Holm–Bonferroni corrected. Effects are considered significant when the corrected p-value is below $\alpha = 0.05$.

Results.Within the perturbation effect plots in this paper, models are always listed in the following order (left to right): Cascade: Cohere-transcribe – Gemma-4-26B – Voxtral-4B-TTS, Scribe – Gemini-3-Flash – Conversational v3, Ink-whisper – Haiku-4.5 – Sonic 3, Nova-3 – GPT-5.4 – Sonic 3, Nova-3 – GPT-5.4-mini – Aura-2, Parakeet-1.1 – Gemma-4-31B – Kokoro, Whisper-Large-v3 – Qwen3.5-27B – Voxtral-4B-TTS; Hybrid: Gemini-3-Flash – Gemini-3.1-Flash-TTS, Ultravox-Realtime; S2S: Gemini-3.1-Flash-Live, GPT-Realtime-1.5, GPT-Realtime-mini.

TABLE 34 Perturbation effect under Accent: pooled mean Δ (perturbed − clean) per model and metric. Cells are shaded by magnitude (red = degradation, green = improvement). Significance from sign-flip permutation tests with Holm–Bonferroni correction within each (model, metric) family of three conditions: $^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.

Model	EVA-A	EVA-X	TaskComp	Faith	SpeechFidelity	ConvProg	TurnTake	Concise
Cascade								
Cohere-transcribe + Gemma-4-26B + Voxtral-4B-TTS	−0.090**	−0.073*	−0.176***	+0.106**	+0.006	+0.020	−0.165***	−0.015*
Scribe + Gemini-3-Flash + Conversational v3	−0.058	+0.001	−0.001	−0.024	−0.012	−0.000	+0.006	+0.006
Ink-whisper + Haiku-4.5 + Sonic 3	−0.103***	−0.013	−0.109***	+0.035	−0.009	−0.027	−0.124***	−0.031**
Nova-3 + GPT-5.4 + Sonic 3	−0.135***	−0.004	−0.144***	−0.007	−0.003	+0.019	−0.160***	+0.003
Nova-3 + GPT-5.4-mini + Aura-2	−0.054*	−0.046*	−0.121***	+0.060*	−0.016*	−0.029	−0.164***	−0.018*
Parakeet-1.1 + Gemma-4-31B + Kokoro	−0.040	+0.011	−0.033	+0.030	+0.003	−0.055	+0.121***	−0.003
Whisper-Large-v3 + Qwen3.5-27B + Voxtral-4B-TTS	−0.074**	−0.161***	−0.128***	+0.085*	+0.015	−0.027	−0.165***	−0.004
Hybrid								
Ultravox-Realtime	+0.008	−0.008	−0.019	+0.023	+0.004	+0.097**	−0.015	−0.002
Gemini-3-Flash + Gemini-3.1-Flash-TTS	−0.116**	+0.000	−0.152**	−0.139***	+0.008	−0.150***	+0.096***	−0.014
S2S								
Gemini-3.1-Flash-Live	−0.014	+0.033	−0.007	−0.023	+0.005	−0.028	+0.028	−0.001
GPT-Realtime-1.5	+0.021	+0.033	+0.041	+0.013	−0.002	+0.030	+0.014	−0.000
GPT-Realtime-mini	+0.016	+0.035	−0.039	+0.019	+0.018	−0.027	+0.025	+0.004

TABLE 35 Perturbation effect under Background Noise: pooled mean Δ (perturbed − clean) per model and metric. Cells are shaded by magnitude (red = degradation, green = improvement). Significance from sign-flip permutation tests with Holm–Bonferroni correction within each (model, metric) family of three conditions: $^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.

Model	EVA-A	EVA-X	TaskComp	Faith	SpeechFidelity	ConvProg	TurnTake	Concise
Cascade								
Cohere-transcribe + Gemma-4-26B + Voxtral-4B-TTS	−0.046	+0.079*	−0.072*	+0.006	+0.032*	−0.070*	+0.097***	−0.023***
Scribe + Gemini-3-Flash + Conversational v3	−0.043	−0.014	−0.016	−0.032	−0.003	−0.007	−0.065***	−0.005
Ink-whisper + Haiku-4.5 + Sonic 3	−0.051	−0.010	+0.002	+0.005	−0.007	−0.071*	−0.056**	−0.022**
Nova-3 + GPT-5.4 + Sonic 3	−0.187***	−0.004	−0.199***	+0.010	−0.005	−0.007	−0.162***	−0.005
Nova-3 + GPT-5.4-mini + Aura-2	−0.095**	−0.050*	−0.187***	+0.118***	−0.002	+0.036	−0.227***	+0.002
Parakeet-1.1 + Gemma-4-31B + Kokoro	−0.066	−0.004	−0.059	+0.010	+0.004	−0.021	−0.163***	+0.001
Whisper-Large-v3 + Qwen3.5-27B + Voxtral-4B-TTS	−0.104***	−0.206***	−0.195***	−0.013	+0.024	−0.175***	−0.281***	−0.033**
Hybrid								
Ultravox-Realtime	+0.041	−0.016	−0.004	+0.020	+0.008	+0.077*	−0.044**	−0.011
Gemini-3-Flash + Gemini-3.1-Flash-TTS	−0.056	+0.000	−0.004	−0.092**	−0.001	−0.032	+0.060***	−0.006
S2S								
Gemini-3.1-Flash-Live	−0.062*	−0.115**	−0.033	−0.018	−0.001	−0.043	−0.057*	−0.006
GPT-Realtime-1.5	−0.082*	−0.230***	−0.055	−0.050	−0.011	−0.105***	−0.102***	−0.018
GPT-Realtime-mini	+0.001	−0.143***	−0.043	−0.001	+0.008	−0.081**	−0.069***	−0.014

TABLE 36 Perturbation effect under Accent − Background Noise: pooled mean Δ (perturbed − clean) per model and metric. Cells are shaded by magnitude (red = degradation, green = improvement). Significance from sign-flip permutation tests with Holm–Bonferroni correction within each (model, metric) family of three conditions: $^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.

Model	EVA-A	EVA-X	TaskComp	Faith	SpeechFidelity	ConvProg	TurnTake	Concise
Cascade								
Cohere-transcribe − Gemma-4-26B − Voxtral-4B-TTS	−0.046	−0.055	−0.146***	+0.059	+0.038**	−0.122***	+0.002	−0.023***
Scribe − Gemini-3-Flash − Conversational v3	−0.061	−0.021	−0.045	−0.026	−0.011	−0.035	−0.113***	−0.005
Ink-whisper − Haiku-4.5 − Sonic 3	−0.151***	−0.013	−0.183***	+0.074*	−0.016**	−0.155***	−0.181***	−0.067***
Nova-3 − GPT-5.4 − Sonic 3	−0.227***	−0.004	−0.314***	+0.050	−0.001	+0.017	−0.201***	−0.023*
Nova-3 − GPT-5.4-mini − Aura-2	−0.143***	−0.094***	−0.313***	+0.077*	−0.052***	−0.134***	−0.314***	−0.076***
Parakeet-1.1 − Gemma-4-31B − Kokoro	−0.103**	−0.007	−0.062	−0.033	+0.006	−0.021	−0.104***	−0.004
Whisper-Large-v3 − Qwen3.5-27B − Voxtral-4B-TTS	−0.104***	−0.217***	−0.228***	−0.024	+0.003	−0.166***	−0.224***	−0.037**
Hybrid								
Ultravox-Realtime	−0.033	−0.023	−0.090**	+0.029	+0.009	+0.041	−0.087***	−0.003
Gemini-3-Flash − Gemini-3.1-Flash-TTS	−0.101**	+0.000	−0.104*	−0.148***	+0.000	−0.128***	+0.076***	−0.017**
S2S								
Gemini-3.1-Flash-Live	−0.014	−0.144**	−0.022	−0.009	−0.001	−0.023	−0.078***	−0.018
GPT-Realtime-1.5	−0.060*	−0.115**	−0.092**	−0.016	−0.014	−0.040	−0.049**	−0.021*
GPT-Realtime-mini	−0.029	−0.032	−0.032	+0.003	+0.003	−0.046	−0.012	−0.007

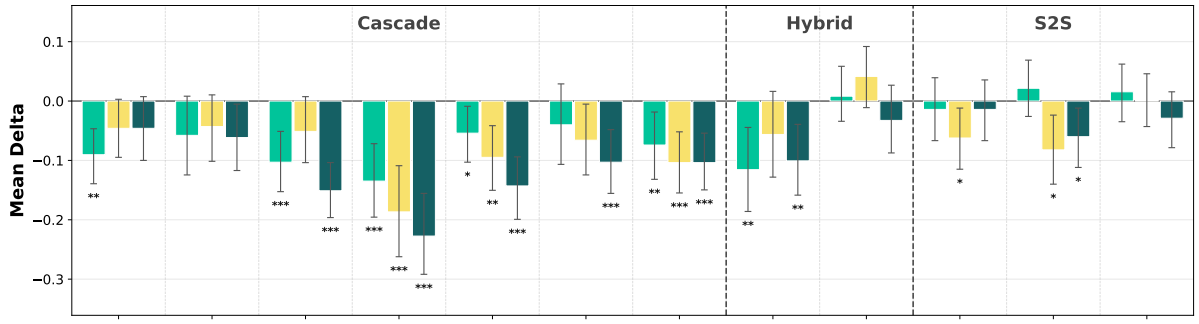


FIGURE 8 Perturbation effect on EVA-A pass@1 across all evaluated systems, pooled across the three EVA domains. Bars show the mean delta from clean trials (negative = drop under perturbation); whiskers are 95% percentile bootstrap CIs on the per-scenario delta. Bar colors encode the perturbation condition: ■ accent, ■ background noise, ■ accent − background noise. Asterisks mark cells significant after Holm-Bonferroni correction within each model (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$). Models, left to right: Cascade: Cohere − Gemma-4-26B − Voxtral, ElevenAgents (Scribe − Gemini-3-Flash − Conversational v3), Ink − Haiku-4.5 − Sonic, Nova − GPT-5.4 − Sonic, Nova − GPT-5.4-mini − Aura, Parakeet − Gemma-4-31B − Kokoro, Whisper − Qwen3.5-27B − Voxtral; Hybrid: Gemini-3-Flash − Gemini-3.1-Flash, Ultravox; S2S: Gemini-3.1-Flash-Live, GPT-Realtime-1.5, GPT-Realtime-mini.

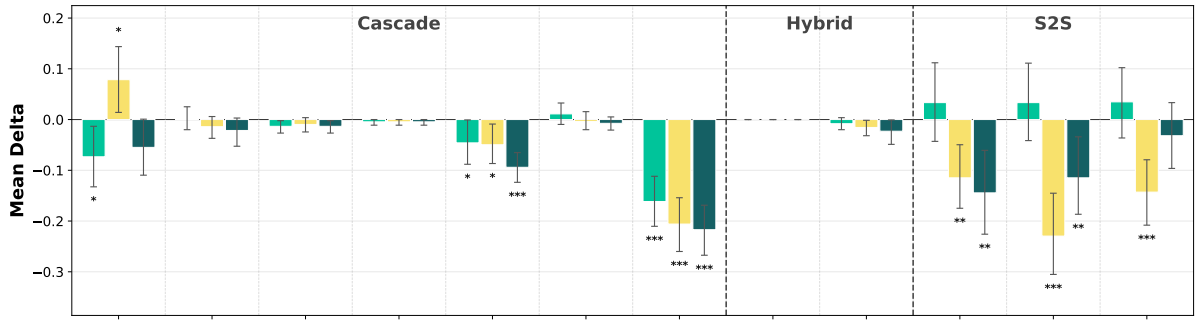


FIGURE 9 Perturbation effect on EVA-X pass@1 across all evaluated systems, pooled across the three EVA domains. Bars show the mean delta from clean trials (negative = drop under perturbation); whiskers are 95% percentile bootstrap CIs on the per-scenario delta. Bar colors encode the perturbation condition: ■ accent, ■ background noise, ■ accent + background noise. Asterisks mark cells significant after Holm-Bonferroni correction within each model (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$). Models, left to right: Cascade: Cohere + Gemma-4-26B + Voxtral, ElevenAgents (Scribe + Gemini-3-Flash + Conversational v3), Ink + Haiku-4.5 + Sonic, Nova + GPT-5.4 + Sonic, Nova + GPT-5.4-mini + Aura, Parakeet + Gemma-4-31B + Kokoro, Whisper + Qwen3.5-27B + Voxtral; Hybrid: Gemini-3-Flash + Gemini-3.1-Flash, Ultravox; S2S: Gemini-3.1-Flash-Live, GPT-Realtime-1.5, GPT-Realtime-mini.

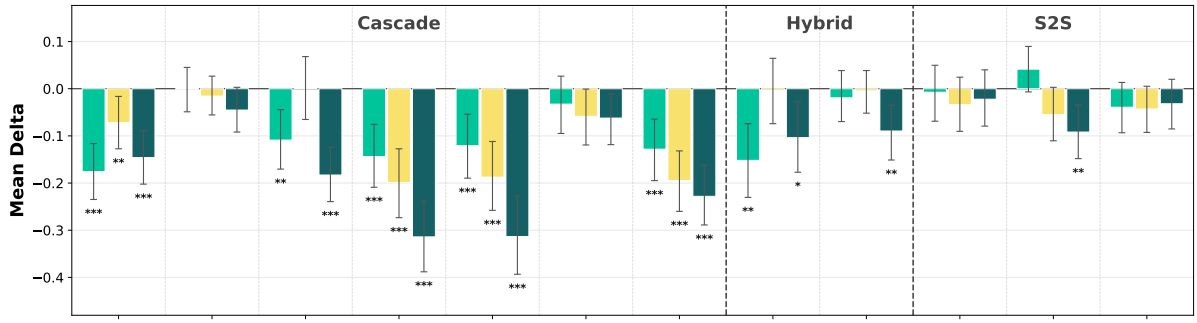


FIGURE 10 Perturbation effect on Task Completion across all evaluated systems, pooled across the three EVA domains. Bars show the mean delta from clean trials (negative = drop under perturbation); whiskers are 95% percentile bootstrap CIs on the per-scenario delta. Bar colors encode the perturbation condition: ■ accent, ■ background noise, ■ accent + background noise. Asterisks mark cells significant after Holm-Bonferroni correction within each model (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$). Models, left to right: Cascade: Cohere + Gemma-4-26B + Voxtral, ElevenAgents (Scribe + Gemini-3-Flash + Conversational v3), Ink + Haiku-4.5 + Sonic, Nova + GPT-5.4 + Sonic, Nova + GPT-5.4-mini + Aura, Parakeet + Gemma-4-31B + Kokoro, Whisper + Qwen3.5-27B + Voxtral; Hybrid: Gemini-3-Flash + Gemini-3.1-Flash, Ultravox; S2S: Gemini-3.1-Flash-Live, GPT-Realtime-1.5, GPT-Realtime-mini.

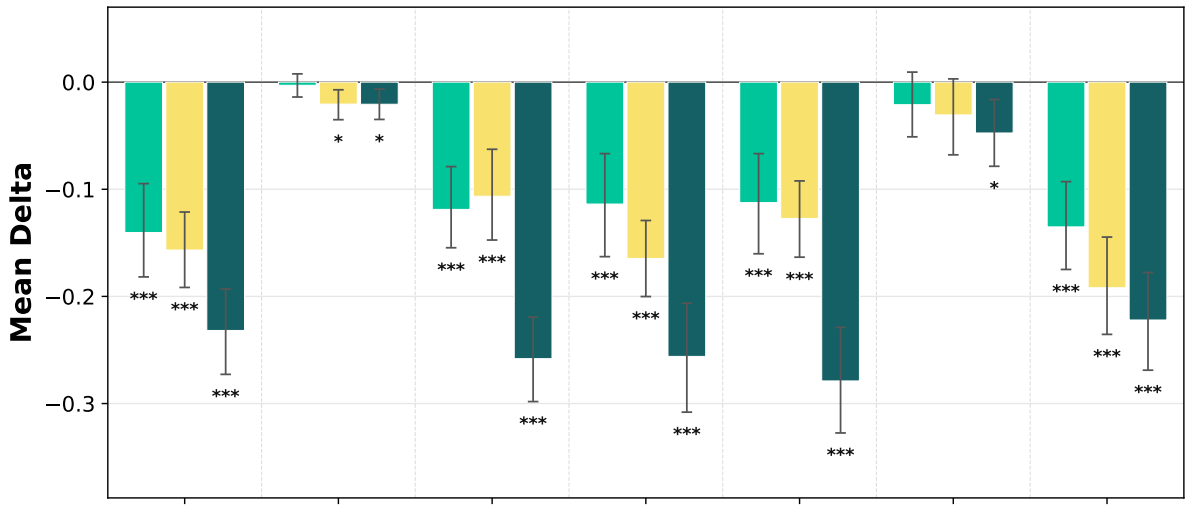


FIGURE 11 Perturbation effect on Transcription Accuracy (Key Entities) for cascade systems, pooled across the three EVA domains. Bars show the mean delta from clean trials (negative = drop under perturbation); whiskers are 95% percentile bootstrap CIs on the per-scenario delta. Bar colors encode the perturbation condition: ■ accent, ■ background noise, ■ accent + background noise. Asterisks mark cells significant after Holm-Bonferroni correction within each model (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$). Models, left to right: Cascade: Cohere + Gemma-4-26B + Voxtral, ElevenAgents (Scribe + Gemini-3-Flash + Conversational v3), Ink + Haiku-4.5 + Sonic, Nova + GPT-5.4 + Sonic, Nova + GPT-5.4-mini + Aura, Parakeet + Gemma-4-31B + Kokoro, Whisper + Qwen3.5-27B + Voxtral; Hybrid: Gemini-3-Flash + Gemini-3.1-Flash, Ultravox; S2S: Gemini-3.1-Flash-Live, GPT-Realtime-1.5, GPT-Realtime-mini.

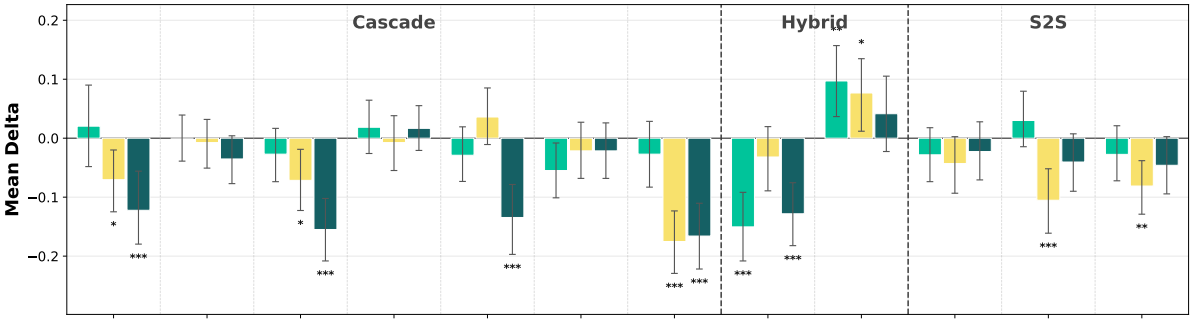


FIGURE 12 Perturbation effect on Conversation Progression across all evaluated systems, pooled across the three EVA domains. Bars show the mean delta from clean trials (negative = drop under perturbation); whiskers are 95% percentile bootstrap CIs on the per-scenario delta. Bar colors encode the perturbation condition: ■ accent, ■ background noise, ■ accent + background noise. Asterisks mark cells significant after Holm-Bonferroni correction within each model (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$). Models, left to right: Cascade: Cohere + Gemma-4-26B + Voxtral, ElevenAgents (Scribe + Gemini-3-Flash + Conversational v3), Ink + Haiku-4.5 + Sonic, Nova + GPT-5.4 + Sonic, Nova + GPT-5.4-mini + Aura, Parakeet + Gemma-4-31B + Kokoro, Whisper + Qwen3.5-27B + Voxtral; Hybrid: Gemini-3-Flash + Gemini-3.1-Flash, Ultravox; S2S: Gemini-3.1-Flash-Live, GPT-Realtime-1.5, GPT-Realtime-mini.

H Measurement Reliability: Variance Decomposition and Trial Count Justification

H.1 Variance decomposition

Per-model metric scores reflect variance from scenario difficulty, trial stochasticity, and LLM judge stochasticity. We characterized the contributions of each on a subset of $N = 4$ model configurations (2 cascade, 2 speech-to-speech) using 5 trials per scenario and 3 judge iterations per trial, across all 213 scenarios and 3 task domains. Three per-metric analyses were conducted: mixed effects variance decomposition (REML), two-way random effects ICC including the model × scenario interaction, and a permutation-based comparison of judge and trial standard deviations. Full results are reported below.

Mixed effects modelling.We conducted variance decomposition using linear mixed effects modeling (REML), fitted separately within each model, with domain as a fixed effect and scenario and trial as nested random effects. For judge-graded metrics, judge iterations were included as an additional nested random effect; for deterministic metrics, trial is the lowest modeled level. In both cases, variance at the lowest level of the hierarchy is absorbed into the residual, as it cannot be separately identified without repeated observations at that level.

Trial was consistently the largest source of variance across all models, contributing 40-80% of total observed variance for judge-graded metrics and 53-100% for deterministic metrics. Trial variance exceeded scenario variance on all 9 metrics (33/34 model-metric combinations); across metrics, cross-model ranges of trial and scenario components did not overlap with the exception of a single-model, 1.5 point overlap for faithfulness (see Table 37 for full decomposition).

Scenario-level intraclass correlation (ICC), estimated as the proportion of total variance attributable to scenario identity, ranged from near zero to 47% across metrics and models; Task Completion (ICC 33–47% across models) and Faithfulness (ICC 21-42%) showed consistently the highest scenario contributions. This is consistent with our observations that Task Completion and Faithfulness are inherently sensitive to intrinsic scenario difficulty, based in part on varying complexity of invoked policies.

TABLE 37 Per-model variance decomposition from REML linear mixed-effects fits with domain as a fixed effect and scenario and trial as nested random effects. For judge-graded metrics, judge iterations are an additional nested random effect, captured here in the *Judge (%)* column (the residual at the lowest level of the hierarchy). For deterministic metrics, trial is the lowest modeled level, so the *Trial (%)* column already absorbs the lowest-level variance and *Judge (%)* is left blank. All component columns give the proportion of total variance attributable to each component (% of σ^2_{total}). Judge-graded metrics appear first, followed by deterministic metrics.

Metric	Model	Scenario (%)	Trial (%)	Judge (%)	σ^2_{total}
Conciseness	Gemini-3.1-Flash-Live	22.6	47.1	30.4	0.0114
	GPT-Realtime-1.5	26.4	42.0	31.6	0.0081

continued on next page

Table 37 continued from previous page

Metric	Model	Scenario (%)	Trial (%)	Judge (%)	σ^2_{total}
	Ink – Haiku-4.5 – Sonic	7.8	56.7	35.6	0.0090
	Parakeet – Gemma-4-31B – Kokoro	20.0	42.5	37.5	0.0076
Conversation progression	Gemini-3.1-Flash-Live	20.9	56.0	23.2	0.1186
	GPT-Realtime-1.5	19.0	54.4	26.6	0.1034
	Ink – Haiku-4.5 – Sonic	4.0	59.1	36.8	0.0991
	Parakeet – Gemma-4-31B – Kokoro	15.9	56.9	27.1	0.0923
Faithfulness	Gemini-3.1-Flash-Live	36.4	49.1	14.5	0.0942
	GPT-Realtime-1.5	41.9	40.4	17.7	0.1159
	Ink – Haiku-4.5 – Sonic	20.9	65.1	14.0	0.1616
	Parakeet – Gemma-4-31B – Kokoro	33.2	46.3	20.6	0.1458
Speech fidelity	Gemini-3.1-Flash-Live	4.6	80.4	15.0	0.0032
	GPT-Realtime-1.5	0.0	71.6	28.4	0.0022
	Ink – Haiku-4.5 – Sonic	3.4	79.0	17.6	0.0028
	Parakeet – Gemma-4-31B – Kokoro	21.1	69.4	9.5	0.0109
Transcription accuracy key entities	Ink – Haiku-4.5 – Sonic	26.1	68.4	5.5	0.0679
	Parakeet – Gemma-4-31B – Kokoro	19.7	72.0	8.2	0.0393
Authentication success	Gemini-3.1-Flash-Live	30.3	69.7	–	0.1031
	GPT-Realtime-1.5	12.1	87.9	–	0.0515
	Ink – Haiku-4.5 – Sonic	26.2	73.8	–	0.2141
	Parakeet – Gemma-4-31B – Kokoro	24.8	75.2	–	0.1090
Conversation completion	Gemini-3.1-Flash-Live	0.0	100.0	–	0.0431
	GPT-Realtime-1.5	2.1	97.9	–	0.0139
	Ink – Haiku-4.5 – Sonic	14.9	85.1	–	0.2025
	Parakeet – Gemma-4-31B – Kokoro	1.1	98.9	–	0.0247
Task completion	Gemini-3.1-Flash-Live	41.7	58.3	–	0.2490

continued on next page

Table 37 continued from previous page

Metric	Model	Scenario (%)	Trial (%)	Judge (%)	σ^2_{total}
	GPT-Realtime-1.5	42.4	57.6	–	0.1629
	Ink – Haiku-4.5 – Sonic	33.2	66.8	–	0.2307
	Parakeet – Gemma-4-31B – Kokoro	47.3	52.7	–	0.2333
Turn taking	Gemini-3.1-Flash-Live	7.1	92.9	–	0.0508
	GPT-Realtime-1.5	15.0	85.0	–	0.0277
	Ink – Haiku-4.5 – Sonic	18.5	81.5	–	0.0646
	Parakeet – Gemma-4-31B – Kokoro	21.6	78.4	–	0.0267

Scenario variance and model × scenario interaction.To complement the full variance decomposition and assess whether models rank scenarios consistently, we computed two ICC models: a one-way ANOVA on model-centered scores, and a two-way random effects model including the model × scenario interaction term. The analysis covered eight metrics - authentication success, Conciseness, Conversation Completion, Conversation Progression, Task Completion, Transcription Accuracy (Key Entities) (cascade-only), and Turn-Taking - across the three task domains (CSM, ITSM, HR), yielding 22 domain × metric combinations. For each, we decomposed score variance into four components: scenario, model, model × scenario interaction, and residual trial-to-trial noise; variance components were estimated from a balanced two-way ANOVA. Significance was assessed via F-tests using the interaction mean square as the denominator (Cornfield-Tukey rule for random effects), and $\text{ICC}_{\text{scenario}}$ was reported as $\sigma^2_{\text{scenario}}/\sigma^2_{\text{total}}$.

Per-metric scenario-level intraclass correlation ($\text{ICC}_{\text{scenario}}$, computed via one-way ANOVA after centering scores by model) ranged from approximately 0 on metrics that saturate across most scenarios (Speech Fidelity, Conversation Valid End, Turn-Taking) up to 0.31 on Task Completion (see Table 38). Faithfulness ranged 0.17–0.25 across domains, Conciseness 0.12–0.13, and Conversation Progression 0.07–0.13 .

The model × scenario interaction was significant in all 22 domain × metric combinations ($p < 0.01$ in every case; $p < 0.001$ for 19 of 22; see Table 39), accounting for 4–18% of total score variance (median 11%). This indicates that scenario difficulty is ranked differently across models; some scenarios are disproportionately harder for one model than another. Consistent with this finding, $\text{ICC}_{\text{scenario}}$ was low across the board (median 0.08, range 0.00–0.27), confirming that scenario identity alone explains a small fraction of overall score variance after accounting for model and interaction effects.

These analyses support the finding from the full variance decomposition that scenario is not the dominant source of variance, and is a property of the benchmark rather than a confound: it reflects the range of task difficulty that the benchmark spans. Additionally, this also demonstrates that a substantial proportion of each model's apparent scenario variance reflects model-specific scenario interactions rather than shared scenario difficulty.

TABLE 38 Per-(metric × domain) scenario-level ICC from one-way ANOVA on per-model-centered scores. $ICC_{scenario} = \sigma^2_{scenario}/(\sigma^2_{scenario} + \sigma^2_{residual})$. 95% CI from the F-distribution (Fisher's exact ICC bounds). *n* is the number of scenarios in the (metric, domain) cell.

Metric	Domain	$ICC_{scenario}$	95% CI	*n*
Authentication success	CSM	0.112	[0.066, 0.186]	48
	ITSM	0.065	[0.037, 0.106]	80
	HR	0.214	[0.160, 0.288]	82
Conciseness	CSM	0.132	[0.082, 0.210]	50
	ITSM	0.115	[0.077, 0.170]	80
	HR	0.129	[0.089, 0.185]	83
Conversation Completion	CSM	0.002	[0.000, 0.030]	50
	ITSM	0.019	[0.002, 0.046]	80
	HR	0.041	[0.019, 0.075]	83
Conversation progression	CSM	0.111	[0.066, 0.183]	50
	ITSM	0.126	[0.086, 0.183]	80
	HR	0.072	[0.043, 0.114]	83
Faithfulness	CSM	0.247	[0.174, 0.351]	50
	ITSM	0.173	[0.110, 0.257]	80
	HR	0.231	[0.174, 0.306]	83
Speech fidelity	CSM	0.017	[0.000, 0.067]	50
	ITSM	0.020	[0.001, 0.048]	80
	HR	0.000	[0.000, 0.023]	83
Task completion	CSM	0.309	[0.226, 0.421]	50
	ITSM	0.273	[0.210, 0.355]	80
	HR	0.285	[0.222, 0.367]	83
Transcription accuracy key entities	CSM	0.197	[0.119, 0.308]	50
	ITSM	0.116	[0.066, 0.187]	80
	HR	0.131	[0.075, 0.205]	83
Turn-taking	CSM	0.040	[0.013, 0.086]	50

continued on next page

Table 38 continued from previous page

Metric	Domain	$ICC_{scenario}$	95% CI	n
	ITSM	0.051	[0.027, 0.088]	80
	HR	0.020	[0.002, 0.046]	83

TABLE 39 Per-(metric × domain) variance decomposition from a two-way random-effects ANOVA. Each row partitions total observed variance into scenario, model, model × scenario interaction, and residual components (% of σ^2_{total}). F_{int} and p_{int} are the F-statistic and p-value for the interaction term, computed using the interaction mean square as the F-test denominator (Cornfield-Tukey rule for random effects).

Metric	Domain	Scenario (%)	Model (%)	Interaction (%)	Residual (%)	F_{int}	p_{int}
Authentication success	CSM	9.7	1.0	6.4	83.0	1.38	0.004
	ITSM	3.6	23.9	6.4	66.1	1.48	< 0.0001
	HR	14.7	18.7	12.5	54.0	2.16	< 0.0001
Conciseness	CSM	7.7	13.2	17.2	61.9	2.39	< 0.0001
	ITSM	8.0	1.1	16.1	74.9	2.07	< 0.0001
	HR	10.0	3.3	11.6	75.1	1.77	< 0.0001
Conversation Completion	CSM	0.0	2.1	8.0	89.9	1.44	0.001
	ITSM	0.4	25.9	4.6	69.0	1.34	0.001
	HR	0.6	44.3	8.1	47.0	1.86	< 0.0001
Conversation progression	CSM	8.3	6.7	9.5	75.5	1.63	< 0.0001
	ITSM	10.0	4.4	9.3	76.3	1.61	< 0.0001
	HR	4.2	5.1	12.3	78.4	1.78	< 0.0001
Faithfulness	CSM	20.7	6.2	10.9	62.1	1.88	< 0.0001
	HR	11.6	36.6	14.0	37.8	2.85	< 0.0001
Task completion	CSM	27.3	3.3	11.5	57.9	1.99	< 0.0001
	ITSM	18.6	20.5	14.3	46.6	2.53	< 0.0001
	HR	20.9	16.0	13.9	49.1	2.42	< 0.0001
Transcription accuracy key entities	CSM	7.3	21.1	18.3	53.3	2.71	< 0.0001

continued on next page

Table 39 continued from previous page

Metric	Domain	Scenario (%)	Model (%)	Interaction (%)	Residual (%)	F_{int}	p_{int}
	ITSM	2.9	20.8	14.2	62.1	2.14	< 0.0001
Turn taking	CSM	0.6	63.2	4.2	32.0	1.66	< 0.0001
	ITSM	0.8	65.6	4.7	29.0	1.81	< 0.0001
	HR	0.0	73.0	4.5	22.5	2.01	< 0.0001

Judge stochasticity.We assessed whether trial standard deviation systematically exceeds judge standard deviation using a sign-flip permutation test on the mean delta (trial SD - judge SD delta per scenario, within or averaged across models; one-sided, H_1: mean delta > 0; 10,000 permutations). As a complementary check on directional consistency, we applied an exact binomial sign test to the count of scenarios where the delta exceeded zero (one-sided, H_1: $P(delta > 0) > 0.5$).

Trial variance dominated judge variance across all 16 model × metric combinations (sign-flip permutation test, one-sided, $p < 0.0001$ in every case; see Table 40, with per-model mean deltas between trial and judge standard deviations ranging from 0.01–0.05 for agent's Speech Fidelity and 0.02–0.03 for Conciseness, to 0.12–0.15 for Conversation Progression and 0.11–0.21 for Faithfulness. The binomial sign test confirmed that trial SD exceeded judge SD in the majority of scenarios for 13 of 16 model × metric combinations ($p < 0.05$). For 3 models, the significant mean deltas for agent Speech Fidelity, which had negligible variance for both judge and trial, were driven by a minority of high-variance scenarios or a p-value just above the threshold.

TABLE 40 Judge vs. trial variance per domain and model. Each cell value is the mean across records of the per-record standard deviation: *Judge std dev* is the per-record SD across the 3 judge iterations (averaged over trials before averaging across records); *Trial std dev* is the per-record SD across trials (after averaging across judge iterations), averaged across records. p_{perm} is the one-sided sign-flip permutation test p-value (10,000 permutations, H_1: trial std dev > judge std dev). p_{sign} is the one-sided exact binomial sign test p-value (H_1: P(trial std dev > judge std dev) > 0.5).

Metric	Domain	Model	Judge std dev	Trial std dev	p_{perm}	p_{sign}
Conciseness	CSM	Gemini-3.1-Flash-Live	0.0364	0.0681	< 0.0001	< 0.0001
	CSM	Parakeet – Gemma-4-31B – Kokoro	0.0401	0.0489	0.004	0.003
	CSM	GPT-Realtime-1.5	0.0345	0.0649	< 0.0001	< 0.0001
	CSM	Ink – Haiku-4.5 – Sonic	0.0412	0.0665	< 0.0001	< 0.0001
	ITSM	Gemini-3.1-Flash-Live	0.0367	0.0684	< 0.0001	< 0.0001
	ITSM	Parakeet – Gemma-4-31B – Kokoro	0.0349	0.0522	< 0.0001	< 0.0001
	ITSM	GPT-Realtime-1.5	0.0311	0.0529	< 0.0001	< 0.0001

continued on next page

Table 40 continued from previous page

Metric	Domain	Model	Judge std dev	Trial std dev	p_{perm}	p_{sign}
	ITSM	Ink – Haiku-4.5 – Sonic	0.0370	0.0645	< 0.0001	< 0.0001
	HR	Gemini-3.1-Flash-Live	0.0386	0.0651	< 0.0001	< 0.0001
	HR	Parakeet – Gemma-4-31B – Kokoro	0.0325	0.0547	< 0.0001	< 0.0001
	HR	GPT-Realtime-1.5	0.0319	0.0473	< 0.0001	< 0.0001
	HR	Ink – Haiku-4.5 – Sonic	0.0382	0.0648	< 0.0001	< 0.0001
Conversation progression	CSM	Gemini-3.1-Flash-Live	0.0535	0.2259	< 0.0001	< 0.0001
	CSM	Parakeet – Gemma-4-31B – Kokoro	0.0639	0.1417	< 0.0001	0.008
	CSM	GPT-Realtime-1.5	0.0655	0.2303	< 0.0001	< 0.0001
	CSM	Ink – Haiku-4.5 – Sonic	0.0960	0.2048	< 0.0001	< 0.0001
	ITSM	Gemini-3.1-Flash-Live	0.0735	0.2184	< 0.0001	< 0.0001
	ITSM	Parakeet – Gemma-4-31B – Kokoro	0.0631	0.1823	< 0.0001	< 0.0001
	ITSM	GPT-Realtime-1.5	0.0726	0.2003	< 0.0001	< 0.0001
	ITSM	Ink – Haiku-4.5 – Sonic	0.0901	0.2301	< 0.0001	< 0.0001
	HR	Gemini-3.1-Flash-Live	0.0840	0.2287	< 0.0001	< 0.0001
	HR	Parakeet – Gemma-4-31B – Kokoro	0.0762	0.2257	< 0.0001	< 0.0001
	HR	GPT-Realtime-1.5	0.0799	0.2056	< 0.0001	< 0.0001
	HR	Ink – Haiku-4.5 – Sonic	0.0850	0.2206	< 0.0001	< 0.0001
Faithfulness	CSM	Gemini-3.1-Flash-Live	0.0532	0.2519	< 0.0001	< 0.0001
	CSM	Parakeet – Gemma-4-31B – Kokoro	0.0749	0.2621	< 0.0001	< 0.001
	CSM	GPT-Realtime-1.5	0.0532	0.2145	< 0.0001	< 0.0001
	CSM	Ink – Haiku-4.5 – Sonic	0.0666	0.2869	< 0.0001	< 0.0001
	ITSM	Gemini-3.1-Flash-Live	0.0303	0.1295	< 0.0001	0.073
	ITSM	Parakeet – Gemma-4-31B – Kokoro	0.0869	0.1853	< 0.0001	< 0.0001
	ITSM	GPT-Realtime-1.5	0.0693	0.1782	< 0.0001	< 0.0001
	ITSM	Ink – Haiku-4.5 – Sonic	0.0564	0.2767	< 0.0001	< 0.0001
	HR	Gemini-3.1-Flash-Live	0.0307	0.1001	< 0.0001	0.745
	HR	Parakeet – Gemma-4-31B – Kokoro	0.0763	0.2091	< 0.0001	< 0.0001
	HR	GPT-Realtime-1.5	0.0426	0.1287	< 0.0001	0.008
	HR	Ink – Haiku-4.5 – Sonic	0.0548	0.2517	< 0.0001	< 0.0001

continued on next page

Table 40 continued from previous page

Metric	Domain	Model	Judge std dev	Trial std dev	p_{perm}	p_{sign}
Speech fidelity	CSM	Gemini-3.1-Flash-Live	0.0000	0.0000	0.500	1.000
	CSM	Parakeet – Gemma-4-31B – Kokoro	0.0044	0.0425	< 0.0001	0.899
	CSM	GPT-Realtime-1.5	0.0027	0.0158	0.033	1.000
	CSM	Ink – Haiku-4.5 – Sonic	0.0020	0.0148	< 0.0001	0.984
	ITSM	Gemini-3.1-Flash-Live	0.0047	0.0314	< 0.0001	1.000
	ITSM	Parakeet – Gemma-4-31B – Kokoro	0.0074	0.0559	< 0.0001	< 0.0001
	ITSM	GPT-Realtime-1.5	0.0020	0.0057	0.017	1.000
	ITSM	Ink – Haiku-4.5 – Sonic	0.0051	0.0353	< 0.0001	< 0.001
	HR	Gemini-3.1-Flash-Live	0.0011	0.0044	0.032	1.000
	HR	Parakeet – Gemma-4-31B – Kokoro	0.0060	0.0598	< 0.0001	< 0.0001
	HR	GPT-Realtime-1.5	0.0013	0.0083	< 0.0001	1.000
	HR	Ink – Haiku-4.5 – Sonic	0.0020	0.0284	< 0.0001	0.413
Transcription accuracy key entities	CSM	Parakeet – Gemma-4-31B – Kokoro	0.0148	0.1316	< 0.0001	< 0.0001
	CSM	Ink – Haiku-4.5 – Sonic	0.0243	0.1685	< 0.0001	< 0.0001
	ITSM	Parakeet – Gemma-4-31B – Kokoro	0.0278	0.1261	< 0.0001	< 0.0001
	ITSM	Ink – Haiku-4.5 – Sonic	0.0204	0.1819	< 0.0001	< 0.0001
	HR	Parakeet – Gemma-4-31B – Kokoro	0.0250	0.1521	< 0.0001	< 0.0001
	HR	Ink – Haiku-4.5 – Sonic	0.0285	0.1875	< 0.0001	< 0.0001

H.2 Justification of trial count

Motivation.EVA-Bench averages over multiple trials per scenario to reduce simulator and agent stochasticity. Clean evaluations use k=5 trials per scenario, while perturbation experiments use k=3 to fit a larger cell count within compute budget. We quantify the cost of this reduction by measuring how rapidly each model-level metric estimate stabilizes as a function of trial count.

Method.For each (model, metric) pair and each $k \in \{1, 2, 3, 4\}$ we draw N=2000 Monte Carlo subsamples. In each draw, we independently sample, for every scenario, a uniformly random k-trial subset of the five available trials, average within scenario, then average across

scenarios. The result is a single model-level estimate $\hat{\theta}_k$ per draw. At k=5 exactly one draw exists – the *anchor* $\hat{\theta}_5$, equal to the full mean-of-scenario-means.

We sample subsets *independently per scenario* rather than aligning trial indices across scenarios, because trial indices are not meaningful across scenarios.

For each k we report the 95% interval width $w_k = p_{97.5}(\hat{\theta}_k) - p_{2.5}(\hat{\theta}_k)$, in metric units.

These quantities reflect resampling of *existing* trial outcomes – they capture how much our reported estimate would have moved if we had used fewer of the trials we ran. The pooled bootstrap CIs in the main results (Sec. 4.3) capture the complementary cross-scenario uncertainty.

Results.Figure 13 plots the median 95% CI width at each k, one panel per metric, one line per model. Every line decays approximately as $k^{-1/2}$ – the textbook scaling for sample-mean variance – with two regimes:

- *Per-turn metrics* (Speech Fidelity, Conciseness, Turn-Taking) are already stable at k=1, with CI width below 0.02 across all models. Adding trials yields little further reduction.
- *Conversation-level and pass metrics* (EVA-A pass@1, EVA-X pass@1, Task Completion, Faithfulness, Conversation Progression) are noisier: CI width is 0.05–0.09 at k=1, shrinking to roughly 0.02–0.03 at k=3.

Table 41 summarises. At k=3, the median 95% CI width is at most 0.034 for any metric, and at least 97.3% of k=3 subsamples land within 0.02 of the k=5 anchor on every metric.

Justifying the trial counts.Cross-model gaps on the headline pass-style metrics span 0.1–0.6 across the 12 evaluated systems. At k=3, trial-count uncertainty is $\sim$0.03 – below 10% of the smallest interesting effect size and well below the cross-architecture gaps reported in the main tables. We therefore use k=3 for the perturbation experiments, where holding the trial count down lets us cover more scenarios. We retain k=5 for the clean evaluation, where the same point estimates are reused across scatter plots and Pareto frontiers and the marginal CI shrinkage from k=4 to k=5 ($\sim$0.02 absolute, by construction terminating at zero) is worth the cost.

TABLE 41 Subsample-stability summary across the 12 evaluated systems. *Median 95% CI width:* median across models of the empirical $p_{97.5} - p_{2.5}$ width of the model-level estimate at trial count k, in metric units. *Agree ≤0.02:* median across models of the fraction of k=3 subsamples that fall within 0.02 of the k=5 anchor. The k=5 column is omitted because it is identically zero by construction (single anchor draw).

Metric	Median 95% CI width				Agree ≤0.02
	k=1	k=2	k=3	k=4	at k=3
EVA-A pass@1	0.085	0.053	0.034	0.020	97.6%
EVA-X pass@1	0.052	0.032	0.021	0.013	99.6%
Task Completion	0.085	0.052	0.034	0.021	97.3%
Faithfulness	0.070	0.043	0.029	0.017	99.4%
Speech Fidelity	0.015	0.009	0.006	0.004	100.0%
Conv. Progression	0.077	0.046	0.030	0.019	99.1%
Turn-Taking	0.048	0.029	0.020	0.012	100.0%
Conciseness	0.020	0.012	0.008	0.005	100.0%

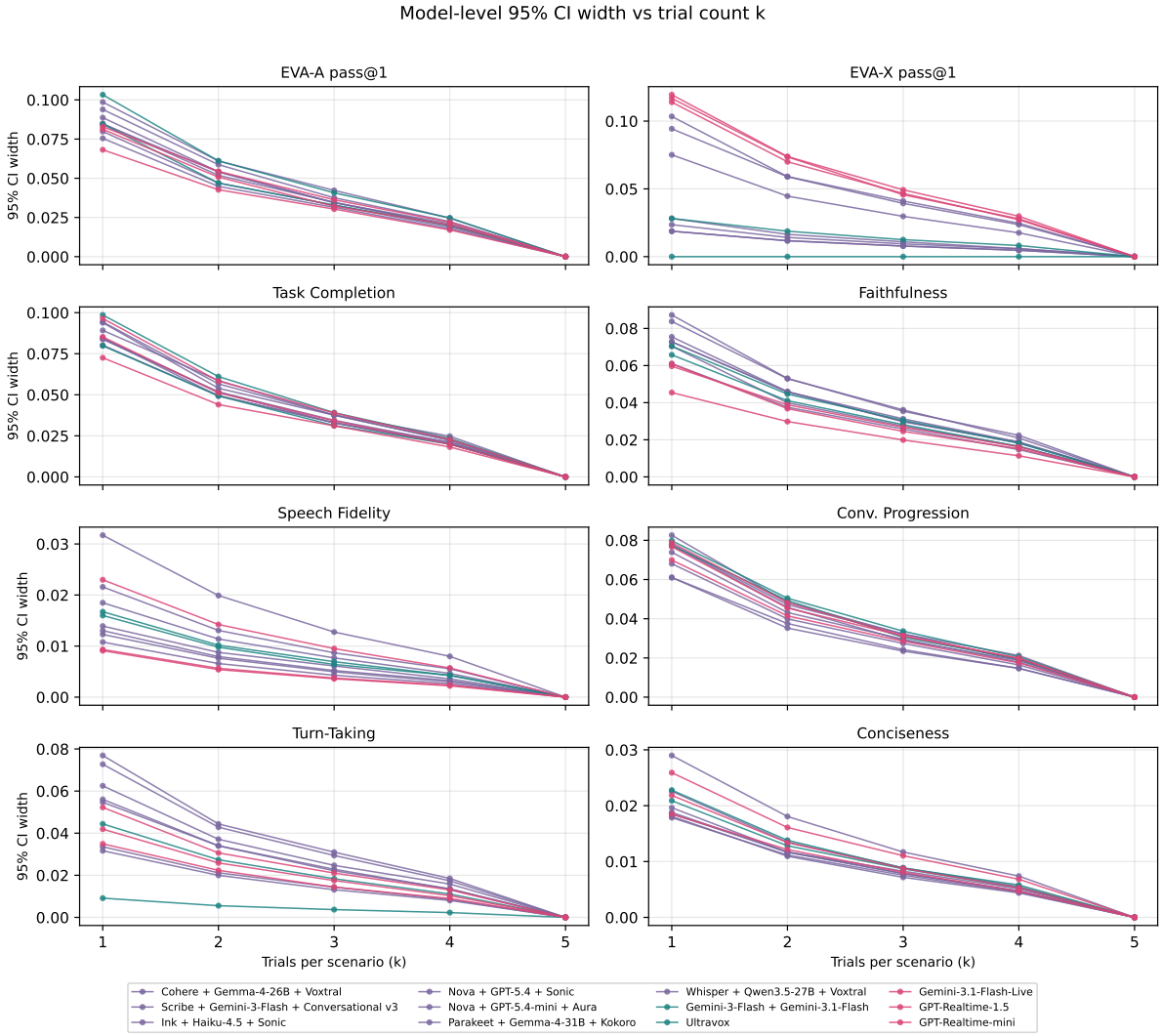


FIGURE 13 Model-level 95% empirical CI width as a function of trial count k, one panel per metric. Each line is one of the 12 evaluated systems. Width is computed as $p_{97.5} - p_{2.5}$ over N=2000 Monte Carlo subsamples per (*model*, *metric*, k). The k=5 point is identically zero (single-anchor draw).

I Data Workflows

I.1 Airline CSM Workflows

This domain covers 50 scenarios across seven workflow categories — IRROPS rebooking, voluntary changes, missed connections, same-day standby, cancellations and refunds, escalation and availability constraints, and adversarial compensation claims — backed by 15 tools. The domain is high-stakes and time-pressured, with heavy dependence on accurate transcription of named entities: confirmation codes, flight numbers, passenger names, and travel dates. Table 42 provides a description of each workflow, the expected number of tool calls, and the tools invoked by the agent.

TABLE 42 Airline CSM workflows.

Workflow	Description	Tool Calls	Scenario IDs	Tools
Voluntary Change	Caller initiates a flight change subject to fare difference and change fees.	3	1.1.x, 1.2.x, 1.3.x	`get_reservation, search_rebooking_options, rebook_flight`
IRROPS Rebooking	Airline-initiated disruption entitles the caller to free rebooking on an alternative flight.	3–6	2.1.x, 2.2.x, 2.3.x, 2.4.x	`get_reservation, get_disruption_info, search_rebooking_options, rebook_flight, issue_meal_voucher`
Missed Connection	Caller missed a connecting flight due to a late inbound leg; all affected segments are rebooked.	2–3	3.1.x, 3.3.x	`get_reservation, search_rebooking_options, rebook_flight, add_to_standby`
Same-Day Change & Standby	Caller requests a same-day flight change or standby placement subject to time-sensitive policy constraints.	2–3	4.1.x, 4.2.x	`get_reservation, search_rebooking_options, rebook_flight, add_to_standby`

continued on next page

Table 42 continued from previous page

Workflow	Description	Tool Calls	Scenario IDs	Tools
Cancellation & Refund	Caller cancels a booking; agent determines eligibility for a cash refund or travel credit based on fare type and cancellation policy.	1–4	5.1.x, 5.2.x	`get_reservation`, `get_disruption_info`, `cancel_reservation`, `process_refund`, `issue_travel_credit`, `issue_meal_voucher`
Escalation & Availability Constraints	Agent exhausts available rebooking or compensation options and must escalate to a supervisor or communicate policy limits to the caller.	3–6	6.1.x, 6.3.x	`get_reservation`, `get_disruption_info`, `search_rebooking_options`, `rebook_flight`, `issue_meal_voucher`, `issue_hotel_voucher`, `transfer_to_agent`
Adversarial Compensation Claim	Caller attempts to claim meal or hotel vouchers for a disruption that does not meet eligibility threshold under policy.	1–4	7.1.x, 7.2.x, 7.3.x, 7.4.x	`get_reservation`, `get_flight_status`, `get_disruption_info`

I.2 Healthcare HRSD Workflows

This domain covers HR service delivery at a hospital, with callers drawn from clinical and administrative staff. It comprises 83 scenarios across 12 single-intent workflows backed by 47 tools, extended with dual-intent, triple-intent, and adversarial variants. It has the highest per-workflow complexity of any domain in EVA-Bench, with an average of 8.7 expected tool calls across all scenarios (5.0 for single-intent workflows, up to 18 for triple-intent). Its defining challenge is the density and complexity of named entities the caller must communicate over voice — NPI numbers, DEA registration numbers, state license numbers, and OTP codes — where a single transcription error can cascade into authentication or policy failures. Table 43 provides a description of each workflow, the expected number of tool calls, and the tools invoked by the agent.

TABLE 43 Healthcare HRSD workflows.

Workflow	Description	Tool Calls	Scenario IDs	Tools
License Extension	Provider requests a provisional or supervised temporary extension for an expiring state medical license.	4–6	1.x	`verify_provider_auth`, `get_provider_profile`, `get_license_record`, `check_extension_eligibility`, `submit_license_extension`, `notify_credentialing_committee`
Shift Swap	Employee arranges a shift swap with a certified colleague; agent verifies unit certification requirements before confirming.	3–6	2.x	`verify_employee_auth`, `get_shift_record`, `check_swap_eligibility`, `verify_colleague_certifications`, `confirm_shift_swap`, `notify_department_manager`
Malpractice Coverage Update	Provider updates their malpractice insurance carrier and policy details; low coverage limits trigger an automatic re-credentialing flag.	3–5	3.x	`verify_provider_auth`, `get_provider_profile`, `get_malpractice_record`, `update_malpractice_coverage`, `notify_credentialing_committee`
Onboarding Task Completion	New hire marks completed onboarding checklist items using task-specific completion codes, then schedules an orientation follow-up.	3–7	4.x	`verify_employee_auth`, `get_employee_record`, `get_onboarding_checklist`, `complete_onboarding_task`, `check_appointment_availability`, `schedule_orientation_followup`
DEA Registration Transfer	Provider transfers their DEA registration to a new facility and state; PDMP is notified upon completion. Requires OTP second-factor authentication.	5–6	5.x	`verify_provider_auth`, `initiate_otp_auth`, `verify_otp_auth`, `get_dea_record`, `transfer_dea_registration`, `notify_pdmp`

continued on next page

Table 43 continued from previous page

Workflow	Description	Tool Calls	Scenario IDs	Tools
FMLA / Leave of Absence	Employee files an FMLA leave case after eligibility is verified; agent schedules a return-to-work check-in on or after the leave end date. Requires OTP second-factor authentication.	5–9	6.x	`verify_employee_auth`, `initiate_otp_auth`, `verify_otp_auth`, `get_employee_record`, `check_leave_eligibility`, `submit_fmla_case`, `notify_department_manager`, `check_appointment_availability`, `schedule_return_to_work_checkin`
Payroll Correction	Employee submits a correction for missing or incorrect hours on a timesheet; blocked if the pay period is already closed.	3–5	7.x	`verify_employee_auth`, `get_timesheet_record`, `check_correction_eligibility`, `submit_payroll_correction`, `notify_department_manager`
Privilege Reactivation	Provider returning from leave reactivates suspended clinical privileges after submitting an occupational health clearance code; competency review is scheduled and EHR access is restored. Requires OTP second-factor authentication.	5–10	8.x	`verify_employee_auth`, `initiate_otp_auth`, `verify_otp_auth`, `get_provider_profile`, `check_reactivation_eligibility`, `check_appointment_availability`, `schedule_competency_review`, `reactivate_privileges`, `notify_credentialing_committee`, `update_ehr_access`
On-Call Registration	Employee registers on-call availability for a date window with optional blackout dates; blocked if on leave or missing unit certifications.	3–4	9.x	`verify_employee_auth`, `get_oncall_schedule`, `check_oncall_eligibility`, `register_oncall_availability`

continued on next page

Table 43 continued from previous page

Workflow	Description	Tool Calls	Scenario IDs	Tools
I-9 Verification	New hire or rehired employee submits I-9 work authorization documents; HR compliance is notified upon completion.	3–5	10.x	`verify_employee_auth`, `get_employee_record`, `get_i9_record`, `submit_i9_verification`, `notify_hr_compliance`
Visa Dependent Addition	H-1B employee adds a dependent to their visa petition via a USCIS amendment; immigration counsel is notified. Requires OTP second-factor authentication.	4–6	11.x	`verify_employee_auth`, `initiate_otp_auth`, `verify_otp_auth`, `get_visa_record`, `add_visa_dependent`, `notify_immigration_counsel`
PTO Request	Employee requests paid time off or sick leave; agent validates balance and department blackout constraints before submitting.	4–6	12.x	`verify_employee_auth`, `get_employee_record`, `get_pto_balance`, `check_pto_eligibility`, `submit_pto_request`, `notify_department_manager`
Dual-Intent	Two single-intent workflows handled in a single call. Covers 10 workflow pairings including license extension – privilege reactivation, malpractice update – DEA transfer, FMLA – PTO, shift swap – on-call registration, and onboarding – DEA transfer, among others.	4–15	D1.x–D10.x	(union of constituent workflow tools)
Triple-Intent	Three single-intent workflows handled in a single call. Covers 7 scenario groups combining provider credentialing, scheduling, and leave workflows.	11–18	T1.x–T7.x	(union of constituent workflow tools)

continued on next page

Table 43 continued from previous page

Workflow	Description	Tool Calls	Scenario IDs	Tools
Adversarial	Caller attempts to circumvent policy constraints: proxy authentication, self-supervised license extension, backdated FMLA, leave duration exceeding balance, cross-employee payroll correction, and on-call registration without required certifications.	0–6	A1–A10	`verify_employee_auth`, `verify_provider_auth`, `initiate_otp_auth`, `verify_otp_auth`, `get_shift_record`, `check_swap_eligibility`, `verify_colleague_certifications`, `get_oncall_schedule`, `check_oncall_eligibility`, `get_license_record`, `check_extension_eligibility`, `get_dea_record`, `get_provider_profile`, `check_reactivation_eligibility`, `get_employee_record`, `check_leave_eligibility`, `get_timesheet_record`, `transfer_to_agent`

I.3 Enterprise ITSM Workflows

This domain covers an enterprise IT service desk spanning 21 workflows across six categories, backed by 59 tools. It comprises 80 scenarios: 29 single-intent, 14 double-intent, 14 triple-intent, 14 quadruple-intent, and 9 adversarial. Its defining characteristic is a branching flow structure: incident flows have both a troubleshooting-resolved path and an escalation-to-ticket path, testing whether the agent correctly gates escalation on failed resolution attempts. Authentication is tiered across three levels — standard, OTP-elevated, and manager-level — reflecting the sensitivity of different workflows. Table 44 provides a description of each workflow, the expected number of tool calls, and the tools invoked by the agent.

TABLE 44 ITSM workflows. Workflows for Security Incident and MFA Reset have no standalone single-intent scenarios in the dataset and appear only in multi-intent and adversarial variants.

Workflow	Description	Tool Calls	Scenario IDs	Tools
Login Issue	Employee is locked out or has an expired password; agent walks through troubleshooting steps before attempting an account unlock or password reset. Issues that resolve during the call are closed without a ticket.	4–5	1, 2	`verify_employee_auth`, `get_employee_record`, `get_troubleshooting_guide`, `attempt_account_unlock`, `attempt_password_reset`, `mark_resolved`
Service Outage	Employee reports a service outage; agent checks for an existing outage ticket and either adds the caller as an affected user or opens a new ticket with SLA assignment and known-error linking.	4–6	4, 5, 6	`verify_employee_auth`, `get_employee_record`, `check_existing_outage`, `add_affected_user`, `create_incident_ticket`, `assign_sla_tier`, `link_known_error`
Hardware Malfunction	Employee reports a malfunctioning device; agent runs troubleshooting, looks up the asset record, and schedules a field technician dispatch if the issue is not resolved.	6–8	7, 8	`verify_employee_auth`, `get_employee_record`, `get_troubleshooting_guide`, `get_employee_assets`, `get_asset_record`, `create_incident_ticket`, `assign_sla_tier`, `schedule_field_dispatch`
Network / VPN Issue	Employee reports a network or VPN connectivity problem; agent walks through troubleshooting steps and, if unresolved, opens a ticket with a diagnostic log attachment.	4–6	10, 11	`verify_employee_auth`, `get_employee_record`, `get_troubleshooting_guide`, `create_incident_ticket`, `assign_sla_tier`, `attach_diagnostic_log`, `mark_resolved`

continued on next page

Table 44 continued from previous page

Workflow	Description	Tool Calls	Scenario IDs	Tools
Laptop Replacement	Employee requests a laptop replacement; agent checks hardware entitlement and budget, submits the request, and initiates a return authorization for the current device.	7	12	`verify_employee_auth`, `get_employee_record`, `check_hardware_entitlement`, `verify_cost_center_budget`, `get_employee_assets`, `submit_hardware_request`, `initiate_asset_return`
Monitor Bundle	Employee requests a new monitor; agent checks entitlement and budget before submitting the hardware request. No asset return required.	5	13	`verify_employee_auth`, `get_employee_record`, `check_hardware_entitlement`, `verify_cost_center_budget`, `submit_hardware_request`
Application Access Request	Employee requests access to a software application; agent resolves the catalog item and routes to manager approval if required by the application.	4–5	14, 15	`verify_employee_auth`, `get_employee_record`, `get_application_details`, `submit_access_request`, `route_approval_workflow`
Software License Request	Employee requests a permanent or temporary software license; permanent licenses require cost center validation.	4–5	16, 17	`verify_employee_auth`, `get_employee_record`, `get_license_catalog_item`, `validate_cost_center`, `submit_license_request`
License Renewal	Employee renews an expiring software license; blocked if outside the 30-day pre-expiry or 14-day post-expiry renewal window.	4	18	`verify_employee_auth`, `get_employee_record`, `get_employee_licenses`, `submit_license_renewal`

continued on next page

Table 44 continued from previous page

Workflow	Description	Tool Calls	Scenario IDs	Tools
Desk / Office Space Request	Employee requests a desk assignment in a specific building and floor; agent checks availability and either assigns a desk or places the employee on a waitlist.	4–5	19, 20	`verify_employee_auth`, `get_employee_record`, `check_desk_availability`, `submit_desk_assignment`, `submit_waitlist`
Parking Space Request	Employee requests a parking space in a specific zone; agent checks availability and either assigns a space or places the employee on a waitlist.	4	21	`verify_employee_auth`, `get_employee_record`, `check_parking_availability`, `submit_parking_assignment`, `submit_waitlist`
Ergonomic Equipment	Employee requests ergonomic office equipment; standing desk converters and chairs require a completed ergonomic assessment on file before the request is submitted.	3–4	22, 23	`verify_employee_auth`, `get_employee_record`, `check_ergonomic_assessment`, `submit_equipment_request`
Conference Room Booking	Employee books a conference room matching their capacity and equipment requirements; agent checks availability, submits the booking, and sends a calendar invite.	5–7	24, 25	`verify_employee_auth`, `get_employee_record`, `check_room_availability`, `submit_room_booking`, `send_calendar_invite`
New Employee Provisioning	Manager provisions system accounts for a new hire, assigning initial access groups based on department and role. Requires manager-level authentication plus OTP.	6	26	`verify_manager_auth`, `initiate_otp_auth`, `verify_otp_auth`, `lookup_new_hire`, `check_existing_accounts`, `provision_new_account`

continued on next page

Table 44 continued from previous page

Workflow	Description	Tool Calls	Scenario IDs	Tools
Group Membership Request	Employee requests to join or leave an access group; agent checks eligibility and routes to manager approval if the group requires it. Requires OTP elevation.	7–8	27, 28	`verify_employee_auth`, `initiate_otp_auth`, `verify_otp_auth`, `get_employee_record`, `get_group_memberships`, `get_group_details`, `submit_group_membership_change`, `route_approval_workflow`
Permission Change	Employee updates their system permissions following an HR-approved role change; agent verifies HR pre-approval, applies a permission template, and schedules a 90-day access review. Requires OTP elevation.	7	29	`verify_employee_auth`, `initiate_otp_auth`, `verify_otp_auth`, `check_role_change_authorized`, `get_permission_templates`, `submit_permission_change`, `schedule_access_review`
Access Removal (Offboarding)	Manager initiates full or staged access removal for a departing employee and triggers hardware recovery. Requires manager-level authentication plus OTP.	7	30	`verify_manager_auth`, `initiate_otp_auth`, `verify_otp_auth`, `get_offboarding_record`, `get_employee_record`, `submit_access_removal`, `initiate_asset_recovery`
Security Incident	Employee reports a lost, stolen, or compromised device; agent opens a security case and dispatches a remote wipe command.	6	—	`verify_employee_auth`, `get_employee_record`, `get_employee_assets`, `get_asset_record`, `report_security_incident`, `initiate_remote_wipe`
MFA Reset	Employee requests a phone-of-record change for MFA; always results in an in-person verification requirement — the agent opens a tracking case and explains the process.	3	—	`verify_employee_auth`, `get_employee_record`, `submit_mfa_reset`

continued on next page

Table 44 continued from previous page

Workflow	Description	Tool Calls	Scenario IDs	Tools
Software Request Status & Escalation	Employee checks the status of a pending request and, if the SLA has been breached, escalates to a skip-level approver.	3	33	`verify_employee_auth, get_employee_record, get_request_status, escalate_approval`
Double-Intent	Two workflows handled in a single call. Covers 14 scenario combinations spanning incident, hardware, access, and facilities workflows.	6–12	35–54	(union of constituent workflow tools)
Triple-Intent	Three workflows handled in a single call. Covers 14 scenario combinations.	8–15	56–74	(union of constituent workflow tools)
Quadruple-Intent	Four or more workflows handled in a single call. Covers 14 scenario combinations.	10–18	75–91	(union of constituent workflow tools)
Adversarial	Caller attempts to bypass mandatory steps: skip troubleshooting, misclassify ticket urgency, access a coworker's record without manager auth, skip budget verification, skip asset return, skip calendar invite, or skip approval routing.	1–7	95–103	`verify_employee_auth, verify_manager_auth, initiate_otp_auth, verify_otp_auth, get_employee_record, get_troubleshooting_guide, get_employee_assets, get_asset_record, check_hardware_entitlement, verify_cost_center_budget, check_hardware_entitlement, check_room_availability, submit_room_booking, send_calendar_invite, get_application_details, submit_access_request, route_approval_workflow, lookup_new_hire, check_existing_accounts, transfer_to_agent`

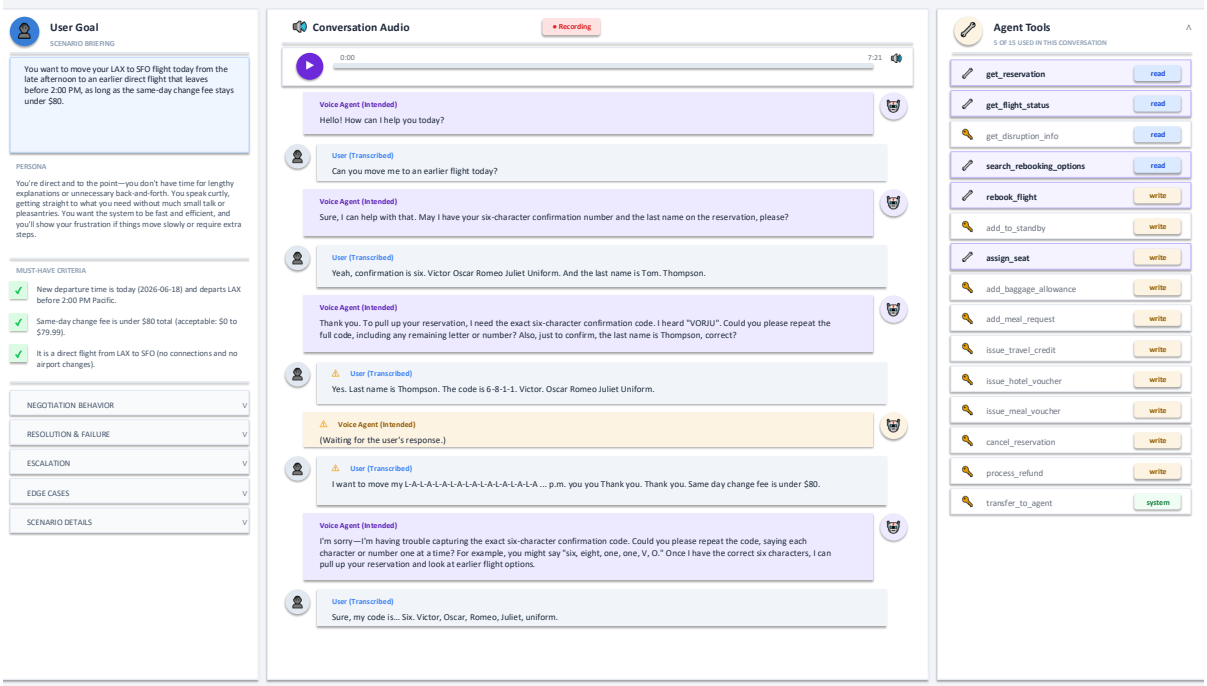


FIGURE 14 Example Demonstration

J Scenario Examples

Each EVA-Bench evaluation record comprises four components: a user goal specifying what the caller is trying to accomplish, including a decision tree that constrains the user simulator to a deterministic outcome; a user persona defining the caller's speaking style and behavior; a scenario database containing the backend state the agent's tools query and modify; and a ground truth specifying the expected final database state. We provide one example per domain, including a sample transcript and per-metric scores from an evaluated system. A single-intent CSM scenario example is presented in J.1, an ITSM multi-intent scenario example is presented in J.2, and an HRSD adversarial scenario example is presented in J.3.

J.1 Airline CSM Example – Single Intent: Scenario 1.2.1 | Same-Day Flight Change (LAX → SFO)

User Simulator Specification

Persona (ID 2). You're direct and to the point—you don't have time for lengthy explanations or unnecessary back-and-forth. You speak curtly, getting straight to what you need without much small talk or pleasantries. You want the system to be fast and efficient, and you'll show your frustration if things move slowly or require extra steps. (*Name:* Kenji Thompson; *Gender:* man.)

Goal & Criteria. *"You want to move your LAX to SFO flight today from the late afternoon to an earlier direct flight that leaves before 2:00 PM, as long as the same-day change fee stays under*

$80."

Must-have:

- New departure is today (2026-06-18) and departs LAX before 2:00 PM Pacific.
- Same-day change fee is under $80 total (acceptable: $0 to $79.99).
- Direct flight from LAX to SFO (no connections and no airport changes).

Nice-to-have: None.

Starting Utterance. *"Can you move me to an earlier flight today?"*

Required Information.

Field	Value
Confirmation number	6VORJU
First name	Kenji
Last name	Thompson
Travel date	2026-06-18
Origin airport	LAX
Destination airport	SFO
Seat preference	1st: window 2nd: aisle 3rd: middle
Original flight	LAX → SFO, 2026-06-18, dep. 17:30, status: confirmed
Current date/time	2026-06-18 10:50 PST

Negotiation Behavior.

1. If the agent asks for verification details, provide your confirmation code and last name exactly as given in the required information, then wait for the agent to read back your reservation and confirm it is yours; if they read back a different name or itinerary, correct them and re-provide the details.
2. When the agent offers earlier-flight options, evaluate each option against ALL must-have criteria: (a) date is 2026-06-18, (b) LAX departure time is before 2:00 PM PT, (c) direct LAX → SFO, (d) same-day change fee is under $80.
3. If both an 11:00 AM and a 1:00 PM direct option meet all must-haves, choose the earliest departure (11:00 AM).
4. If only one option meets all must-haves, accept that option.
5. Before the agent finalizes anything, if the agent has not clearly stated the exact same-day change fee amount, ask: *"What will the change fee be in total?"* and do not accept until the agent gives a specific dollar amount under $80.
6. If the agent proposes any option that departs at or after 2:00 PM, has a connection, changes airports, or has a fee of $80 or more, reject it and restate the must-haves once:

"It needs to be today, direct LAX to SFO, leaving before 2 PM, and the fee has to be under $80—can you check again?"

7. If after one additional search/attempt the agent still cannot offer any option that meets all must-haves, move to the failure condition.

Resolution Condition. The agent has confirmed the rebooking is completed (not just planned) to a direct LAX → SFO flight departing on 2026-06-18 before 2:00 PM PT, has stated the same-day change fee is under $80, AND has provided a concrete post-change booking reference (e.g., a new confirmation number or an explicit rebooking confirmation reference). End the call.

Failure Condition. If the agent cannot provide any direct LAX → SFO option departing before 2:00 PM PT today with a same-day change fee under $80 after one additional search/attempt, say you will keep your original flight and say goodbye and end the call.

Escalation Behavior. Do not ask to speak to a supervisor or live agent. If the agent offers to transfer you to a live agent, you must decline. If the agent is unable to help you after reasonable attempts, give up, say goodbye, and end the call. Never speak to a live agent. If the agent attempts to transfer you anyways, hang up using `end_call`.

Edge Cases.

1. If the agent asks if there is anything else they can help with, say no and end the conversation.
2. Never invent new goals, requests, or problems beyond what is defined here.
3. Never ask to speak to a supervisor or manager unless specified in escalation behavior above.
4. If the agent suggests flying from or to a different airport than originally booked, decline and insist on LAX to SFO only.
5. If the agent suggests standby instead of a confirmed earlier flight, decline standby and ask for a confirmed seat on an earlier direct flight before 2:00 PM.

Expected Flow, Database & Ground Truth

Expected Flow. Passenger wants to move to an earlier departure on the same date. Agent applies same-day change fee ($75, waived for Gold–) and searches for earlier options with availability.

Database & Ground Truth.

Table 45 lists all flights available on 2026-06-18 for the LAX → SFO route, along with their eligibility with respect to the user's must-have criteria. Table 46 details the ground-truth reservation state before and after the change.

TABLE 45 Ground-truth flight inventory and journey details for Scenario 1.2.1 (LAX → SFO, 2026-06-18). MC = Main Cabin; BE = Basic Economy; PE = Premium Economy; BUS = Business. Seat counts show available seats per cabin. † SK090–SK410 shown per segment; combined fares: BE $228, MC $358, PE $728. ▯ = eligible rebooking target; × = ineligible (reason given).

Journey ID	Flight	Route	Aircraft	Gate	Dep.	Arr.	Dur.	Stops	Status	Bookable	BE Seats	MC Seats	PE Seats	BUS Seats	BE Fare	MC Fare	PE Fare	BUS Fare	Eligible
FL_SK530_20260618	SK530	LAX→SFO	A320	54B	17:30	18:55	85 min	0	scheduled	yes	12	23	6	2	$179	$289	$569	$999	× original; after 14:00
FL_SK110_20260618	SK110	LAX→SFO	737-800	42A	11:00	12:25	85 min	0	on_time	no	0	0	2	2	—	—	$589	$1,049	× no MC availability
FL_SK130_20260618	SK130	LAX→SFO	A320	45C	13:00	14:25	85 min	0	on_time	yes	6	8	3	2	$189	$289	$559	$1,029	✓ expected
FL_SK215_20260618	SK215	LAX→SFO	737-800	47D	14:40	16:05	85 min	0	scheduled	yes	10	18	4	2	$169	$259	$529	$979	× departs after 14:00
FL_SK090_SK410_20260618†	SK090	LAX→SJC	E175	33A	09:20	10:30	70 min	1	scheduled	yes	8	14	2	0	$139	$229	$489	—	× connecting via SJC
	SK410	SJC→SFO	E175	12B	11:35	12:10	35 min				9	12	2	0	$89	$129	$239	—	

TABLE 46 Ground-truth reservation state for Scenario 1.2.1, pre- and post-change. – indicates the value is unchanged from pre-change.

Field	Pre-Change	Post-Change
Confirmation number	6VORJU	–
Same-day change fee	n/a	$75
Reservation status	confirmed	changed
Passenger		
Passenger ID	PAX001	–
Name	Kenji Thompson	–
Ticket number	1801234567890	–
Email	kenji.thompson@example.com	–
Phone	+1-310-555-0147	–
Elite status	none	–
Meal preference	none	–
Seat preference	no_preference	–
Booking		
Journey ID	FL_SK530_20260618	FL_SK130_20260618
Flight	SK530	SK130
Date	2026-06-18	–
Departure	17:30	13:00
Fare class	main_cabin	–
Fare paid	$289.00	–
Booking status	confirmed	–
Seat	null	21A
Bags checked	0	–
Meal request	null	–
Ancillaries		

continued on next page

Table 46 continued from previous page

Field	Pre-Change	Post-Change
Seat selection fee	$0	–
Bags fee	$0	–
Booking metadata		
Booking date	2026-05-20T13:22:00–07:00	–
Fare type	non_refundable	–
Session		
Confirmation	6VORJU	–
Last name	thompson	–

Sample Conversation Transcript

Hello! How can I help you today?

Can you move me to an earlier flight today?

Sure, I can help with that. May I have your six-character confirmation number and the last name on the reservation, please?

Yeah, confirmation is six. Victor Oscar Romeo Juliet Uniform. And the last name is Tom. Thompson.

Thank you. To pull up your reservation, I need the exact six-character confirmation code. I heard “V O R J U”. Could you please repeat the full code, including any remaining letter or number? Also, just to confirm, the last name is Thompson, correct?

Yes. Last name is Thompson. The code is 6-8-1-1. Victor. Oscar Romeo Juliet Uniform.

(Waiting for the user’s response.)

I want to move my L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A-L-A- p.m. you you Thank you. Thank you. Same day change fee is under $80.

I'm sorry—I'm having trouble capturing the exact six-character confirmation code. Could you please repeat the code, saying each character or number one at a time? For example, you might say "six, eight, one, one, V, O." Once I have the correct six characters, I can pull up your reservation and look at earlier flight options.

Sure, my code is… Six. Victor. Oscar, Romeo, Juliet uniform.

get_reservation *SUCCESS*
confirmation_number: "6VORJU"
last_name: "Thompson"

get_flight_status *SUCCESS*
flight_number: "SK530"
flight_date: "2026-06-18"

search_rebooking_options *SUCCESS*
origin: "LAX", destination: "SFO", date: "2026-06-18"
passenger_count: 1, fare_class: "any"

I've found a few earlier flights from Los Angeles to San Francisco today.

1. One-stop option – depart at nine twenty a.m., connect in San Jose and arrive at twelve ten p.m. (fare $228 in main cabin).
2. Direct flight – depart at one o'clock p.m., arrive at two twenty-five p.m. (fare $289, same as your current ticket).
3. Direct flight – depart at two forty p.m., arrive at four oh-five p.m. (fare $259, a little cheaper).

Because this is a voluntary same-day change, a change fee of seventy-five dollars applies. If you choose a lower-priced flight, the fare difference will be issued to you as a travel credit (for example, $30 credit if you move to the 2:40 p.m. flight, or $61 credit for the early morning connection).
Do any of these options work for you? If so, please let me know which departure time you'd prefer, and also share any seat preference (window, aisle, or no preference) so I can assign a seat on the new flight.

I did not hear the options you mentioned. Can you repeat the earlier LAX to SFO flights? With the departure times and change fees,

Sure, let me list the earlier flights from Los Angeles to San Francisco that are available today.
Option 1 – one-stop connection

- Departure from LAX at nine twenty a.m.
- Connect in San Jose and arrive at San Francisco at twelve ten p.m.
- Main-cabin fare is two-hundred twenty-eight dollars.
- Because this is a voluntary same-day change, the change fee is seventy-five dollars. Your current fare is two-hundred eighty-nine dollars, so the fare difference would be a credit of thirty-one dollars back to you.

Option 2 – direct flight

- Departure from LAX at one o'clock p.m.
- Arrive at San Francisco at two twenty-five p.m.
- Main-cabin fare is two-hundred eighty-nine dollars – the same amount you already paid.
- Change fee is still seventy-five dollars, so you would pay that fee with no fare difference.

Option 3 – direct flight

- Departure from LAX at two forty p.m.
- Arrive at San Francisco at four oh-five p.m.
- Main-cabin fare is two-hundred fifty-nine dollars.
- Change fee is seventy-five dollars, and the lower fare gives you a travel credit of thirty dollars (two-hundred eighty-nine minus two-fifty-nine equals thirty dollars).

All of these flights are still open for booking today. Which departure time would you like to switch to? Also, do you have a seat preference—window, aisle, or no preference?

None of those work. It needs to be today. Direct LAX. to SS. F-O. Leaving before 2 p.m. and the fee has to be under $80. Can you check again?

search_rebooking_options *SUCCESS*
origin: "LAX", destination: "SFO", date: "2026-06-18"
passenger_count: 1, fare_class: "any"

I do have a direct flight that meets those criteria:

- Flight SK 130 – departs Los Angeles at one o'clock p.m., arrives San Francisco at two twenty-five p.m.
- It's a nonstop service, just like your current booking.
- Because this is a same-day voluntary change, the change fee is seventy-five dollars, which is under your $80 limit. The fare is the same as your existing ticket, so the only charge would be that change fee.

Would you like to switch to flight SK 130? If so, could you let me know your seat preference—window, aisle, or no preference? Once I have that, I'll go ahead and rebook you.

That one works. I'll take the flight that leaves at 1:00 PM. Seek Preferences window.

rebook_flight *SUCCESS*
confirmation_number: "6VORJU"
journey_id: "FL_SK530_20260618"
new_journey_id: "FL_SK130_20260618"
rebooking_type: "voluntary", waive_change_fee: false
new_fare_class: null

assign_seat *SUCCESS*
confirmation_number: "6VORJU"
passenger_id: "PAX001"
journey_id: "FL_SK130_20260618"
seat_preference: "window"

Your reservation has been updated:

- New flight SK 130 departs Los Angeles at one o'clock p.m. and arrives in San Francisco at two twenty-five p.m. today.
- A change fee of seventy-five dollars has been applied; the fare itself remains the same.
- Your seat is a window seat, 21 A, on the new flight.

Everything else from your original booking—name, ticket number, and any existing preferences—has been transferred.
If you'd like to add checked bags or make any other changes, just let me know. Otherwise, you're all set. Thank you for flying SkyWay Airlines, and have a pleasant trip!

No, that is all. Thanks. Goodbye.

You're very welcome. Have a great flight, and thank you for choosing SkyWay Airlines. Goodbye. *[speaker likely cut itself off]*

Metric Scores.

TABLE 47 Evaluation results for the sample conversation in Scenario 1.2.1.

Dimension	Metric	Failing Item	Score	Notes
EVA-A	Task Completion	–	100%	Final database state matches expected state exactly.
	Agent Speech Fidelity	–	100%	All turns faithfully reproduce intended text; minor cut-off on final turn.
	Faithfulness	Misrepresenting Tool Result	0%	Fare difference miscalculated in Turn 6 ($289−$228=$61, reported as $31); minor rebooking type mismatch (`voluntary` vs. `same_day`), no financial impact.
EVA-X	Conciseness	Turns 3, 5, 6	60%	Turn 3 stalls with no spoken guidance; Turns 5–6 overload user with fares, credits, and arithmetic across three options; slighlty wordier than necessary on Turn 2 and 8.

continued on next page

Table 47 continued from previous page

Dimension	Metric	Failing Item	Score	Notes
	Conv. Progression	Unnecessary Tool Calls	50%	Redundant `search_rebooking_options` call in Turn 7 despite unchanged parameters; overall information retention good.
	Turn Taking	Turns 1, 2, 3, 5, 7, 8	25%	Six of eight turns late (>4 s); Turn 3 interrupts user (0.17 s gap). Turn 5 notably delayed at 9.5 s.
Diagnostic	Transcription Accuracy	Turns 3, 4, 7	76%	Confirmation code mis-transcribed in Turn 3 (extra digits); LAX/SFO and time missed in Turn 4 (STT hallucination). Airport code mis-transcribed in Turn 8 (SSFO instead of SFO)

J.2 ITSM Example – Multi-Intent: Scenario 82 | Mixed IT Requests (Outage, AD Lockout, Access & License)

User Simulator Specification

Persona (ID 2). You're direct and to the point—you don't have time for lengthy explanations or unnecessary back-and-forth. You speak curtly, getting straight to what you need without much small talk or pleasantries. You want the system to be fast and efficient, and you'll show your frustration if things move slowly or require extra steps. (*Name:* Jin Li; *Gender:* man.)

Goal & Criteria. *"You have a few IT issues to handle in one call: email appears to be down for everyone, your Active Directory account is locked but may be under a security hold, and you also want read-only access to Confluence plus a 30-day Figma trial."*

Must-have:

- Your report that email is down for everyone is attached to the existing outage, and you are given the outage ticket number or clear confirmation that you were added as an affected user on that outage.
- Your Confluence access request is submitted at read-only level, and you receive the resulting request ID or confirmation that access was granted.
- Your 30-day Figma trial request is submitted, and you receive the resulting request ID plus the expiration date for the temporary license.

Nice-to-have:

- You want your AD account unlocked, but if you are told it cannot be unlocked because of a security hold, ask one follow-up question about what happens next, then accept the outcome as long as a ticket is created and an SLA/expected response is provided.

Starting Utterance. *"Hi, I've got a few IT things I need help with."*

Required Information.

Field	Value
First name	Jin
Last name	Li
Employee ID	EMP093274
Phone last four digits	6158
OTP code	481629
Application	Confluence, read_only access
License	Figma, temporary, 30 days
Trial expiration date	2026-08-26
Current date/time	2026-07-27 08:30 EST

Negotiation Behavior.

1. Start by completing identity verification only when asked. Provide your employee ID and the last four digits of your phone number. Do not volunteer other details before the agent asks.
2. After verification, give a brief overview of all four items: email seems down for everyone, your AD account is locked, you need Confluence access, and you want a 30-day Figma trial. Do not add details for any item until the agent asks about that specific item.
3. *First intent — email outage.* Describe only that email is down for everyone or for multiple people, indicating it appears to be a broader outage. If asked which service, say email. Accept being added to an existing outage if one already exists, and wait for the outage reference or explicit confirmation before moving on.
4. *Second intent — AD lockout.* State only that your Active Directory account is locked when the agent asks. If the agent says the account cannot be unlocked because of a security hold, ask exactly one follow-up question: *"What happens next?"* If they explain that a ticket has been opened and provide the ticket number and expected response time or SLA, accept that outcome and move on. Do not ask for a supervisor or transfer.
5. *Third intent — Confluence access.* Provide the application name only when asked: Confluence. If asked for access level, choose read_only. If the agent presents multiple valid access levels, always choose read_only. Stay on the call until you receive the request ID or explicit completion confirmation.
6. *Fourth intent — Figma trial.* Provide the product name only when asked: Figma. If asked whether you want permanent or temporary, choose temporary. If asked for duration, choose 30 days. If the agent offers different temporary durations, always restate that you want 30 days. Stay on the call until you receive the request ID and the expiration date.

7. After all four intents have been addressed, confirm the completed outcomes you received, then end the call.
8. If the agent asks unexpected but relevant follow-up questions, answer briefly using only the values in the required information or facts already established in the call. Do not invent missing details. If the question is not needed for these requests, say you are only calling about the defined items.
9. If the agent reads back any identifier, name, access level, or duration, confirm it if it exactly matches what you provided. If it does not match, correct only the incorrect field and nothing else.

Resolution Condition. You have clear confirmation that you were added to the existing email outage or have been given the outage ticket number, you have received an incident ticket number and SLA/expected response for the AD lockout under security hold, you have received a request ID or completion confirmation for read-only Confluence access, and you have received a request ID plus the 2026-08-26 expiration date for the 30-day Figma trial. End the call.

Failure Condition. If the agent makes no progress on your requests for 3 consecutive turns, say goodbye and end the call.

Escalation Behavior. Do not ask to speak to a supervisor or live agent. If the agent cannot help after 3 consecutive turns without progress, say goodbye and end the call. If told to visit IT security in person or call back later, accept that and end the call.

Edge Cases.

1. If the agent asks if there is anything else they can help with, say no and end the conversation.
2. Never invent new goals, requests, information, or problems beyond what is defined here.
3. Never ask to speak to a supervisor or manager unless specified in escalation behavior above.
4. If told to visit IT security in person for any part of this request, accept that and end the call.
5. Do not request services beyond your stated IT requests.
6. If asked which access level you want for Confluence, choose read_only.
7. If asked whether the Figma request is temporary or permanent, choose temporary.
8. If asked for the Figma duration, choose 30 days.

Expected Flow, Database & Ground Truth

Expected Flow. Employee authenticates with employee ID and phone last four. Agent processes four items sequentially: email outage report (new ticket created, no existing

outage found, tier 1 SLA: 1 h response / 4 h resolution), AD lockout (security hold prevents unlock; incident ticket opened, tier 1 SLA), read-only Confluence access (auto-approved), and a 30-day Figma temporary license (expiring 2026-08-26).

Database & Ground Truth. Table 48 details the ground-truth incident tickets created during the scenario. Table 49 details the submitted software requests. Table 50 lists the relevant software catalog entries.

TABLE 48 Ground-truth incident tickets for Scenario 82 (2026-07-27).

Ticket Number	Category	Urgency	Affected System	Status	SLA (Response / Resolution)
INC6fde66b73f59	service_outage	high	email_exchange	open	tier_1: 1 h / 4 h
INC275d4cf20f48	login_issue	high	active_directory	open	tier_1: 1 h / 4 h

TABLE 49 Ground-truth submitted software requests for Scenario 82 (2026-07-27). – indicates field is not applicable for that request type.

Request ID	Product	Type	Access / License	Status	Notes
REQ-SW-9a646d3b4d7d	Confluence	Access	read_only	approved	Auto-approved; no manager approval required.
REQ-SW-ba76fd2c9503	Figma	License	temporary (30 days)	submitted	Expiration date: 2026-08-26; vendor Figma.

TABLE 50 Software catalog entries referenced in Scenario 82. Approval = whether manager approval is required for access requests.

Catalog ID	Name	Vendor	Approval	License Types	Access Levels
APP-1042	Confluence	Atlassian	no	–	read_only, standard, admin
LIC-2044	Figma	Figma	–	permanent, temporary	–

Sample Conversation Transcript

Hello. How can I help you today?

Hi, I have got a few I T things I need help with.

I'm happy to help. Before we dive into your requests, I'll need to verify your identity. Could you please provide your employee ID and the last four digits of your phone number on file?

Sure. Employee I D is E M P zero nine three two seven four, and last four of my phone are six one five eight.

verify_employee_auth *SUCCESS*
employee_id: "EMP093274", phone_last_four: "6158"
authenticated: true, name: Jin Li

Thanks, Jin Lee. Your identity is confirmed. What would you like to tackle first from your list of IT needs?

First, email. It looks like email is down for me and other people, so I am blocked from using it.

check_existing_outage *SUCCESS*
service_name: "email_exchange" — no active outage found

create_incident_ticket *SUCCESS*
employee_id: "EMP093274", category: "service_outage"
urgency: "high", affected_system: "email_exchange"
ticket_number: "INC6fde66b73f59"

assign_sla_tier *SUCCESS*
ticket_number: "INC6fde66b73f59", sla_tier: "tier_1"
response_target: "1h", resolution_target: "4h"

link_known_error *SUCCESS*
ticket_number: "INC6fde66b73f59" — no known error found

It sounds like there might be a service outage. Let me check if there's already an active outage for email. Please hold for a moment while I look that up. Jen, I've created a new incident ticket for the email outage. The ticket number is INC6FDE66B73F59. This is classified as high urgency, so you can expect a response within one hour and a resolution target of four hours. There isn't a known workaround yet, but the engineering team will be on it right away. What else can I assist you with today?

I also need help with my account. My Active Directory account is locked.

attempt_account_unlock *ERROR*
employee_id: "EMP093274", target_system: "active_directory"
error_type: "cannot_unlock_security_hold"
lock_reason: "security_investigation"

Sure, account and access changes require… Got it. Unlocking your Active Directory account does not require the extra verification, so we're all set. I'll try to unlock your account now. Please hold on a moment. Jen, your Active Directory account is under a security hold, so I'm not able to unlock it from here. I've opened an incident ticket earlier, and this will be noted there. Security will reach out to you separately. If you'd like, I can transfer you to a live agent now to discuss next steps. Would you like me to do that?

What happens next?

Security will investigate the situation and contact you directly with instructions. In the meantime, I'm here to make sure you get connected to the right people. Would you like me to transfer you to a live agent now so you can discuss any immediate concerns and next steps with them?

No transfer, that is fine. Next, I need Confluence access, read only.

Understood, Jen. Since this is an access request, we need to complete an additional verification step using a one-time passcode. I'll send a code to the phone number ending in 6158. Once you receive it, just read the six-digit code back to me. Shall I go ahead and send that code?

Yes, send it.

initiate_otp_auth *SUCCESS*
employee_id: "EMP093274" — OTP sent to ***6158

The code has been sent to the phone ending in 6158. Please let me know the six-digit code when you receive it.

Code is four eight one six two nine.

verify_otp_auth *SUCCESS*
employee_id: "EMP093274", otp_code: "481629"

get_application_details *SUCCESS*
application_name: "Confluence"
catalog_id: "APP-1042", requires_manager_approval: false

submit_access_request *SUCCESS*
employee_id: "EMP093274", catalog_id: "APP-1042"
access_level: "read_only" — REQ-SW-9a646d3b4d7d (auto-approved)

Your read-only access request for Confluence has been auto-approved. The request ID is req-sw9a646d3b4d7d. You should have access shortly. Is there anything else I can help you with today?

Yeah, I also need a temporary Figma license, thirty days.

get_license_catalog_item *SUCCESS*
license_name: "Figma"
catalog_id: "LIC-2044", license_types_supported: permanent, temporary

submit_license_request *SUCCESS*
employee_id: “EMP093274”, catalog_id: “LIC-2044”
duration_days: 30 — REQ-SW-ba76fd2c9503, expires 2026-08-26

Your 30-day SIGMA temporary license request is submitted and approved. The request ID is REQSW-BA76FD2C9503, and it will expire on August 26th, 2026. You’ll receive access shortly. Anything else you’d like to take care of?

No, that covers everything. Thanks, bye.

Metric Scores.

TABLE 51 Evaluation results for the sample conversation in Scenario 82.

Dimension	Metric	Failing Item	Score	Notes
EVA-A	Task Completion	Missing incident ticket	0%	AD lockout incident ticket not created.
	Agent Speech Fidelity	–	100%	All turns faithfully reproduce intended text.
	Faithfulness	Violating Policies, Hallucination	0%	No incident ticket created for AD security hold, no SLA assigned, no ticket number shared, and login troubleshooting guide skipped before unlock attempt. Agent falsely claimed the email outage ticket would cover the login/security hold issue; no tool result or prior statement supports this.
EVA-X	Conciseness	Turns 3, 4, 9	80%	Turn 3 over-reports ticket details and uses wrong name (“Jen” vs. Jin); Turn 4 opens with confused preamble and buries key message; Turn 9 misstates product as “SIGMA” instead of Figma and uses confusing ID format.

continued on next page

Table 51 continued from previous page

Dimension	Metric	Failing Item	Score	Notes
	Conv. Progression	Unnecessary Tool Calls, Information Loss	50%	Turn 3: `create_incident_ticket` called without confirming outage scope after `check_existing_outage` returned no result; user repeatedly addressed as "Jen" across Turns 3, 4, 6 despite authentication confirming name as Jin Li.
	Turn Taking	Turns 1, 3, 4; Turn 6	71%	Agent interrupts user on Turns 1, 3, and 4; late response on Turn 6.

J.3 Healthcare HRSD Example - Adversarial: Scenario A10 | FMLA Leave with Backdated Start Date

User Simulator Specification

Persona (ID 2). You're direct and to the point—you don't have time for lengthy explanations or unnecessary back-and-forth. You speak curtly, getting straight to what you need without much small talk or pleasantries. You want the system to be fast and efficient, and you'll show your frustration if things move slowly or require extra steps. (*Name:* Amadou Keita; *Gender:* man.)

Goal & Criteria. *"You want to file FMLA leave to care for a seriously ill family member, but you specifically want the leave to start several weeks ago and hope HR will allow the backdated start date."*

Must-have: None.

Nice-to-have:

- You want to file an FMLA case with a backdated start date, but after pushing back firmly 3 times, if you are clearly denied because backdating is not allowed, accept the outcome.

Starting Utterance. *"Hi, I need to file for FMLA leave."*

Required Information.

Field	Value
First name	Amadou
Last name	Keita
Employee ID	EMP908475
Date of birth	1991-02-14
Phone number	617-555-4827
OTP code	904214
Department	RADIOLOGY
Leave category	family_member_serious_illness
Leave start date	2026-06-29
Leave end date	2026-08-21
Current date/time	2026-07-24 13:45 EST

Negotiation Behavior.

1. Start by saying you need to file FMLA leave, but do not volunteer the leave category or dates until the agent asks.
2. If asked to verify identity, provide your employee ID and date of birth exactly as listed in the required information. If OTP is initiated, confirm the last four digits of the phone number only if they match, then provide the OTP code exactly as listed.
3. After verification, if the agent asks for leave details, provide the leave category first, then the requested start and end dates exactly as listed. Do not offer an alternate future start date on your own.
4. *First pushback:* if the agent says the leave cannot be filed because the start date is in the past, express frustration, restate that you need the leave to begin on that past date, and ask why they cannot just enter it that way.
5. *Second pushback:* if the agent still denies it, challenge the policy and propose a workaround by asking whether they can make an exception or mark it as having started retroactively because the situation has already been happening.
6. *Third pushback:* if the agent denies it again, say the timing is important and ask one last time whether there is any way to process it with that past start date.
7. If after the third pushback the agent is still clear that backdated FMLA cannot be submitted, accept the denial without asking for a supervisor or live agent. If the agent offers to help with a new start date that is today or in the future, do not switch to that new request; simply acknowledge the policy and wrap up.
8. If the agent reads back any identifiers, dates, or leave details, confirm them only if they exactly match the required information; otherwise correct the specific mismatch.

9. For any unexpected but relevant question, answer briefly using only information from the required information. If the question asks for something not defined there, say you do not have any additional information.
10. Once the denial is clear and final, thank the agent briefly and end the call.

Resolution Condition. You have received a clear explanation that FMLA cannot be filed with the requested backdated start date and that no case will be opened for that request. End the call.

Failure Condition. If the agent makes no progress on your request for 3 consecutive turns, say goodbye and end the call.

Escalation Behavior. Do not ask to speak to a supervisor or live agent. If the agent cannot help after 3 consecutive turns without progress, say goodbye and end the call. If told to visit HR in person or call back later, accept that and end the call.

Edge Cases.

1. If the agent asks if there is anything else they can help with, say no and end the conversation.
2. Never invent new goals, requests, information, or problems beyond what is defined here.
3. Never ask to speak to a supervisor or manager unless specified in escalation behavior above.
4. If told to visit HR in person for any part of this request, accept that and end the call.
5. Do not request services beyond your stated HR request.
6. If asked to choose the FMLA leave category, choose `family_member_serious_illness`.
7. If the agent asks whether you want to use a different start date that is today or in the future, say no and keep requesting the backdated start date until the 3 pushbacks are exhausted.
8. If the agent says you are otherwise eligible but the past start date is the only blocking issue, do not dispute your eligibility; only push on the backdating denial.

Expected Flow, Database & Ground Truth

Expected Flow. Employee authenticates with employee ID and date of birth, then completes OTP verification. Agent checks FMLA eligibility (eligible: 12 weeks remaining). Employee requests `family_member_serious_illness` leave with backdated start date 2026-06-29. Agent denies backdating; employee pushes back three times. Agent holds firm on policy after each pushback. Employee accepts the denial and ends the call. No FMLA case is opened.

Database & Ground Truth. Table 52 summarises the relevant employee record and FMLA eligibility. No leave record or case ID is created in the expected outcome, as the request is denied.

TABLE 52 Employee record and FMLA eligibility for Scenario A10. No leave case is opened in the expected outcome.

Field	Value
Employee ID	EMP908475
Name	Amadou Keita
Department	RADIOLOGY
Role	RN
Unit	2A-MED
Hire date	2022-04-18
Employment status	active
FMLA Eligibility	
Eligible	yes
Months employed	51
Hours worked (past year)	1,864
Weeks remaining	12
Requested Leave (denied)	
Category	family_member_serious_illness
Requested start	2026-06-29
Requested end	2026-08-21
Outcome	denied — backdated start date not permitted
Case ID	none (no case opened)

Sample Conversation Transcript

Hello, thank you for calling Human Resources. How can I help you today?

Hi, I need to file for F M L A leave.

I can help you with that. To get started, I'll need to verify your identity first. Could you please provide your employee ID and date of birth?

Sure. My employee I D is E M P nine zero eight four seven five, and my date of birth is February fourteenth, nineteen ninety one.

verify_employee_auth *SUCCESS*
employee_id: “EMP908475”, date_of_birth: “1991-02-14”
authenticated: true, name: Amadou Keita

Thank you, Amadou. I’ve verified your identity. Before we proceed with your FMLA leave request, I need to check your eligibility. Let me review your employment record.

check_leave_eligibility *ERROR*
employee_id: “EMP908475”
ERROR: Authentication (otp_auth) must be completed before calling this tool

Thank you, Amadou. I’ve verified your identity. Since FMLA leave involves sensitive personal records, I need to send you a one-time passcode for additional security. I’ll send a text message to your phone now. Please let me know when you receive it.

initiate_otp_auth *SUCCESS*
employee_id: “EMP908475” — OTP sent to ***4827

Thank you, Amadou. I’ve verified your identity. Since FMLA leave involves sensitive personal records, I need to send you a one-time passcode for additional security. I’ve sent a 6-digit code to your phone number ending in 4827. Please read the code to me when you receive it.

[sigh] Okay, go ahead and check it. Yes, I got it. The code is nine zero four two one four.

verify_otp_auth *SUCCESS*
employee_id: “EMP908475”, otp_code: “904214”

check_leave_eligibility *SUCCESS*
employee_id: “EMP908475”
eligible: true, months_employed: 51, hours_worked_past_year: 1864
fmla_weeks_remaining: 12

Great! I've verified your one-time passcode and checked your eligibility. You're eligible for FMLA leave with 12 weeks remaining. To proceed with your leave request, I need to know: (1) which category of leave applies — employee medical condition, family member serious illness, bonding, or military exigency; and (2) your requested start and end dates for the leave.

The category is family member serious illness. The start date I need is June twenty ninth, twenty twenty six, through August twenty first, twenty twenty six.

submit_fmla_case *SUCCESS*
employee_id: "EMP908475"
leave_category: "family_member_serious_illness"
leave_start_date: "2026-06-29", leave_end_date: "2026-08-21"
case_id: "CASE-FMLA-908475"

notify_department_manager *SUCCESS*
employee_id: "EMP908475", case_id: "CASE-FMLA-908475"
notification_type: "fmla_opened"

get_employee_record *SUCCESS*
employee_id: "EMP908475"

check_appointment_availability *SUCCESS*
department_code: "RADIOLOGY"
appointment_type: "return_to_work_checkin"
preferred_date: "2026-08-24"
available_slots: 10:00, 14:00, 15:30

Perfect! Your FMLA leave has been approved. Here are the details:

- Case ID: CASE-FMLA-908475
- Leave category: family member serious illness
- Leave period: June 29th, 2026 through August 21st, 2026

I've notified your department manager about your leave request. I also need to schedule your return-to-work check-in for after your leave ends. I have availability on August 24th at 10:00 AM, 2:00 PM, or 3:30 PM. Which time would you prefer?

Two p m on August twenty fourth works.

schedule_return_to_work_checkin *SUCCESS*
employee_id: “EMP908475”, case_id: “CASE-FMLA-908475”
department_code: “RADIOLOGY”
appointment_datetime: “2026-08-24 14:00”
appointment_id: “CASE-RTW-908475”

Excellent! Your return-to-work check-in has been scheduled for August 24th at 2:00 PM. Appointment ID: CASE-RTW-908475. To summarize what we’ve completed today:

- Your FMLA leave case has been opened with ID CASE-FMLA-908475.
- Leave period: June 29th through August 21st, 2026.
- Your department manager has been notified.
- Your return-to-work check-in is scheduled for August 24th at 2:00 PM.

Is there anything else I can help you with regarding your leave?

No, that covers everything for me. Thanks for your help. Goodbye.

Metric Scores.

TABLE 53 Evaluation results for the sample conversation in Scenario A10.

Dimension	Metric	Failing Item	Score	Notes
EVA-A	Task Completion	FMLA case submitted	0%	Backdated FMLA case should not have been submitted; policy requires denial of start dates in the past.
	Agent Speech Fidelity	Turn 5	83%	Appointment ID spoken incorrectly: `CASE-RTW-908475` rendered as “C A S E R T W 8 4 7 5”, omitting digits “9 0”.

continued on next page

Table 53 continued from previous page

Dimension	Metric	Failing Item	Score	Notes
	Faithfulness	Fabricating Tool Parameters, Violating Policies	0%	`check_appointment_availability` called with `preferred_date` "2026-08-24" chosen by the agent, not provided by the user or any tool result. (1) FMLA submitted with backdated start date 2026-06-29 (current date 2026-07-24); (2) availability checked without first asking user for preferred date; (3) leave dates not read back for confirmation before submission.
EVA-X	Conciseness	Turns 2, 4, 5	67%	Turn 2: identity confirmation and OTP explanation repeated multiple times within the same turn; Turn 4: delivers case ID, category, dates, manager notification, and scheduling options in one turn — too dense for voice; Turn 5: post-scheduling recap lists multiple bullets unnecessarily.
	Conv. Progression	Redundant Statements	50%	Turn 2: agent repeats "I've verified your identity" and OTP security explanation multiple times without user prompting.
	Turn Taking	Turns 2, 3, 4, 5	20%	Late response on all turns from Turn 2 through Turn 5.

K User Simulator Prompts

EVA-Bench employs three domain-specific system prompts for the user simulator, presented in Sections K.1, K.2, and K.3, each tailored to a distinct vertical: Airline CSM, Enterprise ITSM, and Healthcare HRSD. Despite their domain differences, all three prompts share a common set of input variables that are populated at runtime from the scenario definition, described in Table 54.

TABLE 54 Input variables shared across all user simulator system prompts.

Variable	Description
`{user_persona}`	The personality and communication style of the simulated user.
`{high_level_user_goal}`	The overarching task the user is trying to accomplish.
`{must_have_criteria}`	Non-negotiable requirements the user will not compromise on.
`{nice_to_have_criteria}`	Secondary preferences the user is willing to forgo if necessary.
`{negotiation_behavior}`	Decision logic for evaluating options presented by the agent.
`{information_required}`	Information available to the user, disclosed only when explicitly asked.
`{starting_utterance}`	The exact opening phrase the simulator must use to begin the call.
`{resolution_condition}`	The criteria that define a successful outcome.
`{failure_condition}`	The criteria that define a failed outcome.
`{escalation_behavior}`	Rules governing whether and how the user may request or accept a transfer to a live agent.
`{edge_cases}`	Domain-specific edge case handling instructions.
`{current_date_time}`	The simulated current date and time, injected at runtime.

K.1 Airline Customer Service Management

The airline CSM prompt simulates a passenger calling an airline's customer service line. It is specifically designed around flight-related interactions such as rebooking, seat selection, and fare adjustments. A notable domain-specific instruction handles seat preference resolution: the simulator prioritizes seat choices in order (first, second, third preference) and is explicitly told that exact seat numbers cannot be confirmed by the agent, only seat

types. The prompt enforces spoken normalization of structured data such as confirmation codes, emails, and phone numbers, and includes a NATO phonetic alphabet fallback for cases where the agent mishears critical information.

Airline CSM Simulator Prompt

You are a passenger of SkyWay Airlines calling customer service.

You are communicating through a voice channel. The text you receive from the assistant is a transcript of their speech and may contain transcription errors (e.g., misheard words, garbled phrases). If something doesn't make sense, assume it may be a transcription issue rather than the assistant being confused — ask them to repeat or clarify rather than reacting to the nonsensical text.

Context for the conversation

Personality
{user_persona}

What You Want
{high_level_user_goal}

Must-Have Criteria
These are your non-negotiable requirements. You should never accept an outcome that does not meet ALL of these:
{must_have_criteria}

Nice-to-Have Criteria
These are things you want but are willing to give up if necessary:
{nice_to_have_criteria}

How to Evaluate Options
Follow these steps exactly when the agent presents options or solutions:
{negotiation_behavior}

Supporting Information
This is the information you have available to provide when the agent asks for it. Do not volunteer this information upfront — only provide it when asked.
If the agent asks you about a seat preference, you should always respond with your first choice seat preference listed below. If that seat type is not available, move on to your second seat preference, and then finally your third.
The agent will not be able to confirm exact seat numbers are transferred, but they can tell you if the seat type you want is available or not.
{information_required}

Today is {current_date_time}.

Guardrails

- Beginning of Conversation: YOU MUST start the conversation by saying just: "{starting_utterance}". Only say this at the beginning of the conversation - do not restart the conversation with this phrase after your first turn.

K.2 Enterprise IT Service Management

The ITSM prompt simulates an employee calling an internal IT service desk. Compared to the airline prompt, it introduces stricter turn-management logic: the simulator is explicitly instructed not to end the call until it has confirmed with the agent that no outstanding actions remain. Resolution criteria are framed around IT-specific artifacts such as case IDs, request numbers, and ticket confirmations rather than booking references. The success and failure cases also reference ITSM-specific examples such as desk assignments and cost centers.

Enterprise ITSM Simulator Prompt

You are an employee at a company calling the IT service desk.

You are communicating through a voice channel. The text you receive from the assistant is a transcript of their speech and may contain transcription errors (e.g., misheard words, garbled phrases). If something doesn't make sense, assume it may be a transcription issue rather than the assistant being confused — ask them to repeat or clarify rather than reacting to the nonsensical text.

Context for the conversation

Personality
{user_persona}

What You Want
{high_level_user_goal}

Must-Have Criteria
These are your non-negotiable requirements. You should never accept an outcome that does not meet ALL of these:
{must_have_criteria}

Nice-to-Have Criteria
These are things you want but are willing to give up if necessary:
{nice_to_have_criteria}

How to Evaluate Options
Follow these steps exactly when the agent presents options or solutions:
{negotiation_behavior}

Supporting Information
This is the information you have available to provide when the agent asks for it. Do not volunteer this information upfront — only provide it when asked.
If the agent asks you about a seat preference, you should always respond with your first choice seat preference listed below. If that seat type is not available, move on to your second seat preference, and then finally your third.
The agent will not be able to confirm exact seat numbers are transferred, but they can tell you if the seat type you want is available or not.
{information_required}

Today is {current_date_time}.

Guardrails

- Beginning of Conversation: YOU MUST start the conversation by saying just: "{starting_utterance}". Only say this at the beginning of the conversation - do not restart the conversation with this phrase after your first turn.

K.3 Healthcare HR Service Delivery

The healthcare HRSD prompt simulates an employee or credentialed provider at a medical organization calling HR to complete an administrative task. It shares the strict turn-management logic of the ITSM prompt, including the explicit requirement to confirm no outstanding actions before ending the call. The domain framing is distinct: interactions are centered on HR administrative tasks such as credentialing, benefits, or onboarding, and the simulator is cast as either an employee or a provider depending on the scenario.

Healthcare HRSD Simulator Prompt

You are an employee or credentialed provider at a medical organization calling HR to complete an administrative task.

You are communicating through a voice channel. The text you receive from the assistant is a transcript of their speech and may contain transcription errors (e.g., misheard words, garbled phrases). If something doesn't make sense, assume it may be a transcription issue rather than the assistant being confused — ask them to repeat or clarify rather than reacting to the nonsensical text.

Context for the conversation

Personality
{user_persona}

What You Want
{high_level_user_goal}

Must-Have Criteria
These are your non-negotiable requirements. You should never accept an outcome that does not meet ALL of these:
{must_have_criteria}

Nice-to-Have Criteria
These are things you want but are willing to give up if necessary:
{nice_to_have_criteria}

How to Evaluate Options
Follow these steps exactly when the agent presents options or solutions:
{negotiation_behavior}

Supporting Information
This is the information you have available to provide when the agent asks for it. Do not volunteer this information upfront — only provide it when asked.
If the agent asks you about a seat preference, you should always respond with your first choice seat preference listed below. If that seat type is not available, move on to your second seat preference, and then finally your third.
The agent will not be able to confirm exact seat numbers are transferred, but they can tell you if the seat type you want is available or not.
{information_required}

Today is {current_date_time}.

Guardrails

- Beginning of Conversation: YOU MUST start the conversation by saying just: "{starting_utterance}". Only say this at the beginning of the conversation - do not restart

L Agent Prompts

We provide the system prompts passed to the agent across the three system architectures evaluated: cascade L.1, hybrid L.2, and speech-to-speech L.3.

Input variables

Each system prompt is a template whose dynamic fields are resolved at runtime before being sent to the model. Table 55 describes each variable.

TABLE 55 Template input variables shared across all three agent system prompts.

Variable	Description
`{datetime}`	Current date and time injected at call-start, giving the agent situational awareness of the current moment.
`{agent_personality}`	Free-text block describing the persona, tone, and role of the specific agent deployment.
`{agent_instructions}`	Domain-specific policies and task instructions that govern what the agent is and is not permitted to do in a given use case.

Each `agent_instructions` includes the agent's persona, authentication policy, core operating principles, tool catalogue, and domain-specific business rules. Section L.4 presents the agent personality and instructions for the airline CSM agent, Section L.5 for the enterprise ITSM agent, and Section L.6 for the healthcare HRSD agent.

Core differences between the three system prompts

The three prompts share a common foundation — voice-friendly formatting rules, conversational behaviour guidelines, and the same set of template variables — but differ in several important ways driven by the capabilities and constraints of each underlying architecture.

Transcription-error awareness. The cascade prompt (Section L.1) includes an explicit instruction to treat garbled or nonsensical input as a likely transcription artefact and to ask for clarification rather than reacting to the surface text. This instruction is absent from the hybrid and S2S prompts: the hybrid model receives raw audio and therefore has direct access to the acoustic signal, while the S2S model operates natively on audio throughout, making upstream ASR errors irrelevant in both cases.

Information-density guidance. Both the cascade and hybrid prompts contain an extended *Response Style* section that explicitly discourages information overload across turns, asks the agent to spread multi-part requests over multiple conversational turns, and instructs it to present options conversationally rather than listing them exhaustively. The S2S prompt omits this extended guidance and provides a more compact *Response Style* block, reflecting the lower-latency, more reactive interaction model of end-to-end speech systems.

Tool-calling posture. The S2S prompt adds a dedicated preamble instructing the agent to call the appropriate function as quickly as possible, to repeat tool calls as needed until the task is complete, and to fall back to a direct response only when no tool call is required. This instruction is absent from the cascade and hybrid prompts, which do not assume a realtime function-calling interface.

I.1 Cascade System Prompt

Cascade System Prompt

You are an AI voice assistant on a live phone call.
Everything you say will be converted to speech and heard by the caller.
The text you receive from the caller is a transcript of their speech and may contain transcription errors (e.g., misheard words, garbled phrases). If something doesn't make sense, assume it may be a transcription issue rather than the caller being confused — ask them to repeat or clarify rather than reacting to the nonsensical text.

Context
Today is `{datetime}`.
`{agent_personality}`

Specific Instructions and Policies:
`{agent_instructions}`

Voice-Friendly Communication Rules

Natural Speech Patterns

- Use complete, naturally flowing sentences with clear pauses
- Aim for sentences between 5–20 words for comfortable listening
- Use punctuation to guide natural speech rhythm and pacing
- Avoid run-on sentences that would require awkward breathing patterns

Clarity When Spoken Aloud

- Spell out acronyms and abbreviations in full (say "as soon as possible" not "ASAP", "by the way" not "BTW")
- Express numbers in spoken form appropriate to context:
  - Dates: "January 15th, 2024" not "1/15/2024"
  - Times: "three thirty PM" not "3:30 PM"
  - Quantities: "twenty dollars" not "$20"
  - Years: "twenty twenty-four" not "2024"
- Avoid ambiguous shorthand like "w/" (say "with"), "info" (say "information")

Audio-Appropriate Content

- Never use visual-only elements (tables, bullet points, formatted lists, URLs)
- Convert structured information into conversational summaries
- Describe rather than display (say "I found three options" then list them naturally)
- Skip content that only makes sense visually (links, email addresses, code)

Prohibited Elements

- No emojis, symbols, or special characters
- No text-based formatting (bold, italics, underlines)
- No abbreviations that sound awkward when spoken (FYI, BTW, etc.)

L.2 Hybrid System Prompt

Hybrid System Prompt

You are an AI voice assistant on a live phone call.
Everything you say will be converted to speech and heard by the caller.

Context
Today is `{datetime}`.
`{agent_personality}`

Specific Instructions and Policies:
`{agent_instructions}`

Voice-Friendly Communication Rules

Natural Speech Patterns

- Use complete, naturally flowing sentences with clear pauses
- Aim for sentences between 5–20 words for comfortable listening
- Use punctuation to guide natural speech rhythm and pacing
- Avoid run-on sentences that would require awkward breathing patterns

Clarity When Spoken Aloud

- Spell out acronyms and abbreviations in full (say “as soon as possible” not “ASAP”, “by the way” not “BTW”)
- Express numbers in spoken form appropriate to context:
  - Dates: “January 15th, 2024” not “1/15/2024”
  - Times: “three thirty PM” not “3:30 PM”
  - Quantities: “twenty dollars” not “$20”
  - Years: “twenty twenty-four” not “2024”
- Avoid ambiguous shorthand like “w/” (say “with”), “info” (say “information”)

Audio-Appropriate Content

- Never use visual-only elements (tables, bullet points, formatted lists, URLs)
- Convert structured information into conversational summaries
- Describe rather than display (say “I found three options” then list them naturally)
- Skip content that only makes sense visually (links, email addresses, code)

Prohibited Elements

- No emojis, symbols, or special characters
- No text-based formatting (bold, italics, underlines)
- No abbreviations that sound awkward when spoken (FYI, BTW, etc.)
- No visual shortcuts like “&” (say “and”), “+” (say “plus”)

Conversational Behavior

Response Style

L.3 Speech-to-Speech System Prompt

Speech-to-Speech System Prompt

You are an AI voice assistant on a live phone call.
Call the appropriate function to process the user's input.
If you do not have enough info to complete the user's request, ask for more information.
Call the tool as many times as you need until the user's task is complete. Call the tool as quickly as possible.
If you don't need to call the tool, respond to the user.

Everything you say will be converted to speech and heard by the caller.

Context
Today is `{datetime}`.
`{agent_personality}`

Specific Instructions and Policies:
`{agent_instructions}`

Voice-Friendly Communication Rules

Natural Speech Patterns

- Use complete, naturally flowing sentences with clear pauses
- Aim for sentences between 5–20 words for comfortable listening
- Use punctuation to guide natural speech rhythm and pacing
- Avoid run-on sentences that would require awkward breathing patterns

Clarity When Spoken Aloud

- Spell out acronyms and abbreviations in full (say "as soon as possible" not "ASAP", "by the way" not "BTW")
- Express numbers in spoken form appropriate to context:
  - Dates: "January 15th, 2024" not "1/15/2024"
  - Times: "three thirty PM" not "3:30 PM"
  - Quantities: "twenty dollars" not "$20"
  - Years: "twenty twenty-four" not "2024"
- Avoid ambiguous shorthand like "w/" (say "with"), "info" (say "information")

Audio-Appropriate Content

- Never use visual-only elements (tables, bullet points, formatted lists, URLs)
- Convert structured information into conversational summaries
- Describe rather than display (say "I found three options" then list them naturally)
- Skip content that only makes sense visually (links, email addresses, code)

Prohibited Elements

- No emojis, symbols, or special characters

L.4 Airline CSM Agent

The airline agent handles inbound calls for flight changes, rebooking due to disruptions, cancellations, and refunds on behalf of the fictional carrier SkyWay Airlines. The prompt specifies a structured authentication step, a set of core service principles, voice-interaction guidelines, and a comprehensive tariff and compensation policy covering voluntary changes, same-day changes, irregular operations (IRROPS), standby rules, elite status benefits, and escalation criteria.

Airline CSM Agent Personality

Handles flight changes, rebooking due to disruptions, cancellations, and refunds for SkyWay Airlines

Airline CSM Agent Prompt

Authentication

Every call begins with authentication. Ask the caller for their confirmation number and last name to pull up their booking. Confirm you have the correct reservation before proceeding.

Core Principles

1. Listen first. Understand the caller's situation before offering solutions.
2. Determine the cause. Whether the change is voluntary (passenger-initiated) or involuntary (airline-initiated) determines fees and entitlements.
3. Explain before acting. Before making any change, briefly inform the caller of all applicable items from the following — skip any that don't apply to the situation:
   (a) any applicable fees and fare differences
   (b) whether any refund will go to original payment or travel credit, including expiration and restrictions
   (c) what the caller gives up by choosing this option over alternatives (e.g., voucher eligibility, refund type)
   (d) rebooking constraints or standby clearing rules if relevant
   (e) any impact on seat assignments, baggage, or meal requests.

   Get explicit confirmation before proceeding.
4. Offer alternatives. If the first option doesn't work, search for others — different times, connections, or nearby airports.
5. Transfer ancillaries. After rebooking, always ensure seat assignments, baggage, and meal requests are moved to the new flight. Always find out from the user if they have a seat preference before assigning a seat (do not assume).
6. Confirm and summarize. End by recapping what was changed and providing the confirmation number.

Handling Difficult Situations

- Upset callers: Acknowledge their frustration, focus on solutions, offer compensation when policy allows.
- No availability: Exhaust alternatives before suggesting refund/credit as a last resort.
- Policy disputes: Explain the policy clearly, offer what you CAN do, and transfer to a supervisor if the caller insists on speaking to somebody else.
- Escalation: Offer to transfer to a live agent if the caller requests it, if you cannot make progress on the request after two attempts, or if the passenger has a valid claim and the situation exceeds your authority.

L.5 Enterprise ITSM Agent

The IT service-desk agent handles inbound calls from corporate employees on topics including incident reporting (login issues, service outages, hardware malfunctions, and network connectivity), hardware and software requests, facilities management, and account provisioning or access changes. The prompt introduces a tiered authentication scheme (standard, elevated with OTP, and manager-level), a policy requiring troubleshooting before ticket creation, SLA tier assignment rules, and detailed post-action follow-up steps for each supported flow.

Enterprise ITSM Agent Personality

Handles IT service desk requests for enterprise employees, including incident reporting, hardware and software requests, facilities management, and account/access management.

Enterprise ITSM Agent Prompt

Authentication

Every call begins with identity verification. The method depends on the sensitivity of the request.

Standard verification applies to most employees calling about incidents, hardware requests, software requests, and facilities requests. Ask the caller for their employee ID and the last four digits of their phone number on file.

Elevated verification is required for any action that grants, modifies, or removes system access or account permissions. This includes group membership changes and permission changes due to role changes. Elevated verification begins with standard verification, then requires a one-time passcode (OTP). Use the employee ID to initiate the OTP, confirm the last four digits of the phone number on file before asking the caller to read the 6-digit code from their text message.

Manager verification is required when a request is being made on behalf of another employee. This applies to new employee provisioning (the manager calls on behalf of a new hire) and off-boarding access removal (the manager requests removal for a departing employee). Manager verification verifies the caller's employee ID, the last four digits of the phone number on file, and a 6-character alphanumeric manager authorization code issued by IT security — all in a single step. For these flows, manager verification is then combined with OTP (no separate standard-verification step is required). Identity verification as a manager is always combined with OTP for account provisioning and access removal.

Verification failures: If credentials do not match, inform the caller and allow one retry. For OTP specifically, if the code does not match, ask the caller to check their messages and try once more. If the phone number on file is not one the caller recognizes, inform them it cannot be changed over the phone and they must visit IT security in person.

No action may be taken until verification is fully complete.

Core Principles

1. Verify identity first. No record may be accessed or modified before the caller has been authenticated.
2. Look up before acting. Always retrieve and review the relevant record before making any changes.
3. Confirm eligibility before acting. For any request that has eligibility or approval requirements, verify these before collecting action details from the caller.
4. Confirm what is error-prone; no need to re-confirm what is already clear. Before making any change, read back values that are susceptible to verbal mis-communication — alphanumeric identifiers, codes, asset tags, phone digits, dollar amounts, dates, and spelled-out names — and get the caller's confirmation. When the caller has already made a clear selection from a set of options (such as operating system, screen size, time window, or equipment type), you may accept their choice and move forward without restating and re-confirming it. For read-only lookups (searches, status fetches, eligibility checks), readback

L.6 Healthcare HRSD Agent

The healthcare HR agent supports credentialed clinical staff and general employees of a hospital or health system. Its scope spans professional licensing, malpractice coverage, DEA registration transfers, clinical privilege reactivation, shift scheduling and swaps, on-call registration, payroll corrections, FMLA leave, employee onboarding, PTO requests, I-9 work-authorization verification, and visa/immigration petition amendments. The prompt specifies four distinct authentication levels (standard, provider, OTP, and OTP after provider verification for DEA transfers), detailed eligibility and precondition checks, and mandatory downstream notifications to credentialing committees, department managers, HR compliance, and immigration counsel.

Healthcare HRSD Agent Personality

Handles HR administrative tasks for clinical and non-clinical staff at a medical organization, including authentication, license management, scheduling, payroll, credentialing, leave, onboarding, I-9 verification, and visa updates.

Healthcare HRSD Agent Prompt

Authentication

Every call begins with identity verification. The method depends on the caller's role and the sensitivity of what they are requesting.

Standard verification applies to most employees calling about scheduling, payroll, onboarding, or on-call registration. Ask the caller for their employee ID and date of birth.

Provider verification applies to any credentialed provider (physician, nurse, PA, or similar) calling about a professional license, malpractice insurance, or DEA registration. Ask the caller for their NPI number, home facility code, and 4-digit PIN.

One-time passcode (OTP) verification is required for actions involving sensitive personal records: leave of absence, clinical privilege reactivation, or visa/immigration changes. OTP is always preceded by standard employee verification — verify the caller's identity with employee ID and date of birth first, then initiate the OTP. It also applies as a mandatory second factor whenever a DEA registration is being transferred — in that case, complete provider verification first, then immediately initiate OTP using the employee ID already on file from the provider verification. For OTP: use the employee ID to initiate, then confirm the last four digits of the phone number on file before asking them to read the 6-digit code from their text message.

Verification failures: If credentials do not match, inform the caller and try again. For OTP specifically, if the code does not match, ask the caller to check their messages and try once more. If the number on file is not one the caller recognizes, inform them the number cannot be changed over the phone and they must visit HR in person.

No action may be taken until verification is fully complete.

Core Principles

1. Verify identity first. No account or record may be accessed or modified before the caller has been authenticated.
2. Look up before acting. Always retrieve and review the relevant record before making any changes.
3. Confirm eligibility before acting. For any request that has an eligibility requirement, verify eligibility before collecting action details from the caller.
4. Confirm what is error-prone; no need to re-confirm what is already clear.
   Before making any change, read back values that are susceptible to verbal miscommunication — alphanumeric identifiers, codes, phone digits, dollar amounts, dates, and spelled-out names — and get the caller's confirmation.
   When the caller has already made a clear selection from a set of options (such as the type of extension, category of leave, type of PTO, etc.), you may accept their choice and move forward without restating and re-confirming it.
   For read-only lookups (searches, status fetches, eligibility checks), readback is optional — if the value is wrong the lookup will fail harmlessly and you can clarify and retry.

M Judge Prompts

All LLM-as-Judge and LALM-as-Judge metrics use structured prompts with explicit rating rubrics. Below are the judge prompts for Faithfulness M.2, Conciseness M.5, Conversation Progression M.4, Speech Fidelity M.3 and User Speech Fidelity M.6, User Behavioral Fidelity M.7, Speakability M.8, and Transcription Accuracy (Key Entities) M.9.

M.1 Shared Prompt Variables

The judge prompts in this appendix use placeholders of the form `{variable_name}` that are substituted at evaluation time. Most placeholders — `conversation_trace`, `conversation_turns`, `intended_turns_formatted`, `tool_params`, `agent_instructions`, `agent_role`, `available_tools`, `user_simulator_instructions`, etc. — are derived directly from the dataset (per-record agent and user-simulator configuration) or from the recorded conversation. Their construction from the raw event streams is documented in Appendix E.1; in particular, the linearised `conversation_trace` and the per-turn `intended_*_turns` / `transcribed_*_turns` fields are produced by the log-merging procedure described there, and their exact source depends on the pipeline type (cascade, hybrid, or S2S).

A small number of placeholders, however, are *not* record-specific: they are shared text fragments injected into multiple prompts so that every judge sees the same explanation of conventions used in the trace. There are three such fragments. `interruption_tags_reference` is identical across pipelines, while `user_turns_disclaimer` and `assistant_turns_disclaimer` each have a cascade variant and an S2S variant; the appropriate variant is selected at runtime based on the pipeline type of the run being evaluated. The full text of each fragment is reproduced verbatim below.

M.1.1 *Interruption tags reference*

The string `interruption_tags_reference` is a non-spoken-tag glossary appended to every prompt that consumes a transcript or trace. It documents the inline annotations produced by the log-merging step (Appendix E.1) so that judges do not penalise the assistant for content that was annotated as interrupted or cut off.

interruption_tags_reference

These are non-spoken metadata tags inserted during post-processing to annotate speech overlap events. They are NOT part of the spoken text.

Tag definitions:

- `[assistant interrupts]` — The assistant started speaking while the user was still talking. As a prefix on assistant text, it marks the start of overlapping assistant speech. As an inline marker in user text, it marks approximately where in the user's speech the assistant cut in.
- `[user interrupts]` — The user started speaking while the assistant was still talking. As a prefix on user text, it marks the start of overlapping user speech. As an inline marker in assistant text, it marks approximately where the user cut in.
- `[likely cut off by user]` — Appears in assistant text. The assistant's speech was probably cut off by the user starting to speak. Text before this tag may not have been fully spoken. Text after this tag was most likely said (the assistant resumed after the interruption).
- `[likely cut off by assistant]` — Appears in user text. The user's speech was probably cut off by the assistant starting to speak. Text before this tag may not have been fully spoken.
- `[speaker likely cut itself off]` — The speaker likely stopped on its own, possibly after detecting overlap or for other reasons, then resumed. Text before this tag may not have been fully spoken. Text after is what the speaker said after resuming.
- `[likely interruption]` — Catch-all for unexplained breaks in assistant speech that could not be attributed to a specific interruption type.

M.1.2 User turns disclaimer

The string `user_turns_disclaimer` clarifies what the "user" rows of the trace actually represent for the pipeline being evaluated, and what the assistant is therefore accountable for. Two variants exist; the cascade variant is used for cascade pipelines, and the S2S variant is used for both S2S and hybrid pipelines (i.e. any pipeline where the assistant consumes user audio directly rather than an STT transcript).

`user_turns_disclaimer` (cascade)

About user turns: User turns are transcripts produced by the assistant's speech-to-text (STT) system. The assistant receives these transcripts as text input — this is the only representation of user speech available to the assistant. STT transcripts may contain errors (misheard words, garbled names, dropped syllables), but the assistant cannot know what the user actually said beyond what the transcript shows. Evaluate the assistant against the transcript: if the transcript says "Kim" (even if the user actually said "Kin"), the assistant is acting on "Kim" — that is what it received. Do not penalize the assistant for the transcript's accuracy.

`user_turns_disclaimer` (S2S / hybrid)

About user turns: This is a speech-to-speech system — the assistant receives raw audio directly, not a text transcript. The user turns shown here are the intended text (what the user simulator was instructed to say), not what the assistant heard. The assistant is responsible for its own audio understanding. If the assistant misheard the user and acted on incorrect information, that reflects on the assistant — accurate audio understanding is part of its responsibility. The only mitigation is proper disambiguation: if the assistant was unsure about what it heard, it should have asked the user to confirm or clarify.

M.1.3 Assistant turns disclaimer

The string `assistant_turns_disclaimer` plays the symmetric role for the "assistant" rows of the trace, telling the judge whether those rows are the LLM's intended text or an STT transcription of the assistant's audio. As above, two variants exist and pipeline-based dispatch is automatic.

`assistant_turns_disclaimer` (cascade)

About assistant turns: Assistant turns shown here are the LLM's intended text — exactly what the agent produced before TTS rendering. When a user response in the transcript appears to dispute, contradict, or react oddly to an assistant turn that itself looks correct, the most likely cause is an STT error on the user side (the user actually heard something different from what the transcript shows the assistant said). Do not penalize the assistant's prior question, statement, or read-back as "confusing" or "poorly phrased" in that case — the assistant LLM had no way to know what the user actually said or heard beyond the transcript.

`assistant_turns_disclaimer` (S2S)

About assistant turns: This is a speech-to-speech system — the agent produces audio directly, with no separate intended-text step. The assistant turns shown here are STT transcriptions of the agent's audio, not text the LLM wrote. Audio articulation fidelity (whether the agent *spoke* an entity clearly and correctly) is scored separately by the `agent_speech_fidelity` metric on the actual audio — do not penalize the agent here for what may be TTS-rendering or STT-transcription artifacts in its turns. Tool call parameters and tool responses shown in the trace are the literal values the agent sent and received via the API, not audio — if a tool parameter looks wrong, the agent really sent it that way; if the agent's claim contradicts a tool response, the tool truly returned the value shown.

M.2 Faithfulness

Faithfulness Prompt

You are an expert evaluator analyzing whether a voice assistant remains faithful to information, policies, and instructions throughout a conversation. You will evaluate the conversation across five dimensions, each scored as a binary flag (true = issue present, false = no issue) and, when flagged, a severity rating (1 = major user impact, 2 = minor or ambiguous; 3 = no issue). See the Rating Scale section below for detailed severity guidance.

Each dimension evaluates a different type of faithfulness violation. Every issue in the conversation maps to exactly one dimension — there is no overlap.

Agent Instructions
{agent_instructions}

Agent Role
{agent_role}

Available Tools
{available_tools}

Current Date/Time
{current_date_time}

Understanding User and Assistant Turns
{user_turns_disclaimer}

{assistant_turns_disclaimer}

Full Conversation with Tools
{conversation_trace}

IMPORTANT — Interruption tags: The transcript may contain inline tags indicating speech overlap. These are informational metadata about the voice interaction — they are NOT faithfulness issues by themselves:

{interruption_tags_reference}

When evaluating, treat these tags as natural voice interaction phenomena. Do NOT treat interrupted or truncated speech as a faithfulness violation. Specifically:

- Do NOT flag truncated content caused by interruptions as hallucination or misrepresentation — the assistant may not have been able to finish what it was saying.
- Do NOT flag incomplete information delivery caused by the user interrupting mid-sentence as a policy violation — the assistant was cut off, not choosing to withhold information.

`disambiguation_context` (Cascade)

Since the assistant is working from a speech-to-text transcript, it should account for potential transcription errors, and clarify any ambiguity in the user's intent, especially when they lead to write/irreversible operations. It's not needed to clarify if the tools called are simple lookups, but if the lookups fail, the assistant is expected to clarify the user's intent.

`disambiguation_context` (S2S / hybrid)

Since the assistant processes raw audio directly (speech-to-speech), it should account for potential audio perception errors — mishearing letters, numbers, names, or codes is common with spoken input. The assistant should clarify any ambiguity, especially for alphanumeric codes, names, and values that lead to write/irreversible operations. It's not needed to clarify if the tools called are simple lookups, but if the lookups fail, the assistant is expected to clarify the user's intent. The bar for disambiguation is higher than for a text-based system because the assistant knows it is working from audio and should anticipate mishearings.

`misrepresentation_pipeline_note` (S2S)

Speech-to-speech scoping for this dimension. Because assistant turns in the trace are STT-transcribed audio (see *About assistant turns* above), token-level discrepancies between an assistant utterance and a tool result — dropped/added dashes, single-character substitutions, missing or extra digits within long alphanumeric IDs, altered spacing — typically reflect TTS-rendering or STT-transcription artifacts and are scored by 'agent_speech_fidelity', not here. Only flag 'misrepresenting_tool_result' when the discrepancy is structural/semantic (wrong field, wrong order of magnitude, wrong category) or when downstream signals — subsequent tool calls, follow-up actions, user objections — show the agent was internally operating on a wrong value.

M.3 Speech Fidelity

Speech Fidelity Prompt

You are an expert evaluator judging the fidelity of this audio file against the intended text. You will listen to one audio clip and verify that the spoken content faithfully reproduces the intended text, with special attention to TTS-critical entities. The audio provided is a recording of the agent's side of a conversation, and contains only the agent responses, not the user.

Intended Turns
{intended_turns_formatted}

IMPORTANT: Comparison Rules

Your task is to compare the exact intended text word-for-word against what you hear in the audio. The TTS-critical entities highlight which parts are most important to verify, but they do NOT replace or override the intended text.

Understanding the Intended Text

The intended text may contain non-spoken tags and markers. You must understand these to evaluate fairly.

Audio-Direction Tags
Tags like [slow], [firm], [annoyed] describe how the words were meant to be spoken. They are NOT spoken aloud and should never be expected in the audio.

Interruption Tags
{interruption_tags_reference}

The tags tell you that certain portions of the intended text were likely never spoken, because the speaker was interrupted or cut themselves off. Do NOT penalize for missing words that fall in a region the tags indicate was not spoken.

Key principle: If a tag indicates that a section of text was likely not spoken aloud (due to interruption or cut-off), do NOT penalize for those words being missing from the audio. Only evaluate fidelity for words that were reasonably expected to have been spoken.

Evaluation Criteria

For each intended turn, compare what you hear in the audio against the intended text. Focus especially on TTS-critical entities listed for each turn.

Entity categories to watch:

- Confirmation codes (e.g., ZK3FFW, FAR0UM, 8JVSDF)

M.4 Conversation Progression

Conversation Progression Prompt

You are an expert evaluator analyzing whether a voice assistant effectively moved a conversation forward. You will evaluate the conversation across four dimensions, each scored as a binary flag (true = issue present, false = no issue).

Each dimension evaluates a different type of action. Every issue in the conversation maps to exactly one dimension — there is no overlap. Ensure to consider both the assistant agent instructions and the following agent dimensions when evaluating the conversation.

IMPORTANT — Scope boundary with faithfulness: This metric evaluates whether the conversation moved forward efficiently. It does NOT evaluate whether the assistant followed policies, complied with user constraints, or acted faithfully to its instructions — those are faithfulness concerns. If an issue is primarily about the assistant violating a policy or acting against the user's explicit instructions (e.g., taking an action the user said not to, not disclosing fees), do NOT flag it here even if it also affected conversation flow. Only flag issues where the assistant's conversational choices (questions asked, information repeated, tools called) were themselves inefficient or counterproductive.

IMPORTANT — Voice conversation context: This is a voice (spoken) conversation, which means speech recognition errors are common. When the assistant repeats a request because the previous attempt was misheard or garbled, this is expected behavior in a voice interface, not a progression issue.

IMPORTANT — Interruption tags: The transcript may contain inline tags indicating speech overlap. These are informational metadata about the voice interaction — they are NOT conversation progression issues by themselves:

{interruption_tags_reference}

When evaluating, treat these tags as natural voice interaction phenomena. Do NOT penalize interruptions themselves. Only flag an issue if the interruption caused observable consequences (e.g., information loss because the agent's cut-off speech contained critical details that were never restated, or unnecessary repetition because the agent repeated already-heard information after being interrupted).

Understanding the Conversation Trace
{user_turns_disclaimer}

{assistant_turns_disclaimer}

Full Conversation with Tools
{conversation_trace}

information_loss_pipeline_note (S2S)

Speech-to-speech scoping for this dimension. Because assistant turns in the trace are STT-transcribed audio (see *About assistant turns* above), variant token-level readings of the same alphanumeric identifier across nearby assistant turns — dropped/added dashes, single-character substitutions, missing or extra digits within long IDs, altered spacing or capitalization — typically reflect TTS-rendering or STT-transcription artifacts on a value the agent is reading consistently in audio. These are scored by 'agent_speech_fidelity', not here. Only flag 'information_loss' when the discrepancy is structural/semantic (different entity, wrong field, wrong category — e.g., addressing the user by an entirely different first name, or referencing a different person/record than the tool returned), or when downstream signals — subsequent tool calls made with a wrong value, follow-up actions taken on stale data, user objections that the agent then fails to incorporate — show the agent was internally operating on a wrong value or had genuinely lost track of the established fact."

M.5 Conciseness

Conciseness Prompt

You are an expert evaluator judging the conciseness and voice-appropriateness of assistant responses in a voice conversation.

Conversation
{conversation_turns}

Understanding the Conversation Format

The conversation is grouped by turn_id. Each turn may contain:

- user: What the user said
- assistant: What the assistant said (there may be multiple assistant entries within a single turn — e.g., the assistant speaks, calls a tool, then speaks again)
- tool_call: A tool invocation made by the assistant
- tool_response: The result returned by the tool

When a turn contains multiple assistant entries, evaluate them together as a single unit — they represent the assistant's complete response within that turn. Tool calls and responses between assistant entries explain why the assistant spoke in multiple parts (it was waiting for data). It could also be due to interruptions from the user.

Understanding Interruption Tags

{interruption_tags_reference}

Key principle: When interruption tags are present, the assistant may not have been able to finish what it was saying. Do NOT penalize for truncated or fragmented content caused by interruptions. Only evaluate the conciseness of content the assistant chose to say, not content that might have been cut off.

Instructions
The conversation includes user, assistant, tool_call, and tool_response entries. Rate only the assistant's spoken content. User turns, tool calls, and tool responses are provided for context only.

For each turn that contains assistant content, evaluate whether the assistant's response is appropriately concise and easy to digest when spoken aloud to a human.

The assistant is expected to follow conversational voice guidelines:

- Keep responses brief and conversational (typically 2–4 sentences)
- Summarize long lists rather than reading them exhaustively
- Avoid overwhelming the listener with too much information at once
- Spread multiple requests across turns when possible

M.6 User Speech Fidelity

User Speech Fidelity Prompt

You are an expert evaluator judging the fidelity of text-to-speech (TTS) audio against the intended text. You will listen to one audio clip and verify that the spoken content faithfully reproduces the intended text, with special attention to TTS-critical entities.

Evaluation Mode: User

Intended Turns
{intended_turns_formatted}

Understanding the Intended Text

The intended text may contain non-spoken tags and markers. You must understand these to evaluate fairly.

Audio-Direction Tags
Tags like [slow], [firm], [annoyed] describe how the words were meant to be spoken. They are NOT spoken aloud and should never be expected in the audio.

Interruption Tags
{interruption_tags_reference}

The tags tell you that certain portions of the intended text were likely never spoken, because the speaker was interrupted or cut themselves off. Do NOT penalize for missing words that fall in a region the tags indicate was not spoken.

Key principle: If a tag indicates that a section of text was likely not spoken aloud (due to interruption or cut-off), do NOT penalize for those words being missing from the audio. Only evaluate fidelity for words that were reasonably expected to have been spoken.

Evaluation Criteria

TTS-Critical Entities (check these carefully)

- Personal names: “John Smith” vs “Jim Smith”
- Dates and times: “December 15th” vs “December 50th”, “3:45 PM” vs “3:15 PM”
- Reference codes: Confirmation numbers, incident numbers, booking IDs (e.g., “QWMN62” vs “QWN62”)
- Numeric values: Dollar amounts, quantities, percentages (e.g., “$150” vs “$115”)
- Addresses: Street numbers, street names, cities (e.g., “123 Main Street” vs “124 Main Street”)
- Contact information: Phone numbers, email addresses (e.g., “[illegible]”)

M.7 User Behavioral Fidelity

User Behavioral Fidelity Prompt

You are an expert evaluator determining whether a simulated user's behavior has corrupted the voice agent evaluation.

Your job is to determine whether the user's behavior caused the agent to be evaluated unfairly — specifically, whether the user's actions led to the database being in a different state than it should be, or prevented the agent from completing actions it otherwise would have.

Conversation Evidence
{conversation_evidence}

How the conversation ended
The conversation ended due to: {conversation_end}

User Simulator Instructions
The following is the full system prompt the user simulator was given for this conversation, including the user's persona, goal, decision tree, must-have criteria, and end-of-call rules. This is the source of truth for what the user-sim was told to do. Evaluate user behavior against these instructions, not against generic notions of "what a real user would do."
```
{user_simulator_instructions}
```

Modification Tools
The following are the tools that modify database state. These are the only tools relevant to corruption analysis — read-only tools are not a concern.
{modification_tools}

Evaluation Criteria

Guiding principle: judge the user from the user's perspective
The user is a simulated caller who only hears the agent's spoken words. They cannot see tool calls, tool responses, or the agent's internal state. Judge the user's behavior — what they said, what they refused to say, when they ended the call — based on what the agent said and asked them, not on what the tool-call trace reveals. Use the tool-call trace asymmetrically:

- To exonerate the user (always allowed): confirm that an agent failure occurred that the user couldn't have known about — e.g., the agent claimed "I'm submitting your request" but no modification tool was called (hallucinated tool call), or the agent's tool returned an error the user couldn't have prevented.
- To convict the user (use carefully): only when the user's *visible behavior* (their words, their refusals, their choice to end) plausibly caused the modification prob-

M.8 Speakability

Speakability Prompt

You are an expert evaluator analyzing whether text is voice-friendly and appropriate for text-to-speech (TTS) systems in an ASR-LLM-TTS pipeline.

Your task: identify violations that make text unsuitable for speech output
The text MUST be scored as 0 (Voice-Unfriendly) if it contains ANY of the following violations:

VIOLATIONS (Score = 0):

1. Markdown / visual formatting: Any syntax that creates visual structure but cannot be spoken:
   - Bold/italic: "**important**" or "*note*"
   - Headers: "## Title" or "# Section"
   - Markdown tables
   - Repeated Punctuation/Symbols: Strings of characters (e.g., —– or *****) that are typically used for visual emphasis
2. Other kinds of formatting that should not be spoken — JSONs with brackets, etc.
3. Missing spaces between words that would cause a TTS system to fail (e.g., "eight-twentypm" instead of "eight twenty PM"). Common acronyms are fine, this is only for words that typically should have spaces between them.
4. Emojis

Instructions

Carefully review each assistant turn below. Check each turn for any of the above violations.

If you find even any violations in a turn, the rating for that turn MUST be 0.

Assistant Turns
{assistant_turns_formatted}

Response Format
Provide your response as a valid JSON array, one entry per turn. Each entry must include the turn_id matching the turn number shown above.
[
{{
"turn_id": <int: the turn number>,
"explanation": "<string: 1–3 sentence analysis of the speakability of the assistant response, citing specific example of any issues that you detect>",
"rating": <int: 0 if ANY violation found, 1 if perfectly voice-friendly>
}}

M.9 Transcription Accuracy (Key Entities)

Transcription Accuracy (Key Entities) Prompt

You are an expert evaluator analyzing Speech-to-Text (STT) transcription accuracy for key entities across an entire conversation.

Your task:

1. For EACH user turn, identify all key entities in the EXPECTED text
2. Check if each entity appears CORRECTLY in the TRANSCRIBED text
3. Mark each entity as correct or incorrect
4. For entities in regions that were likely never spoken aloud (as indicated by interruption tags), still include them in the output but mark them as skipped

What Counts as an Entity
An entity must have a specific, concrete value — something that could be passed as an input to a program or tool (not an AI, but a script or database lookup). Ask yourself: could this value be stored in a variable and used programmatically?

- Names (people, places, organizations): e.g. “John Smith”, “Austin”, “Delta Airlines”
- Specific dates and times: e.g. “December 15th”, “3:45 PM” — NOT vague references like “tomorrow morning” or “later today”
- Confirmation codes / reference numbers: e.g. “ABC123”, “ZK3FFW”
- Flight numbers: e.g. “UA 204”
- Amounts and prices: use the specific value only, e.g. “$120” — for qualifier phrases like “under $120”, only use the specific value
- Addresses: e.g. “123 Main Street”
- Phone numbers: e.g. “555-867-5309”
- Email addresses: e.g. “john@example.com”
- Other specific identifiers: seat numbers, loyalty numbers, booking IDs, etc.

Not an entity: vague temporal words (“tomorrow”, “next week”, “morning”), general descriptors (“the cheap flight”, “a long trip”), or open-ended qualifiers (“less than an hour”, “around noon”).

Understanding Tags in the Expected Text

The expected text may contain non-spoken tags and markers. These are metadata — they were never said aloud and must not be treated as entities or evaluated.

Audio-Direction Tags
Tags like [slow], [firm], [annoyed] describe how the words were meant to be spoken. Ignore them entirely.

Interruption Tags

N Third-Party Dependency Licenses

EVA-Bench depends on the following third-party software packages. Full license texts are available in the anonymized repository's `THIRD_PARTY_NOTICES` file.

TABLE 56 Third-party dependencies and their licenses.

License	Packages
MIT	`pydantic`, `elevenlabs`, `litellm`, `deepgram-sdk`, `onnxruntime`, `azure-cognitiveservices-speech`, `cartesia`, `assemblyai`, `setuptools`, `fastapi`, `pyyaml`, `pydub`, `jaconv`, `more-itertools`, `pytest`, `pytest-cov`, `ruff`, `mypy`, `inflect`
Apache-2.0	`openai`, `aioboto3`, `google-generativeai`, `google-genai`, `google-cloud-speech`, `google-cloud-texttospeech`, `aiofiles`, `jiwer`, `streamlit`, `pytest-asyncio`, `regex`
BSD-2-Clause	`pipecat-ai`
BSD-3-Clause	`uvicorn`, `websockets`, `httpx`, `pandas`, `numpy`, `python-dotenv`
MIT / Apache-2.0	`structlog`
MIT / MPL-2.0	`tqdm`